

Multiagentní systémy

Učební text ke státní závěrečné zkoušce

SZZ Umělá inteligence, MFF UK

červenec 2026

Text pokrývá okruh „Multiagentní systémy“ magisterské SZZ oboru Umělá inteligence. Vychází z přednášky NAIL106 *Multiagentní systémy* (MFF UK) a z učebnic M. Wooldridge: *An Introduction to MultiAgent Systems* (2. vyd., 2009), Y. Shoham, K. Leyton-Brown: *Multiagent Systems* (2009) a S. Russell, P. Norvig: *Artificial Intelligence: A Modern Approach*. Anglické termíny jsou při prvním výskytu uvedeny kurzívou v závorce, protože zkoušející je běžně používá a literatura je výhradně anglická.

Mapování okruhu na kapitoly

Položka oficiálního okruhu	Kapitola v tomto textu
Architektura autonomního agenta; mechanismus výběru akcí	1 Úvod, 2 Architektura autonomního agenta
Metody pro řízení agentů; symbolické reaktivní plánování	3 Deliberativní architektury, 4.1–4.3
Konekcionistické reaktivní plánování	4.4 Agent network architecture (Maes)
Hybridní přístupy	4.6 Hybridní architektury, 4.7 IDA
Problém hledání cesty	5 Hledání cesty (A*, D* Lite, MAPF)
Komunikace a znalosti v MAS, ontologie	6 Znalosti a ontologie
Řečové akty, FIPA-ACL, protokoly	7 Komunikace agentů
Distribuované řešení problémů, kooperace	8 Distribuované řešení problémů (CNET, BBS, DCOP)
Nashova ekvilibria, Paretova efektivita	9 Interakce agentů a teorie her (vč. koaličních her)
(sociální volba jako doplněk k agregaci preferencí)	10 Sociální volba a volby
Alokace zdrojů, aukce	11 Alokace zdrojů, aukce, vyjednávání (+ návrh mechanismů)
Metody pro učení agentů; zpětnovazební učení	12 Učení agentů
Metodologie návrhu, jazyky a prostředí MAS	13 Metodologie, jazyky a platformy

Obsah

Mapování okruhu na kapitoly	1
1 Úvod: agent a multiagentní systém	4
1.1 Motivace	4
1.2 Co je agent	4
1.3 Vlastnosti inteligentního agenta	5
1.4 Intencionální systémy a intencionální postoj	5
1.5 Shrnutí ke zkoušce	5
2 Architektura autonomního agenta a mechanismus výběru akcí	6
2.1 Prostředí a jeho vlastnosti	6
2.2 Abstraktní architektura agenta	6
2.3 Čistě reaktivní agent a agent s vnitřním stavem	7
2.4 Jak agentovi říct, co má dělat: utilita	8

2.5	Predikátová specifikace úlohy	9
2.6	Mechanismus výběru akcí — přehled	9
2.7	Shrnutí ke zkoušce	10
3	Deliberativní architektury: symbolické uvažování, BDI, PRS	10
3.1	Klasický přístup UI a deduktivní agent	10
3.2	Praktické uvažování: architektura BDI	11
3.3	Plánování: STRIPS	12
3.4	Implementace BDI agenta	13
3.5	Odhodlání agenta (<i>commitment</i>)	14
3.6	Procedurální odvozovací systém (PRS)	14
3.7	Shrnutí ke zkoušce	15
4	Reaktivní a hybridní architektury	16
4.1	Reaktivní přístup a jeho východiska	16
4.2	Symbolické reaktivní plánování: subsumpční architektura	16
4.3	Konekcionalistické reaktivní plánování: agent network architecture	17
4.4	Omezení reaktivních architektur	18
4.5	Hybridní architektury	18
4.6	Komplexní architektury: IDA a globální pracovní prostor	20
4.7	Srovnání architektur	20
4.8	Shrnutí ke zkoušce	20
5	Problém hledání cesty	21
5.1	Formulace a zařazení	21
5.2	A* a příbuzné algoritmy	21
5.3	Hledání cesty v dynamickém prostředí	22
5.4	Hledání cest pro více agentů (MAPF)	23
5.5	Shrnutí ke zkoušce	24
6	Znalosti a ontologie	25
6.1	Od agenta k multiagentnímu systému	25
6.2	Uvažování o znalostech: epistemická logika a společná znalost	25
6.3	Co je ontologie	26
6.4	Ontologické jazyky	26
6.5	Shrnutí ke zkoušce	28
7	Komunikace agentů: řečové akty, ACL, protokoly	28
7.1	Proč je komunikace agentů jiná	28
7.2	Teorie řečových aktů	28
7.3	KQML a FIPA-ACL	29
7.4	Interakční protokoly	30
7.5	Shrnutí ke zkoušce	31
8	Distribučované řešení problémů a kooperace	31
8.1	Koherence a koordinace	31
8.2	Sdílení úloh a sdílení výsledků	32
8.3	Contract Net	32
8.4	Systémy s tabulí (blackboard)	33
8.5	Distribučované omezující podmínky (DisCSP a DCOP)	33
8.6	Společné záměry a sociální zákony	34
8.7	Interakce agentů v prostředí	34
8.8	Shrnutí ke zkoušce	34

9 Interakce agentů a teorie her	35
9.1 Sobecký agent	35
9.2 Hra v normální formě	35
9.3 Klasické hry	36
9.4 Kooperativní (koaliční) hry	38
9.5 Shrnutí ke zkoušce	39
10 Sociální volba a volby	40
10.1 Agregace preferencí	40
10.2 Volební systémy	40
10.3 Řešený příklad: jeden preferenční plán, čtyři vítězové	41
10.4 Vlastnosti volebních systémů	42
10.5 Shrnutí ke zkoušce	42
11 Alokace zdrojů, aukce a vyjednávání	43
11.1 Aukce jako mechanismus alokace zdrojů	43
11.2 Návrh mechanismů (<i>mechanism design</i>)	43
11.3 Čtyři základní typy aukcí	44
11.4 Kombinatorické aukce a VCG	45
11.5 Další typy aukcí a alokace zdrojů	46
11.6 Vyjednávání	46
11.7 Shrnutí ke zkoušce	47
12 Učení agentů a zpětnovazební učení	47
12.1 Proč a jak se agenti učí	47
12.2 Markovský rozhodovací proces	48
12.3 Q-learning a SARSA	49
12.4 Metody založené na politice	50
12.5 Multiagentní zpětnovazební učení	51
12.6 Shrnutí ke zkoušce	52
13 Metodologie návrhu, jazyky a prostředí MAS	52
13.1 Agentově orientované softwarové inženýrství	52
13.2 Standard FIPA a architektura platformy	53
13.3 Jazyky a platformy	53
13.4 Shrnutí ke zkoušce	54
Závěrem: jak na zkoušku	55

1 Úvod: agent a multiagentní systém

1.1 Motivace

Multiagentní systémy (*multiagent systems*, MAS) vznikly jako samostatná oblast informatiky v 90. letech jako odpověď na souběžné trendy: *ubikvitnost* výpočetní techniky, *konektivitu a distribuovanost* (výpočet jako interakce), rostoucí *složitost* úloh, *delegaci* úloh na software a tlak na snadnost použití. Klasické paradigma „program dostane vstup, spočítá výstup, skončí“ přestává stačit ve chvíli, kdy je software trvale zasazen do prostředí, které se mění nezávisle na něm a kde vedle něj běží další nezávislé komponenty.

MAS lze chápat ze čtyř různých pohledů, což vysvětluje širší okruh:

- **Softwarové inženýrství.** Agent je dalším stupněm abstrakce v řadě strojový kód → strukturované programy → objekty → distribuované objekty (CORBA) → agenti. Klasický bonmot: „objekty to dělají zadarmo, agenti proto, že chtějí“.
- **Distribuované a paralelní systémy.** Desítky let výzkumu vyřešily mutex, deadlock či konsensus. Rozdíl je v tom, že agenti jsou *autonomní*, koordinační mechanismy nejsou dané dopředu a vše se musí vyřešit až za běhu.
- **Umělá inteligence.** Tradičně: MAS je podoblast UI. Podle Russella a Norviga je naopak cílem UI návrh inteligentních agentů (tedy $UI \subseteq MAS$); Etzionioho vyhrocené „MAS je 1 % UI a 99 % informatiky“. MAS používá techniky UI (reprezentace znalostí, plánování), ale zdůrazňuje *sociální* aspekty přehlížené klasickou UI.
- **Teorie her a ekonomie.** Formální aparát pro racionální rozhodování více subjektů. Matematická teorie her řeší hlavně otázky *existence*, MAS naopak *výpočetní* aspekty (složitost, praktická použitelnost).
- **Sociologie.** Klíčovým pojmem MAS jsou *společnosti agentů*, a sociologie studuje lidské společnosti — inspirace se tedy nabízí. Platí zde ale stejná výhrada jako u vztahu UI a lidské inteligence: inspirace ano, nutnost ne (letadla nemávají křídly). Opačným směrem je vazba pevnější: sociologie, ekonomie i ekologie používají agenty jako stavební kámen svých simulačních modelů — *agentové modelování* (*agent-based modelling*, ABM), viz platformy v kap. 13.

1.2 Co je agent

Jednotná definice agenta neexistuje — Franklin a Graesser (1996) v článku *Is it an agent, or just a program?* shromáždili desítky vzájemně nekompatibilních definic. Klíčové je odlišit agenta od běžné procedury či objektu. Nejčastěji citované formulace:

Agent — pracovní definice

Russell, Norvig (AIMA): Agent je cokoli, co lze chápat jako entitu *vnímající* své prostředí prostřednictvím senzorů a *působící* na toto prostředí pomocí efektorů.

Pattie Maes: Autonomní agenti jsou výpočetní systémy, které obývají nějaké složité dynamické prostředí, autonomně v něm vnímají a jednají, a tím naplňují množinu cílů či úkolů, pro které byly navrženy.

Wooldridge, Jennings: Agent je počítačový systém zasazený do nějakého prostředí, schopný v tomto prostředí *autonomního jednání* vedoucího ke splnění delegovaných cílů.

Multiagentní systém

Multiagentní systém je systém složený z více agentů, kteří spolu *interagují* — nejčastěji výměnou zpráv po počítačové síti. Agenti *nemusí sdílet společný cíl*; mohou mít vlastní, i protichůdné zájmy. Proto je nutné studovat mechanismy vyjednávání, koordinace a dynamického rozdělování zdrojů.

Autonomie. Člověk je plně autonomní, metoda v Javě vůbec. Agent leží mezi: měl by autonomně

volit *způsob*, jak zadaný problém vyřešit (tedy volbu podcílů), nikoli však hlavní cíl — ten je mu delegován.

1.3 Vlastnosti inteligentního agenta

- **Reaktivita** (*reactive*) — vnímá prostředí a je schopen reagovat na jeho změny v rozumném (reálném) čase.
- **Proaktivita** (*proactive*) — má vlastní cíle a aktivně je naplňuje, z vlastní iniciativy, ne jen jako odpověď na podnět.
- **Sociálnost** (*social ability*) — umí komunikovat s jinými agenty a s lidmi.

Dosáhnout *čistě* reaktivního nebo *čistě* cílově orientovaného chování je poměrně snadné. Obtížné je najít **rovnováhu mezi reaktivitou a proaktivitou**: agent nesmí slepě následovat plán, který se stal nesmyslným, ale ani nesmí neustále přehodnocovat cíle a nikam nedojít. Tento kompromis se v různých podobách vrací v celém textu (viz zejména commitment strategie v kap. 3 a hybridní architektury v kap. 4).

1.4 Intencionální systémy a intencionální postoj

Když o agentech mluvíme, přisuzujeme jim *mentální stavy*: „agent věří, že...“, „agent chce...“. Daniel Dennett definuje **intencionální systém** jako entitu, jejíž chování lze predikovat přisouzením vlastností jako přesvědčení (*beliefs*), touhy (*desires*) a racionalita. Intencionální systémy tvoří hierarchii: systém *prvního řádu* má přesvědčení a touhy, systém *druhého řádu* má přesvědčení a touhy o přesvědčeních a touhách (svých i cizích), atd.

Dennett rozlišuje tři *postoje* (*stances*) k predikci chování systému: *fyzikální* (stačí fyzikální zákony — hodím-li kamenem, nemusím mluvit o jeho touhách, stačí hmotnost, rychlost a gravitace), *návrhový* (*design stance*: neznám všechny fyzikální zákony, ale vím, k jakému účelu byl systém navržen — umím nastavit budík, aniž bych rozuměl jeho elektronice) a *intencionální* (popisují systém pomocí přesvědčení, tužeb a racionality).

Shoham uvádí provokativní příklad: „Vypínač v místnosti je velmi kooperativní agent, který je z vlastní vůle ochoten vést elektrický proud, ale činí tak jen tehdy, věří-li, že si to skutečně přejeme.“ Popis je konzistentní a elegantní, přesto většině lidí připadá absurdní — *protože pro vypínač máme jednodušší a stejně přesný popis*. Intencionální postoj se vyplatí tam, kde fyzikální či návrhový popis neexistuje, nebo je nepraktický (i se schématem počítače je těžké odvodit, proč se po kliknutí na ikonu otevře okno). **Z pohledu informatiky je intencionální postoj především abstrakce pro zvládnutí složitosti** — a právě to je pro mnoho programátorů hlavní přínos MAS.

Zbývá filozofická otázka, kterou Dennett klade: existuje intencionalita entit *sama o sobě*, nebo vzniká teprve *v procesu interpretace* pozorovatelem? Dennett se kloní k druhé možnosti — žádná intencionalita bez interpretace. Pro nás to má praktický důsledek: přisouzení přesvědčení a tužeb agentovi je *naš popis*, ne vlastnost implementace, a je legitimní přesně do té míry, do jaké nám pomáhá predikovat chování.

1.5 Shrnutí ke zkoušce

- Agent = autonomní systém zasazený do prostředí, který v něm vnímá a jedná, aby splnil delegované cíle. MAS = systém interagujících agentů *bez nutně sdíleného cíle*.
- Klíčový problém návrhu agenta je **výběr akce** (*action selection*); architektura agenta je softwarová architektura, která tento výběr realizuje.
- Tři vlastnosti inteligentního agenta: reaktivita, proaktivita, sociálnost; těžké je jejich vyvážení.
- Dennett: intencionální systém a tři postoje (fyzikální, návrhový, intencionální); intencionální postoj je abstrakce pro zvládnutí složitosti.

- Odlišení MAS od: distribuovaných systémů (autonomie, koordinace až za běhu), objektů (agent akci provede, jen když chce), klasické UI (důraz na sociální aspekty), teorie her (důraz na výpočetní aspekty).

2 Architektura autonomního agenta a mechanismus výběru akcí

2.1 Prostředí a jeho vlastnosti

Chování agenta zásadně určuje typ prostředí, ve kterém se nachází. Standardní klasifikace (Russell, Norvig):

Dichotomie	Význam
plně / částečně pozorovatelné (<i>fully / partially observable</i>)	Senzory agenta mu dávají úplný stav prostředí — nebo jen jeho část (skryté informace, šum, omezený dosah).
statické / dynamické (<i>static / dynamic</i>)	Prostředí se mění pouze v důsledku akcí agenta — nebo i jinak (jiní agenti, přírodní děje). Ve statickém prostředí si agent může „v klidu rozmyslet“ další tah.
deterministické / nedeterministické	Každá akce má právě jeden možný výsledek — nebo více (se známým či neznámým rozdělením pravděpodobnosti; pak mluvíme o stochastickém prostředí).
diskrétní / spojitě	Konečný počet stavů a akcí — nebo spojitě veličiny (poloha, rychlost).

Další běžně uváděné dichotomie: *epizodické / sekvenční* (má dnešní akce vliv na budoucí odměny?), *jednoagentní / multiagentní*, *známé / neznámé* (zná agent dopředu přechodovou funkci?).

Nepříjemné konstatování: většina zajímavých prostředí (reálný svět, internet) je spojitá, nedeterministická, dynamická a jen částečně pozorovatelná. Právě proto nestačí naprogramovat agenta jako pevnou tabulku pravidel.

Příklad — termostat. Ne příliš inteligentní agent, zasazený do prostředí (místnost). Dva vjemy („je zima“, „teplota v pořádku“), dvě akce (zapnout, vypnout), rozhodovací jednotka: zima $\Rightarrow$ zapni, OK $\Rightarrow$ vypni. Prostředí je přesto nedeterministické (někdo otevře dveře).

Příklad — softwarový démon. Např. unixový `xbiff`: prostředím je operační systém, vnímá voláním systémových služeb, akcemi jsou softwarové operace (spustit poštovního klienta, změnit ikonu). Rozhodovací algoritmus je na úrovni termostatu.

2.2 Abstraktní architektura agenta

Nyní formalizujeme pohled na agenta a jeho interakci s prostředím. Předpokládáme, že prostředí je diskrétní — což není omezující, každé spojitě prostředí lze diskretizovat s libovolnou přesností.

Prostředí, agent, běh

Prostředí může být v jednom z konečně mnoha diskrétních stavů, $E = \{e, e', \dots\}$. **Agent** má k dispozici konečnou množinu akcí $A = \{a, a', \dots\}$.

Běh (*run*) agenta v prostředí je posloupnost střídajících se stavů prostředí a akcí:

$$r = e_0 \xrightarrow{a_0} e_1 \xrightarrow{a_1} e_2 \xrightarrow{a_2} \dots \xrightarrow{a_{n-1}} e_n.$$

Označme R množinu všech konečných běhů (nad E a A), $R^A \subseteq R$ ty, které končí akcí, a $R^E \subseteq R$ ty, které končí stavem prostředí.

Přechodová funkce prostředí (*state transformer function*)

$$\tau : R^A \rightarrow 2^E$$

Funkce τ zobrazuje běh končící akcí na množinu stavů, do nichž se prostředí může po této akci dostat. Z této definice plynou dva důsledky:

- **Prostředí má historii** — následující stav nezávisí jen na poslední akci, ale na celé historii běhu.
- **Prostředí je nedeterministické** — výsledkem je množina stavů, dopředu nevíme, který nastane.

Je-li $\tau(r) = \emptyset$, běh skončil. Dále předpokládáme, že všechny běhy skončí.

Prostředí je trojice $Env = \langle E, e_0, \tau \rangle$, kde e_0 je počáteční stav.

Agent a systém

Agent je funkce zobrazující běhy končící stavem prostředí na akce:

$$Ag : R^E \rightarrow A.$$

Agent tedy rozhoduje na základě *celé historie* systému a rozhoduje se *deterministicky*. Označme $\mathcal{AG}$ množinu všech agentů.

Systém je dvojice $\langle Ag, Env \rangle$. Posloupnost $(e_0, a_0, e_1, a_1, \dots)$ je *běh agenta Ag v prostředí Env* , právě když

1. e_0 je počáteční stav Env ,
2. $a_0 = Ag(e_0)$,
3. pro každé $u > 0$ platí $e_u \in \tau((e_0, a_0, \dots, a_{u-1}))$ a $a_u = Ag((e_0, a_0, \dots, e_u))$.

Množinu všech běhů agenta Ag v Env značíme $R(Ag, Env)$.

Funkční ekvivalence agentů

Agenti Ag_1 a Ag_2 jsou **funkčně ekvivalentní vzhledem k Env** , právě když $R(Ag_1, Env) = R(Ag_2, Env)$.

Jsou **prostě funkčně ekvivalentní** (*simply functionally equivalent*), pokud jsou funkčně ekvivalentní vzhledem ke *všem* prostředím.

Funkční ekvivalence je důležitá jako nástroj pro srovnávání *vyjadřovací síly* architektúr: říká, že vnitřní implementace je nepodstatná, pokud vede ke stejným pozorovatelným běhům.

2.3 Čistě reaktivní agent a agent s vnitřním stavem

Čistě reaktivní agent (*purely reactive / tropistic agent*)

$$Ag : E \rightarrow A$$

Reaguje pouze na aktuální stav prostředí, historii ignoruje.

Vztah k obecnému agentovi: ke každému čistě reaktivnímu agentovi existuje funkčně ekvivalentní standardní agent; opačně to *neplatí*.

Termostat: $Ag(e) = \text{off}$ pro $e = \text{„teplota OK“}$, jinak $Ag(e) = \text{on}$.

Agent s vnitřním stavem

Nechť I je množina vnitřních stavů agenta a Per množina jeho vjemů. Agent je určen trojicí

funkcí:

$see : E \rightarrow Per$	(vnímání)
$next : I \times Per \rightarrow I$	(aktualizace vnitřního stavu)
$action : I \rightarrow A$	(výběr akce)

Cyklus: agent startuje ve stavu i_0 ; vnímá prostředí ($see(e)$); aktualizuje stav ($i \leftarrow next(i, see(e))$); vybere akci ($a \leftarrow action(i)$); vykoná ji a opakuje.

Věta (bez důkazu): Ke každému agentovi s vnitřním stavem existuje funkčně ekvivalentní standardní agent. Vnitřní stav tedy *nezvyšuje vyjadřovací sílu*, ale je zásadní pro *efektivitu* a *přehlednost* implementace.

Funkce *see* současně formalizuje **pozorovatelnost**: pokud *see* zobrazí dva různé stavy $e_1 \neq e_2$ na tentýž vjem, agent je nedokáže rozlišit — prostředí je pro něj jen částečně pozorovatelné. Krajní případy: $|Per| = |E|$ a *see* prostá (plně pozorovatelné) versus $|Per| = 1$ (agent nevidí nic).

2.4 Jak agentovi říct, co má dělat: utilita

Nechceme agenty s natvrdo zadrátovaným chováním. Chceme agentovi říct, *co* má dělat, nikoli *jak*. Standardní postup UI je definovat cíl *nepřímo*, mírou úspěšnosti.

Utilita

Utilita stavu: $u : E \rightarrow \mathbb{R}$. Cílem agenta je dostat se do stavů s vysokou hodnotou utility. Hodnocení jednotlivých stavů je krátkozraké. Proto zavádíme **utilitu běhu**:

$$u : R \rightarrow \mathbb{R}.$$

Ta lépe odpovídá agentům, kteří běží dlouhodobě. Utilitu běhu lze odvodit ze stavů např. jako utilitu *nejhoršího* navštíveného stavu nebo jako *průměrnou* utilitu navštívených stavů — co je lepší, závisí na úloze.

Optimální agent

Nechť $P(r \mid Ag, Env)$ je pravděpodobnost běhu r agenta Ag v prostředí Env , přičemž $\sum_{r \in R(Ag, Env)} P(r \mid Ag, Env) = 1$. Optimální agent maximalizuje *očekávanou utilitu*:

$$Ag_{opt} = \arg \max_{Ag \in \mathcal{AG}} \sum_{r \in R(Ag, Env)} u(r) P(r \mid Ag, Env).$$

Definice je elegantní, ale **nekonstruktivní**: neříká, jak takového agenta navrhnout. Navíc bývá obtížné utilitní funkci vůbec zadat. (Poznámka: peníze nejsou pro člověka dobrou utilitou — utilita peněz je nelineární a liší se pro bohaté a chudé.)

Příklad: Tileworld. Klasické testovací prostředí pro agentní architektury. Agenti se pohybují po šachovnici do čtyř směrů, na desce jsou díry, překážky a bloky; cílem je zatlačit bloky do děr. Prostředí je *dynamické* — díry, překážky i bloky náhodně vznikají a zanikají. Utilita běhu:

$$u(r) = \frac{N_f}{N_a},$$

kde N_f je počet děr zaplněných agentem během běhu r a N_a počet všech děr, které se během r objevily. Agent musí umět *reagovat* na změny (blok, který tlačím, zmizel) i *využívat příležitosti* (vedle mě se objevil nový blok). Tileworld se historicky používal právě ke zkoumání kompromisu mezi reaktivitou a proaktivitou (viz odhodlání agentů, sekce 3.5).

2.5 Predikátová specifikace úlohy

Speciálním případem utility je zobrazení do $\{0, 1\}$: běh je buď úspěšný ($u(r) = 1$), nebo ne.

Prostředí úlohy (*task environment*)

Nechť $\Psi : R \rightarrow \{0, 1\}$ je **predikátová specifikace**: $\Psi(r)$ platí, právě když $u(r) = 1$. **Prostředí úlohy** je dvojice $\langle Env, \Psi \rangle$; specifikuje jak vlastnosti systému (přes Env), tak kritérium splnění úlohy (přes Ψ).

Označme $R_\Psi(Ag, Env) = \{r \in R(Ag, Env) \mid \Psi(r)\}$.

Kdy agent úlohu řeší? Existují tři výklady:

- **Pesimistický**: $R_\Psi(Ag, Env) = R(Ag, Env) \text{ — všechny běhy splňují } \Psi$.
- **Optimistický**: $\exists r \in R(Ag, Env) : \Psi(r) \text{ — alespoň jeden běh}$.
- **Realistický**: rozšíříme τ o rozdělení pravděpodobnosti a měříme úspěch pravděpodobností $P(\Psi \mid Ag, Env) = \sum_{r \in R_\Psi(Ag, Env)} P(r \mid Ag, Env)$.

Typy úloh

Úlohy dosažení (*achievement tasks*) — „něco najít“. Je dána množina cílových stavů $G \subseteq E$; $\Psi(r)$ platí, pokud se v běhu r vyskytne alespoň jeden stav z G . Typicky každá klasická úloha UI (prohledávání).

Úlohy udržení (*maintenance tasks*) — „něčemu se vyhnout“. Je dána množina „špatných“ stavů $B \subseteq E$; $\Psi(r)$ neplatí, pokud se v r vyskytne stav z B . Typicky hry, kde B jsou prohrané pozice a prostředím je soupeř.

Kombinace: dosáhnout stavu z G a přitom se vyhnout stavům z B .

2.6 Mechanismus výběru akcí — přehled

Mechanismus výběru akcí (*action selection mechanism*)

Procedura, kterou agent v každém okamžiku rozhoduje, kterou z dostupných akcí provede, na základě vjemů z prostředí a svého vnitřního stavu. **Architektura agenta** je softwarová architektura, která takový mechanismus realizuje — podle Maes (1991) „konkrétní metodologie stavby agentů, která určuje, jak lze agenta rozložit na množinu komponent a jak mají tyto komponenty interagovat, aby dohromady odpověděly na otázku, jak data ze senzorů a aktuální vnitřní stav určují akce a budoucí vnitřní stav agenta.“

Následující dvě kapitoly probírají hlavní rodiny mechanismů výběru akcí:

Rodina	Mechanismus výběru akce	Typický zástupce
symbolická / deliberativní	logická dedukce nad databází formulí; plánování	deduktivní agent, STRIPS, BDI, PRS
reaktivní (symbolická)	sada pravidel „situace $\rightarrow$ akce“ s pevnou prioritou	subsumpční architektura (Brooks)
reaktivní (konekcionistická)	šíření aktivace v síti modulů, vyhrává nejaktivnější	agent network architecture (Maes)
hybridní	vrstvy různé abstrakce + arbitráž mezi nimi	TouringMachines, InteRRaP
koaliční	soutěž dynamicky vznikajících koalic procesů o „vědomí“	IDA / globální pracovní prostor
naučená	maximalizace naučené Q -funkce / stochastická politika	Q-learning, actor-critic (kap. 12)

2.7 Shrnutí ke zkoušce

- Prostředí: 4 dichotomie (pozorovatelnost, statické/dynamické, determinismus, diskrétnost); reálná prostředí jsou v nejtěžší variantě.
- Formalismus: E , A , běh $r = e_0 a_0 e_1 \dots e_n$, R^A/R^E , přechodová funkce $\tau : R^A \rightarrow 2^E$ (odtud historie + nedeterminismus), $Env = \langle E, e_0, \tau \rangle$, agent $Ag : R^E \rightarrow A$, systém $\langle Ag, Env \rangle$.
- Funkční ekvivalence (vzhledem k prostředí / prostá).
- Čistě reaktivní agent $Ag : E \rightarrow A$ (slabší); agent s vnitřním stavem (*see*, *next*, *action*) — oba převeditelné na standardního agenta.
- Utilita: $u : E \rightarrow \mathbb{R}$ vs. $u : R \rightarrow \mathbb{R}$; optimální agent maximalizuje *očekávanou* utilitu — definice je nekonstruktivní.
- Prostor úlohy $\langle Env, \Psi \rangle$; pesimista/optimista/realista; úlohy dosažení (G) a udržení (B).
- Tileworld a utilita N_f/N_a jako standardní benchmark reaktivity vs. proaktivity.

3 Deliberativní architektury: symbolické uvažování, BDI, PRS

3.1 Klasický přístup UI a deduktivní agent

Klasický způsob, jakým UI staví „inteligentní systém“, stojí na dvou pilířích:

1. **Symbolická reprezentace** prostředí a chování — zde konkrétně logickými formulemi.
2. **Syntaktická manipulace** s touto reprezentací — tedy logická dedukce, dokazování vět.

Teorie o chování agenta je pak zároveň programem — *spustitelnou specifikací*, která generuje konkrétní akce. Tento přístup naráží na dva klasické problémy:

- **Problém transdukce** (*transduction problem*): jak převést reálný svět do symbolické reprezentace (počítačové vidění, zpracování přirozeného jazyka, učení). Goethem řečeno: „Šedá, milý příteli, je všechna teorie, a zelený je zlatý strom života.“
- **Problém reprezentace a uvažování**: jak znalosti reprezentovat a jak nad nimi efektivně odvozovat (dokazovače, plánovače).

Odbočka: dedukce vs. indukce. *Dedukce* je odvození závěrů, které jsou pravdivé, jsou-li pravdivé předpoklady — postup od obecného ke konkrétnímu (sylogismus: „Všichni lidé jsou smrtelní, Sókratés je člověk, tedy Sókratés je smrtelný“). *Indukce* znamená, že při pravdivých předpokladech je závěr spíše pravdivý než nepravdivý — od konkrétního k obecnému. Sherlock Holmes ve skutečnosti používá indukci, i když jí říká dedukce; matematická indukce je naopak dedukce.

Deduktivní agent (agent jako dokazovač vět)

Nechť L je množina formulí predikátové logiky prvního řádu a $D = 2^L$ množina databází těchto formulí. Vnitřní stav agenta je databáze $DB \in D$. Uvažování je realizováno odvozovacími pravidly ρ :

$$DB \vdash_{\rho} \varphi \quad \text{— formulí } \varphi \text{ lze odvodit z } DB \text{ pomocí pravidel } \rho.$$

Agent je pak trojice funkcí $see : E \rightarrow Per$, $next : D \times Per \rightarrow D$, $action : D \rightarrow A$, přičemž $action$ je definována procedurou níže.

Výběr akce jako dokazování.

- ```

1: function ACTION($DB : D$) : A
2: for all $a \in A$ do
3: if $DB \vdash_{\rho} Do(a)$ then return a
4: end if
5: end for
6: for all $a \in A$ do
7: if $DB \not\vdash_{\rho} \neg Do(a)$ then return a
```

```

8: end if
9: end for
10: return null
11: end function

```

První cyklus hledá akci, kterou lze *dokázat* jako žádoucí (odvodí se  $Do(a)$ ). Pokud žádná taková není, druhý cyklus hledá akci alespoň *konzistentní* s databází (nelze dokázat, že by se dělat neměla). Pokud selže i to, agent nedělá nic.

**Příklad: vysavačový svět  $3 \times 3$ .** Predikáty:  $In(x, y)$  (agent je na pozici  $(x, y)$ ),  $Dirt(x, y)$  (na  $(x, y)$  je špína),  $Facing(d)$  (agent je otočen směrem  $d$ ). Pravidla pro odvození akce:

$$\begin{aligned}
& In(x, y) \wedge Dirt(x, y) \Rightarrow Do(suck) \\
& In(0, 0) \wedge Facing(north) \wedge \neg Dirt(0, 0) \Rightarrow Do(forward) \\
& In(0, 2) \wedge Facing(north) \wedge \neg Dirt(0, 2) \Rightarrow Do(turn) \quad \dots
\end{aligned}$$

Agent tak systematicky projde svět ve tvaru „hada“ 00–01–02–12–11–10–... Všimněte si, že úklid má vyšší prioritu než pohyb (pravidlo pro *suck* je uvedeno první a ostatní pravidla mají v předpokladu negaci *Dirt*).

### Klady a zápory.

- + Elegantní, s jasnou (a krásnou) sémantikou; specifikace = program.
- Prakticky nepoužitelné pro netriviální úlohy: dokazování je pomalé, nemusí skončit.
- **Problém vypočitatelné racionality** (*calculative rationality*): agent odvodí optimální akci pro stav prostředí, jaký byl *na začátku* odvozování — mezitím se prostředí změnilo a akce už optimální není. Katastrofa v rychle se měnícím prostředí.
- Těžko se navrhuje funkce *see* (jak z obrázku udělat formule, jak reprezentovat čas).

Přesto se jednotlivé prvky tohoto přístupu objevují v pozdějších architekturách, zejména v BDI.

## 3.2 Praktické uvažování: architektura BDI

### Praktické uvažování (*practical reasoning*)

Inspirováno lidským rozhodováním. *Teoretické* uvažování (Sókratés) vede jen k tomu, co si o světě myslíme; *praktické* uvažování vede k **akcím**. Má dvě fáze:

- **Deliberace** (*deliberation*) — *čeho chceme dosáhnout* („chci dodělat magisterské studium“).
- **Uvažování o prostředcích** (*means-ends reasoning*), tedy plánování — *jak toho dosáhneme* (vytvořit plán, jak dostudovat).

A obojí nesmí trvat příliš dlouho.

### Beliefs — Desires — Intentions

**Beliefs** (přesvědčení) — informační stav agenta, jeho znalosti o světě. V MAS jim záměrně neříkáme „znalosti“, abychom zdůraznili, že jsou *subjektivní*, *nemusí být pravdivé* a *mohou se v budoucnu změnit*. Mohou obsahovat i odvozovací pravidla umožňující dopředné řetězení.

**Desires** (touhy) — motivační stav agenta, cíle nebo situace, kterých by chtěl dosáhnout. Touhy *mohou být vzájemně nekonzistentní* (chci se učit i jít na pivo).

**Intentions** (záměry) — stavy světa, kterých se agent *zavázal* dosáhnout. Záměry vedou k akcím: agent kvůli nim plánuje. Záměry **přetrvávají**, dokud

1. jich agent nedosáhne, nebo
2. nezačne věřit, že jsou nedosažitelné, nebo
3. nepominou důvody, které k nim vedly.

Záměry navíc *omezují další deliberaci* (nebudu znovu zvažovat možnosti neslučitelné s přijatým záměrem) a *ovlivňují budoucí přesvědčení* (počítám s tím, že se mi záměr podaří).

**Rozdíl mezi touhou a záměrem** ilustruje Bratman (1990): „Moje touha zahrát si dnes odpoledne basketbal je pouze potenciálním vlivem na moje odpolední chování. Musí soupeřit s mými dalšími relevantními touhami... Naproti tomu jakmile *zamýšlím* hrát dnes odpoledne basketbal, je věc rozhodnuta: obvykle už nemusím dál vážit pro a proti; až odpoledne přijde, prostě svůj záměr vykonám.“

#### Funkce deliberace

Označme  $B \subseteq Bel$  aktuální přesvědčení,  $D \subseteq Des$  aktuální touhy a  $I \subseteq Int$  aktuální záměry agenta. Deliberace se skládá ze tří funkcí:

- $brf : 2^{Bel} \times Per \rightarrow 2^{Bel}$  — *belief revision function*, aktualizace přesvědčení podle vjemu;
- $options : 2^{Bel} \times 2^{Int} \rightarrow 2^{Des}$  — generování možností (tužeb) z přesvědčení a stávajících záměrů;
- $filter : 2^{Bel} \times 2^{Des} \times 2^{Int} \rightarrow 2^{Int}$  — výběr záměrů z možností.

Rozdělení deliberace na *generování možností* a *filtrování* je klíčové: generování je levné a široké, filtrování je vlastní rozhodování (bere v úvahu i stávající závazky).

### 3.3 Plánování: STRIPS

#### Uvažování o prostředcích / plánování

Proces rozhodování, jak dosáhnout cíle (záměru) pomocí dostupných prostředků (akcí).

- **Vstup:** cíl (= záměr  $I$ ); aktuální stav prostředí (= přesvědčení  $B$ ); akce dostupné agentovi ( $Ac$ ).
- **Výstup:** plán = posloupnost akcí, po jejímž provedení je cíl splněn.

**STRIPS** (Fikes, Nilsson, 1971) modeluje svět množinou formulí logiky prvního řádu. Každá akce je popsána *schématem* s předpoklady a efekty (přidávané a mazané fakty). Plánovací algoritmus najde rozdíl mezi cílem a aktuálním stavem a vhodnou akci ho zmenší (*means-ends analysis*). Je to hezké, ale ne příliš praktické — algoritmus se často utápí v detailech nízké úrovně.

#### Deskriptor akce, plánovací problém, plán

**Deskriptor akce**  $a$  je trojice  $\langle P_a, D_a, A_a \rangle$ :  $P_a$  — předpoklady (*preconditions*),  $D_a$  — mazaná množina (*delete list*),  $A_a$  — přidávaná množina (*add list*); pro jednoduchost jde o uzemněné atomické formule (bez spojek a proměnných).

**Plánovací problém** je trojice  $\langle B_0, O, G \rangle$ , kde  $B_0$  jsou počáteční přesvědčení,  $O = \{ \langle P_a, D_a, A_a \rangle : a \in Ac \}$  deskriptory všech akcí a  $G$  množina formulí popisujících cíl.

**Plán**  $\pi = (a_1, \dots, a_n)$ ,  $a_i \in Ac$ , určuje posloupnost databází

$$B_i = (B_{i-1} \setminus D_{a_i}) \cup A_{a_i}, \quad i = 1, \dots, n.$$

Plán je **přípustný** (*admissible*), pokud  $B_{i-1} \models P_{a_i}$  pro každé  $i$ . Plán je **korektní/validní** (*correct*), pokud je přípustný a navíc  $B_n \models G$ .

**Plánování** znamená pro zadaný  $\langle B_0, O, G \rangle$  najít korektní plán, nebo prohlásit, že neexistuje.

**Příklad: svět kostek (*blocks world*).** Predikáty:  $On(x, y)$  ( $x$  je na  $y$ ),  $OnTable(x)$ ,  $Clear(x)$  (na  $x$  nic není),  $Holding(x)$  (rameno drží  $x$ ),  $ArmEmpty$ . Akce:

| Akce            | Pre / Del                                                                    | Add                        |
|-----------------|------------------------------------------------------------------------------|----------------------------|
| $Stack(x, y)$   | Pre: $\{Clear(y), Holding(x)\}$<br>Del: $\{Clear(y), Holding(x)\}$           | $\{ArmEmpty, On(x, y)\}$   |
| $UnStack(x, y)$ | Pre: $\{On(x, y), Clear(x), ArmEmpty\}$<br>Del: $\{On(x, y), ArmEmpty\}$     | $\{Holding(x), Clear(y)\}$ |
| $Pickup(x)$     | Pre: $\{OnTable(x), Clear(x), ArmEmpty\}$<br>Del: $\{OnTable(x), ArmEmpty\}$ | $\{Holding(x)\}$           |
| $PutDown(x)$    | Pre: $\{Holding(x)\}$<br>Del: $\{Holding(x)\}$                               | $\{ArmEmpty, OnTable(x)\}$ |

Počáteční stav  $B_0 = \{Clear(A), On(A, B), OnTable(B), OnTable(C), Clear(C), ArmEmpty\}$ , cíl  $G = \{OnTable(A), OnTable(B), OnTable(C)\}$ .

Korektní plán:  $\pi = (UnStack(A, B), PutDown(A))$ . Ověření:

- $B_0 \models \{On(A, B), Clear(A), ArmEmpty\}$ , tedy první akce je přípustná; po ní

$$B_1 = (B_0 \setminus \{On(A, B), ArmEmpty\}) \cup \{Holding(A), Clear(B)\}.$$

- $B_1 \models \{Holding(A)\}$ , tedy i druhá akce je přípustná; po ní

$$B_2 = (B_1 \setminus \{Holding(A)\}) \cup \{ArmEmpty, OnTable(A)\}.$$

- $B_2 \models G$ , plán je korektní.

#### Pomocné funkce nad plány

*Plan* — množina všech plánů nad  $Ac$ ; *pre*( $\pi$ ) — předpoklad plánu; *body*( $\pi$ ) — tělo plánu (posloupnost akcí); *empty*( $\pi$ ) — je plán prázdný?; *execute*( $\pi$ ) — provede všechny akce plánu; *hd*( $\pi$ ) — první akce; *tail*( $\pi$ ) — zbytek; *sound*( $\pi, I, B$ ) — plán  $\pi$  je korektní vzhledem k záměrům  $I$  a přesvědčením  $B$ .

Plánovací funkce agenta:  $plan : 2^{Bel} \times 2^{Int} \times 2^{Ac} \rightarrow Plan$ .

**Knihovna plánů.** Agent nemusí konstruovat plány online — to je drahé. Velmi často je *plan*() implementována jako *knihovna hotových plánů*: stačí ji jednou projít a zkontrolovat, zda předpoklady plánu odpovídají aktuálním přesvědčením a efekty plánu odpovídají cíli. Tento nápad je jádrem architektury PRS (sekce 3.6).

### 3.4 Implementace BDI agenta

Hlavní řídicí smyčka BDI agenta:

```

1: $B \leftarrow B_0; \quad I \leftarrow I_0$
2: while true do
3: $v \leftarrow see()$
4: $B \leftarrow brf(B, v)$
5: $D \leftarrow options(B, I)$
6: $I \leftarrow filter(B, D, I)$
7: $\pi \leftarrow plan(B, I, Ac)$
8: while $\neg(empty(\pi) \vee succeeded(I, B) \vee impossible(I, B))$ do
9: $a \leftarrow hd(\pi); \quad execute(a); \quad \pi \leftarrow tail(\pi)$
10: $v \leftarrow see(); \quad B \leftarrow brf(B, v)$
11: if reconsider(I, B) then
12: $D \leftarrow options(B, I); \quad I \leftarrow filter(B, D, I)$
13: end if
14: if $\neg sound(\pi, I, B)$ then
15: $\pi \leftarrow plan(B, I, Ac)$
```

▷ přeplánování

```

16: end if
17: end while
18: end while

```

Význam predikátů:  $succeeded(I, B)$  — záměry  $I$  platí za předpokladu přesvědčení  $B$ ;  $impossible(I, B)$  — záměry  $I$  nemohou platit za předpokladu  $B$ . Vnitřní cyklus tedy končí, když je plán vyčerpán, cíl splněn, nebo se stal nedosažitelným.

### 3.5 Odhodlání agenta (*commitment*)

#### Strategie odhodlání vůči záměrům

Kdy má agent opustit cíl?

- **Slepé odhodlání** (*blind commitment*, též *fanatical*) — agent udržuje záměr, dokud *nezačne věřit, že ho dosáhl*. (Nikdy se nevzdá.)
- **Jednosměrné odhodlání** (*single-minded commitment*) — udržuje záměr, dokud *nezačne věřit, že ho dosáhl, nebo že už není dosažitelný*.
- **Otevřené odhodlání** (*open-minded commitment*) — záměr přetrvává, dokud agent věří, že je stále dosažitelný (a stále chtěný) — tedy může být opuštěn i při změně motivace.

**Odhodlání vůči plánu** versus **odhodlání vůči cíli**. Agent v algoritmu výše je odhodlán jednomu plánu; opustí ho, když věří, že cíl je splněn, že je nedosažitelný, nebo když je plán prázdný. Zbývá ale těžší otázka: kdy má agent *přehodnotit sám záměr*?

Klasické dilema: deliberace **trvá čas** a prostředí se během ní mění.

- Agent, který záměry *nepřehodnocuje*, se může upnout k cílům, jichž už nelze dosáhnout.
- Agent, který přehodnocuje *příliš často*, je tak zaneprázdněn uvažováním, že nic nestihne.

**Metařízení** (Wooldridge, Parsons, 1999): funkce *reconsider()*, která je *výpočetně jednodušší* než samotné přehodnocení. Kritérium kvality: *reconsider()* funguje dobře, pokud pokaždé, když navrhne přehodnocení, agent po deliberaci *skutečně* záměr změní (a naopak).

#### Odvážný vs. opatrný agent

**Odvážný agent** (*bold agent*) — přehodnocuje záměry až po dokončení celého plánu (nikdy plán nepřeruší kvůli deliberaci).

**Opatrný agent** (*cautious agent*) — přehodnocuje po každé akci plánu.

**Míra odvahy** (*level of boldness*) — kolik akcí agent provede mezi dvěma deliberacemi.

**Dynamičnost prostředí** (*rate of world change*) — kolikrát se prostředí změní během jednoho cyklu agenta.

**Efektivita agenta** = počet dosažených záměrů / počet všech záměrů.

**Experimentální výsledek (Kinny, Georgeff, v Tileworldu):**

- Mění-li se svět *pomalu*, jsou efektivnější **odvážní** agenti — deliberace by byla zbytečná režie.
- Mění-li se svět *rychle*, překonávají je **opatrní** agenti — plány rychle zastarávají.

Toto je konkrétní kvantitativní podoba obecného kompromisu reaktivita vs. proaktivita.

### 3.6 Procedurální odvozovací systém (PRS)

#### PRS — *Procedural Reasoning System*

Georgeff a kol., 80. léta, Stanford. **První implementace architektury BDI** a jedna z prakticky nejúspěšnějších agentních architektur — byla mnohokrát reimplementována: **AgentSpeak/Jason, JAM, JACK, Jadex**.

Praktická nasazení: OASIS (systém řízení letového provozu v Sydney), SPOC (organizace pod-



nikových procesů), SWARMM (letecký simulátor australského letectva).

**Komponenty PRS agenta:** databáze *beliefs* (atomy FOL, prologovského typu), *desires* (cíle), *knihovna plánů* (*plan library*), zásobník *intentions* a *interpret*, který vše propojuje. Data ze senzorů vstupují do *beliefs*, *interpret* vydává akce.

### Plán v PRS

Agent má knihovnu hotových plánů reprezentujících jeho *procedurální znalosti*. **Neplánuje se od nuly**, pouze se z knihovny *vybír*á. Plán má tři části:

- **Goal** (též *invocation condition* / *cue*) — podmínka, která má platit po vykonání plánu; určuje, na jaký cíl se plán hodí.
- **Context** — podmínka nutná pro spuštění plánu (musí být konzistentní s přesvědčeními).
- **Body** — akce, které se mají vykonat.

Tělo plánu *není* jen lineární posloupnost akcí — může obsahovat další *cíle*: „dosáhni *f*“, „dosáhni *f* nebo *g*“, „dosahuj *f*, dokud nenastane *g*“. Tím vzniká hierarchické rozkládání cílů na podcíle.

### Běh interpretu:

1. Inicializace: počáteční přesvědčení  $B_0$  a cíl nejvyšší úrovně (*top-level goal*).
2. Zásobník záměrů obsahuje aktuální cíle v různých stavech rozpracovanosti.
3. Interpret vezme záměr z vrcholu zásobníku a v knihovně plánů hledá plány s odpovídajícím cílem.
4. Z nalezených plánů vybere ty, jejichž *kontext* je konzistentní s aktuálními přesvědčeními — ty tvoří aktuální *options* / *desires*.
5. **Deliberace** = výběr jednoho záměru z možností:
  - Původní PRS používal *metaplány* — plány o plánech, které modifikovaly záměry agenta. Ukázalo se to příliš složité.
  - Praktičtější varianta: každý plán má číselné *ohodnocení* (očekávanou utilitu), vybere se plán s nejvyšší hodnotou.
6. Vybraný plán se vykoná, což může vést k přidání dalších záměrů na zásobník.
7. Selže-li plán, agent zvolí jiný záměr z možností a pokračuje.

**Ukázka — Jason/AgentSpeak** (agent nosí majiteli pivo z ledničky):

```
available(beer,fridge). limit(beer,10). /* počáteční beliefs */
too_much(B) :- .count(consumed(_,_ ,B),Q) & limit(B,L) & Q > L. /* pravidlo */
+!has(owner,beer) /* spouštěcí událost */
: available(beer,fridge) & not too_much(beer) /* kontext */
<- !at(robot,fridge); open(fridge); get(beer); close(fridge); /* tělo */
!at(robot,owner); hand_in(beer).
+!at(robot,P) : not at(robot,P) <- move_towards(P); !at(robot,P).
```

Zápis `!g : c <- b` čte se: „nastane-li událost přidání cíle *g* a platí-li kontext *c*, vykonej tělo *b*“. Symbol `!` označuje *achievement goal* (dosáhni), `+/-` přidání/odebrání přesvědčení nebo cíle, `.send` a `.count` jsou vestavěné akce.

### 3.7 Shrnutí ke zkoušce

- **Deduktivní agent:**  $DB \in 2^L$ ,  $DB \vdash_\rho Do(a)$ ; dvoufázový výběr akce (dokazatelná akce  $\rightarrow$  konzistentní akce  $\rightarrow$  nic). Elegantní, ale nepraktický; problém transdukce a vypočitatelné racionality.
- **BDI:** dvě fáze — deliberace (*brf*, *options*, *filter*) a means-ends reasoning (plánování). Beliefs jsou subjektivní a omylné, desires mohou být nekonzistentní, intentions jsou závazky, které přetrvávají a omezují další deliberaci.
- **STRIPS:** deskriptor akce  $\langle P_a, D_a, A_a \rangle$ , plánovací problém  $\langle B_0, O, G \rangle$ ,  $B_i = (B_{i-1} \setminus D_{a_i}) \cup A_{a_i}$ ; plán *přípustný* ( $B_{i-1} \models P_{a_i}$ ) vs. *korektní* (navíc  $B_n \models G$ ). Umět projít svět kostek.
- **BDI smyčka** a role *reconsider* a *sound*.



- **Commitment:** blind / single-minded / open-minded; bold vs. cautious agent; efektivita = dosažené/všechny záměry; pomalé prostředí → bold, rychlé → cautious.
- **PRS:** první implementace BDI; plán = Goal + Context + Body; knihovna plánů místo plánování; zásobník záměrů; deliberace metaplány nebo užitou; potomci AgentSpeak/Jason, JAM, JACK, Jadex.

## 4 Reaktivní a hybridní architektury

### 4.1 Reaktivní přístup a jeho východiska

V 80. a 90. letech se ukázalo, že drobné úpravy symbolického přístupu nestačí, a vznikly alternativní paradigmaty UI. Jejich společným jmenovatelem je **odmítnutí symbolické reprezentace** a uvažování založeného na syntaktické manipulaci. Tři klíčové teze:

#### Tři principy reaktivního přístupu

**Interakce** (*situatedness*) — inteligentní chování závisí na prostředí, ve kterém je agent *zasazen*; nelze ho studovat izolovaně.

**Ztělesnění** (*embodiment*) — inteligentní chování není jen logika, je produktem agenta, který *má tělo* a fyzicky interaguje se světem.

**Emergence** — inteligentní chování *vzniká* (emerguje) interakcí mnoha jednoduchých chování; není nikde explicitně naprogramováno.

**Charakteristiky reaktivních agentů:** jsou *behaviorální* (důraz na vývoj a kombinaci jednotlivých chování), *situovaní* a *reaktivní* (agent v podstatě jen reaguje na prostředí, nedeliberuje). Reprezentace je **subsymbolická**: konekcionistické sítě, konečné automaty, jednoduchá pravidla typu IF–THEN.

### 4.2 Symbolické reaktivní plánování: subsumpční architektura

#### Subsumpční architektura (*subsumption architecture*)

Rodney Brooks, 1986–1991; nejúspěšnější reaktivní přístup. Tři Brooksovy teze:

1. Inteligentní chování lze generovat *bez symbolické reprezentace*.
2. Inteligentní chování lze generovat *bez explicitního abstraktního uvažování*.
3. Intelligence je *emergentní vlastnost* určitých složitých systémů.

Intelligence agenta je realizována množinou jednoduchých cílově orientovaných **chování** (*behaviours*), pro něž platí:

- Každé chování je samo o sobě mechanismus výběru akce (dvojice *situace* → *akce*).
- Každé chování přijímá vjemy a transformuje je na akce, je odpovědné za jeden dílčí cíl a má jednoduchou strukturu pravidla.
- Chování běží *paralelně* a *soupeří* spolu o kontrolu nad agentem.
- Chování jsou uspořádána do **subsumpční hierarchie**, která definuje jejich priority (vyšší vrstva „pohlcuje“ = potlačuje nižší).

**Mechanismus výběru akce v subsumpční architektuře:**

```

1: function ACTION(vjem p)
2: fired ← { b ∈ Behaviours | p splňuje podmínku b } ▷ chování, která „vystřelí“
3: for all b ∈ fired do
4: if neexistuje b' ∈ fired takové, že b' < b then ▷ b není nikým inhibováno
5: return akce příslušná chování b
6: end if
7: end for
8: return null ▷ nevystřelilo nic, agent nedělá nic
9: end function

```

Formálně: chování je dvojice  $\langle c, a \rangle$ , kde  $c \subseteq Per$  je podmínka a  $a \in A$  akce; na množině chování  $R$  je definována relace inhibice  $\prec \subseteq R \times R$  (striktní totální uspořádání). Pozor na směr — u Wooldridge i na přednášce znamená  $b_1 \prec b_2$ , že  $b_1$  **inhibuje**  $b_2$ , tedy  $b_1$  je *níže* v subsumpční hierarchii a má *přednost*. Zápis  $b_1 \prec b_2 \prec \dots \prec b_n$  tak vyjmenovává chování od nejvyšší priority k nejnižší.

**Vlastnosti:** mechanismus je *jednoduchý* (ale při desítkách chování bývají priority záludné) a *velmi rychlý* — lze ho implementovat hardwarově a má konstantní složitost.

**Příklad: Steelsův průzkumník Marsu.** Cílem je sbírat vzácné vzorky hornin na vzdálené planetě rojem robotických průzkumníků. Prostředky: základna vysílá navigační signál (roboti nemusí umět nic víc než detekovat *gradient* signálu); každý robot má radioaktivní dročky pro *nepřímou komunikaci* (stigmergii) s ostatními. Přímá komunikace není potřeba.

*První iterace* — náhodná procházka a návrat jednoho robota (priorita klesá shora dolů):

---

```

R1 if vidím překážku then změň směr
R2 if nesu vzorek and jsem na základně then polož vzorek
R3 if nesu vzorek and nejsem na základně then jdi po gradientu nahoru
R4 if vidím vzorek then seber ho
R5 if true then pohybuj se náhodně

```

---

*Druhá iterace* — lepší průzkum pomocí stigmergie. Vzorky se často vyskytují ve shlucích; robot, který jeden našel, o tom „řekne“ ostatním tím, že cestou domů rozhazuje dročky:

---

```

R6 if nesu vzorek and jsem na základně then polož vzorek
R7 if nesu vzorek and nejsem na základně then polož 2 dročky a jdi po gradientu nahoru
R8 if cítím drobek then seber 1 drobek a jdi po gradientu dolů

```

---

Priority (v notaci „levé inhibuje pravé“, tedy od nejvyšší priority):  $R1 \prec R6 \prec R7 \prec R4 \prec R8 \prec R5$ . Robot sledující stopu bere po jednom dročku, takže se stopa postupně *vypaří*, jakmile je shluk vytěžen. Roj vykazuje kooperativní chování, přestože žádný robot nemá model světa ani neposílá zprávy.

### 4.3 Konekcionalistické reaktivní plánování: agent network architecture

Subsumpční architektura řeší konflikty *pevnou* prioritou, což je rigidní. Konekcionalistický přístup nahradí pevné priority **spojitou aktivací šířenou sítí** — výběr akce je výsledkem dynamiky sítě, ne pevného pořadí. Jde o „reaktivní plánování“: agent se chová, jako by plánoval, ale žádný explicitní plán nikdy nevytvoří.

#### Agent network architecture / spreading activation networks

Pattie Maes, 1989–1991. Agent je množina **kompetenčních modulů** (*competence modules*), podobných Brooksovým chováním. Každý modul  $i$  má:

- **předpoklady**  $c_i$  (*preconditions*) — co musí platit, aby byl modul *proveditelný* (*executable*);
- **efekty**: *add list*  $a_i$  a *delete list*  $d_i$  (jako u STRIPS);
- **úroveň aktivace**  $\alpha_i$  a globální **práh aktivace**  $\theta$ .

Moduly jsou propojeny v orientovaném grafu na základě svých podmínek — shoda pre- a postpodmínek tvoří hrany, k nimž přibývají hrany reprezentující časovou následnost a konflikty. V každém kroku se šíří aktivace; **vybrán je proveditelný modul s nejvyšší aktivací**, pokud překročí práh  $\theta$ .

**Tři zdroje a tři cesty šíření aktivace:**

- **Ze stavu světa** (*state*): moduly, jejichž předpoklady jsou momentálně splněny, dostávají aktivaci — „teď se dá udělat tohle“.
- **Z cílů** (*goals*): moduly, jejichž *add list* obsahuje cíl, dostávají aktivaci — „tohle vede k cíli“.

- **Z ochranných cílů** (*protected goals*): moduly, jejichž delete list obsahuje již splněný cíl, jsou *inhibovány* — „tohle by zbořilo, co už mám“.

Aktivace se dále šíří mezi moduly třemi typy spojů:

- **Následnické spoje** (*successor links*): modul  $x$  posílá aktivaci modulu  $y$ , pokud  $y$  přidává něco, co  $x$  potřebuje jako předpoklad — *dopředné* řetězení („až budu hotov, půjde to dál“).
- **Předchůdcovské spoje** (*predecessor links*): modul  $x$ , který není proveditelný, posílá aktivaci modulům, které by mu splnily chybějící předpoklad — *zpětné* řetězení („pomoz mi, ať můžu běžet“).
- **Konfliktní spoje** (*conflict links*): modul  $x$  *inhibuje* modul  $y$ , který ruší jeho předpoklady.

Chování sítě řídí pět globálních parametrů:  $\theta$  (práh aktivace, jediný práh v systému),  $\phi$  (aktivace vpravená do sítě stavem světa),  $\gamma$  (aktivace vpravená cíli),  $\delta$  (aktivace odebraná chráněnými cíli) a  $\pi$  (*průměrná* úroveň aktivace, na kterou se síť po každém kroku normalizuje, aby se celková energie neměnila). Poměr  $\gamma/\phi$  přesně vyjadřuje kompromis **proaktivita vs. reaktivita**;  $\theta$  určuje **uvážlivost vs. rychlost** (vysoký práh  $\rightarrow$  agent déle váhá, ale rozhoduje se lépe). Pokud v daném kroku práh nepřekročí žádný proveditelný modul,  $\theta$  se sníží o 10 % a zkouší se znovu — síť tak nemůže „zamrznout“.

**Jak se aktivace v jednom kroku počítá.** Aktualizace probíhá ve čtyřech fázích: (1)  *vstup z prostředí* — každý pravdivý fakt rozdělí kvantum  $\phi$  rovným dílem mezi moduly, které jej mají v předpokladech; (2)  *vstup od cílů* — každý cíl rozdělí  $\gamma$  mezi moduly, které ho přidávají, a každý chráněný cíl *odebere*  $\delta$  modulům, které by ho rušily; (3)  *šíření mezi moduly* — proveditelný modul posílá část své aktivace dopředu následníkům (opět rozdělenou rovným dílem mezi příjemce), neproveditelný modul posílá aktivaci zpět modulům, které mu mohou splnit chybějící předpoklad, a konfliktní spoje odpovídající díl aktivace odečítají; (4) *normalizace* — aktivace všech modulů se přeskálují tak, aby jejich průměr byl  $\pi$  (celková energie sítě zůstává konstantní). Poté se vybere proveditelný modul s aktivací nad prahem  $\theta$ ; provedený modul svou aktivaci vynuluje.

*Poznámka k přednášce:* slidy uvádějí práh aktivace jako vlastnost jednotlivého modulu (a mluví o něm jako o prioritě); v Maesově původním modelu je práh  $\theta$  *globální* a roli „priority“ hraje aktuální úroveň aktivace  $\alpha_i$ , která se dynamicky mění.

**Srovnání se subsumpcí:** pořadí chování není pevné, ale vzniká *dynamicky* z aktuálního stavu světa a cílů. Síť tak dokáže vygenerovat i víceukrové „plánovité“ sekvence (aktivace se řetězí zpět od cíle přes předchůdcovské spoje), aniž by se kdy vytvořila explicitní datová struktura plánu — odtud název *reaktivní plánování*. Cenou je citlivost na nastavení parametrů a nemožnost zaručit optimalitu či úplnost.

#### 4.4 Omezení reaktivních architektur

- Reaktivní agenti **nemají model světa**; vše musí odvodit z aktuálních vjemů. Někdy proto musí *měnit prostředí*, aby si „zapamatovali“ informaci (radioaktivní droby) — externalizovaná paměť, stigmergie.
- Mají jen **krátkodobý pohled**: jednají podle aktuálního stavu a lokální informace, je obtížné zohlednit globální podmínky a dlouhodobé cíle.
- Emergence **není dobrý inženýrský postup**: chování je třeba vyladit experimentálně, chybí metodologie i možnost verifikace.
- Návrh je zvládnutelný pro několik vrstev; s **desítkami chování se stává neřešitelným** (počet interakcí roste kvadraticky).

#### 4.5 Hybridní architektury

Ani čistě reaktivní, ani čistě deliberativní architektura není ideální. Hybridní architektury kombinují obojí:

- **Deliberativní / plánovací komponenta** pracuje na symbolické úrovni, vytváří reprezentace a plány.
- **Reaktivní komponenta** zajišťuje okamžité reakce bez složitých výpočtů.

Komponenty jsou obvykle uspořádány do **vrstev** (*layered architectures*), přičemž reaktivní vrstvy mají zpravidla *přednost* před deliberativními (bezpečnost před optimalitou).

#### Horizontální vs. vertikální vrstvení

**Horizontální vrstvení:** všechny vrstvy jsou *paralelně* připojeny k senzorům i k efektorům. Každá vrstva je jako samostatný agent a navrhuje akci.

- + Koncepčně jednoduché: pro  $n$  druhů chování stačí  $n$  vrstev.
- Vrstvy se navzájem ovlivňují a jejich návrhy si odporují; je nutná **mediační funkce** (*mediator function*), která konflikt rozhodne. Ta je úzkým hrdlem: musí uvažovat  $m^n$  kombinací ( $n$  vrstev,  $m$  možných návrhů) a je kritickým bodem selhání.

**Vertikální vrstvení:** vrstvy jsou zapojeny *sériově*.

- *One-pass*: vjem vstoupí do nejnížší vrstvy, prochází nahoru, akce vystupuje z nejvyšší. Přirozené uspořádání a hierarchie chování.
- *Two-pass*: vjemy putují *zdola nahoru*, řídicí povely (akce) *shora dolů*. Tok připomíná řízení ve skutečné firmě.
- + Odpadá potřeba mediace, složitost interakcí je lineární ( $n - 1$  rozhraní).
- Není robustní: selhání kterékoli vrstvy shodí celého agenta.

#### TouringMachines (Ferguson, 1992)

**Horizontální třívrstvá architektura** pro simulovaného agenta v dopravním prostředí.

- **Modelovací vrstva** (*modelling layer*) — symbolické modely agenta, prostředí a ostatních agentů; předvídá a řeší konflikty, stanovuje cíle a posílá je plánovací vrstvě.
- **Plánovací vrstva** (*planning layer*) — proaktivní chování; vybírá z předpřipravených plánů (podobně jako PRS) a sestavuje z nich plán cesty.
- **Reaktivní vrstva** (*reactive layer*) — klasická reaktivní pravidla *situace* → *akce*, rychlá okamžitá reakce (vyhýbání překážkám).

Vrstvy pracují paralelně a jejich výstupy sjednocuje sada **řídících pravidel** (*control rules*) — konkrétní podoba mediační funkce; ta mohou vstupy a výstupy vrstev potlačovat (*censor* a *suppressor* pravidla).

#### InteRRaP (Müller, 1994)

**Vertikální dvouprůchodová** (*two-pass*) třívrstvá architektura.

- **Vrstva kooperativního plánování** (*cooperative planning layer*) — sociální interakce, společné plány s ostatními agenty.
- **Vrstva lokálního plánování** (*local planning layer*) — běžné individuální plánování.
- **Behaviorální vrstva** (*behaviour-based layer*) — reaktivní chování.

Každá vrstva má *vlastní znalostní bázi* na příslušné úrovni abstrakce (znalosti o světě / plány / společné plány a modely ostatních agentů). Vrstvy komunikují dvěma operacemi: *aktivace zdola nahoru* (nižší vrstva neumí situaci zvládnout a předá ji výš) a *zásah shora dolů* (vyšší vrstva používá nižší k vykonání svých rozhodnutí).

**Reálný příklad: Stanley** (Volkswagen Touareg R5, Stanford). Autonomní vůz, který vyhrál DARPA Grand Challenge 2005 (212 km Mohavskou pouští); přímý předchůdce dnešních autonomních vozidel. Vrstvená architektura:

- vrstva senzorů → vrstva abstraktní percepce → vrstva plánování a řízení (plán trasy a řízení vozu) → vrstva rozhraní vozidla;
- plus vrstva uživatelského rozhraní (panel, start) a vrstva globálních služeb (souborový systém, komunikace, hodiny).

## 4.6 Komplexní architektury: IDA a globální pracovní prostor

Poslední rodina architektur jde opačným směrem než reaktivní minimalismus: snaží se do jednoho agenta integrovat *všechno* — výběr akce, paměť, deliberaci, emoce.

### IDA — *Intelligent Distribution Agent* (S. Franklin)

Jedna z nejsložitějších agentních architektur, jaké byly postaveny. Vznikla pro reálnou úlohu amerického námořnictva — přidělování personálu na nová služební místa (odtud „distribution“); distribuovaná je i sama architektura.

**Východisko: teorie globálního pracovního prostoru** (*global workspace theory*, Baars, 1988–97) — jedna z teorií vědomí:

- Mysl je multiagentní systém: skládá se z mnoha jednoduchých, převážně nevědomých procesů provádějících specializované operace.
- Procesy spolu komunikují *zřídka* a pouze přes sdílenou paměť typu *blackboard*, organizovanou jako asociativní pole (srov. kap. 8).
- Procesy dynamicky vytvářejí **koalice** vyšších řádů.
- **Mechanismus výběru akce:** koalice, která nejlépe odpovídá aktuálním vstupům, se dostane do „vědomí“ (globálního pracovního prostoru) a je vykonána.
- Existuje *hierarchie kontextů* reprezentujících svět na různých úrovních — kontext cílů, vjemů, smyslů, konceptů, kulturní kontext.

**Hodnocení:** kombinuje mnoho přístupů z MAS i zbytku UI (výběr akce, paměť, deliberace, emoce), a právě to je jeho slabina — řada rozhodnutí je *ad hoc* a sladit tak heterogenní kolekci modelů do optimálně spolupracujícího celku je velmi obtížné.

Pro srovnání mechanismů výběru akce je IDA užitečný jako *čtvrtý* typ arbitráže: subsumpce rozhoduje **pevnou prioritou**, síť Maes **úrovní aktivace**, vrstvené architektury **mediační funkcí**, IDA **soutěží dynamicky vznikajících koalic** o přístup do globálního pracovního prostoru.

## 4.7 Srovnání architektur

|              | Deduktivní            | BDI / PRS                    | Reaktivní                | Hybridní            |
|--------------|-----------------------|------------------------------|--------------------------|---------------------|
| Reprezentace | symbolická (FOL)      | symbolická (beliefs, plány)  | subsymbolická / pravidla | smíšená po vrstvách |
| Výběr akce   | dokazování $Do(a)$    | deliberace + plán z knihovny | priorita / aktivace      | vrstvy + arbitráž   |
| Model světa  | úplný                 | částečný, revidovatelný      | žádný                    | po vrstvách         |
| Rychlost     | velmi nízká           | střední                      | vysoká (konst.)          | vysoká v reflexu    |
| Reaktivita   | špatná                | střední (laditelná)          | výborná                  | dobrá               |
| Proaktivita  | dobrá                 | výborná                      | žádná                    | dobrá               |
| Návrh        | obtížný, ale formální | systematický                 | experimentální           | složitý             |

## 4.8 Shrnutí ke zkoušce

- Reaktivní přístup stojí na trojici *situatedness* — *embodiment* — *emergence*; odmítá symbolickou reprezentaci.
- **Subsumpce** (Brooks): množina jednoduchých chování *situace* → *akce*, běžících paralelně, uspořádaných do pevné priority. Umět Steelsova průzkumníka Marsu a roli stigmergie (drobky).
- **Agent network architecture** (Maes) = konekcionistické reaktivní plánování: moduly s pre/add/delete, aktivace se šíří následnickými, předchůdcovskými a konfliktními spoji, zdroje aktivace jsou stav světa, cíle a inhibice od chráněných cílů; vybere se proveditelný modul s nejvyšší aktivací nad prahem. Parametry  $\gamma/\phi$  ladí proaktivitu vs. reaktivitu.
- Omezení reaktivních agentů: bez modelu světa, krátkodobý pohled, neinženýrský návrh, neškálovatelné množství vrstev.

- **Hybridní:** horizontální (paralelní vrstvy + mediátor,  $m^n$  kombinací, úzké hrdlo) vs. vertikální (sériové, one-pass/two-pass,  $n - 1$  rozhraní, křehké).
- **TouringMachines** = horizontální 3 vrstvy (modelling, planning, reactive) + řídicí pravidla. **InteRRaP** = vertikální two-pass 3 vrstvy (kooperativní plánování, lokální plánování, behaviorální) + vlastní KB na vrstvu. **Stanley** jako reálné nasazení.
- **IDA** (Franklin) + *global workspace theory* (Baars): mysl jako MAS jednoduchých nevědomých procesů komunikujících přes blackboard; koalice procesů soutěží o vstup do „vědomí“ a vítězná se vykoná; hierarchie kontextů. Nejsložitější architektura, mnoho ad hoc rozhodnutí.

## 5 Problém hledání cesty

### 5.1 Formulace a zařazení

Hledání cesty (*pathfinding*) je nejčastější konkrétní podúloha, kterou musí situovaný agent řešit — ať už jako součást plánovací vrstvy hybridní architektury, jako akce `move_towards(P)` v BDI plánu, nebo jako služba poskytovaná jiným agentům. V multiagentním kontextu se navíc objevují dvě specifika, která jednoagentní verze nezná: prostředí je **dynamické** (mapa se mění, překážky se objevují) a ostatní agenti jsou **pohyblivé překážky**, se kterými je nutné se koordinovat.

#### Problém hledání cesty

Je dán ohodnocený graf  $G = (V, E)$  s nezáporným ohodnocením hran  $w : E \rightarrow \mathbb{R}_0^+$ , počáteční vrchol  $s \in V$  a cílový vrchol  $g \in V$  (nebo množina cílů  $G_{\text{goal}}$ ). Hledáme cestu  $s = v_0, v_1, \dots, v_k = g$  minimalizující  $\sum_i w(v_{i-1}, v_i)$ .

Graf bývá zadán *implicitně* nástupnickou funkcí (mřížka 4-/8-souvislá, navigační síť, graf viditelnosti, mřížka konfiguračního prostoru robota).

### 5.2 A\* a příbuzné algoritmy

Prohledávání typu *best-first* udržuje pro každý vrchol  $n$  hodnoty

$g(n)$  = cena nejlepší dosud nalezené cesty  $s \rightarrow n$ ,

$h(n)$  = heuristický odhad ceny  $n \rightarrow g_{\text{cil}}$ ,

$f(n) = g(n) + h(n)$  — odhad ceny nejlepší cesty  $s \rightarrow g_{\text{cil}}$  vedoucí přes  $n$ .

**Algoritmus A\*** (Hart, Nilsson, Raphael, 1968):

- 1:  $OPEN \leftarrow \{s\}$  (prioritní fronta podle  $f$ ),  $CLOSED \leftarrow \emptyset$
- 2:  $g(s) \leftarrow 0$ ,  $f(s) \leftarrow h(s)$ , ostatní  $g \leftarrow \infty$
- 3: **while**  $OPEN \neq \emptyset$  **do**
- 4:    $n \leftarrow \arg \min_{x \in OPEN} f(x)$ ; odeber  $n$  z  $OPEN$
- 5:   **if**  $n$  je cíl **then return** rekonstruovaná cesta
- 6:   **end if**
- 7:   přidej  $n$  do  $CLOSED$
- 8:   **for all**  $m \in \text{succ}(n)$  **do**
- 9:      $g_{\text{new}} \leftarrow g(n) + w(n, m)$
- 10:    **if**  $g_{\text{new}} < g(m)$  **then**
- 11:      $g(m) \leftarrow g_{\text{new}}$ ;  $\text{parent}(m) \leftarrow n$ ;  $f(m) \leftarrow g(m) + h(m)$
- 12:     vlož/aktualizuj  $m$  v  $OPEN$  (a odeber z  $CLOSED$ , byl-li tam)
- 13:    **end if**
- 14:   **end for**
- 15: **end while**
- 16: **return** „cesta neexistuje“



## Připustnost a konzistence heuristiky

Heuristika  $h$  je **připustná** (*admissible*), pokud nikdy nenadhadnocuje:  $h(n) \leq h^*(n)$  pro všechna  $n$ , kde  $h^*(n)$  je skutečná cena nejlevnější cesty z  $n$  do cíle.

Heuristika je **konzistentní** (*consistent*, monotónní), pokud pro každou hranu  $(n, m)$  platí trojúhelníková nerovnost

$$h(n) \leq w(n, m) + h(m), \quad h(g_{\text{cil}}) = 0.$$

**Vlastnosti:** konzistence  $\Rightarrow$  připustnost. Při připustné heuristice je  $A^*$  *optimální* (najde nejlevnější cestu). Při konzistentní heuristice je navíc  $f$  podél každé cesty neklesající a každý vrchol je expandován *nejvýše jednou* (netřeba znovu otevírat vrcholy z *CLOSED*).  $A^*$  je také *optimálně efektivní*: žádný jiný optimální algoritmus používající tutéž heuristiku neexpanduje méně vrcholů. Tvzení je ovšem třeba brát s rezervou: platí pro vrcholy s  $f(n) < C^*$  (kde  $C^*$  je cena optima) — ty musí expandovat každý optimální algoritmus — kdežto na vrcholech s  $f(n) = C^*$  záleží na rozhodování remíz, a šťastnější strategie tam může expandovat méně.

### Speciální případy a varianty:

- $h \equiv 0 \Rightarrow A^*$  degeneruje na **Dijkstrův algoritmus** (resp. uniform-cost search).
- Typické heuristiky na mřížce: manhattanská  $|\Delta x| + |\Delta y|$  (pohyb do 4 směrů), oktilová  $\max + (\sqrt{2} - 1) \min$  (8 směrů), eukleidovská (libovolný úhel).
- **Weighted  $A^*$** :  $f = g + \varepsilon h$ ,  $\varepsilon > 1$  — rychlejší, ale  $\varepsilon$ -*suboptimální* (nalezená cesta je nejvýše  $\varepsilon \times$  dražší).
- **IDA\*** — iterativní prohlubování podle  $f$ ; paměť  $O(d)$  místo exponenciální.
- **JPS** (*Jump Point Search*) — na uniformních mřížkách přeskakuje symetrické cesty, řádové zrychlení bez ztráty optimality.
- **Theta\*** / *any-angle* — povoluje hrany mimo mřížku (kontrola přímé viditelnosti), dává realističtější trajektorie.
- **Hierarchické** plánování (HPA\*) — mapa se rozdělí na clustery, cesta se hledá nejprve v abstraktním grafu.

**Malý příklad.** Mřížka  $3 \times 3$ , pohyb do 4 směrů, cena hrany 1, start  $(0, 0)$ , cíl  $(2, 2)$ , překážka na  $(1, 1)$ ; manhattanská heuristika  $h(x, y) = |2 - x| + |2 - y|$  je připustná i konzistentní. Start má  $f = 0 + 4 = 4$ ; nástupci  $(1, 0)$  a  $(0, 1)$  mají  $g = 1$ ,  $h = 3$ ,  $f = 4$ ; dále  $(2, 0)$  a  $(0, 2)$  s  $g = 2$ ,  $h = 2$ ,  $f = 4$ , pak  $(2, 1)$  a  $(1, 2)$  s  $g = 3$ ,  $h = 1$ ,  $f = 4$  a konečně cíl  $(2, 2)$  s  $g = 4$ ,  $h = 0$ ,  $f = 4$ . Všechny vrcholy na optimální cestě mají  $f = 4$  — typický projev konzistentní heuristiky, která zbytek cesty odhaduje přesně.

## 5.3 Hledání cesty v dynamickém prostředí

Agent se typicky pohybuje a průběžně zjišťuje, že mapa je jinde jiná (nová překážka, změněná cena). Přeplánovat  $A^*$  od nuly v každém kroku je drahé. Řešením jsou **inkrementální** prohledávací algoritmy, které při lokální změně přepočítají jen zasaženou část.

### LPA\*, D\* a D\* Lite

**LPA\*** (*Lifelong Planning A\**, Koenig, Likhachev, 2001) — inkrementální verze  $A^*$  pro *pevný* start a cíl. Ke každému vrcholu udržuje kromě  $g(n)$  i hodnotu  $rhs(n)$  (*right-hand side*), tzv. *lookahead* hodnotu odvozenou z  $g$  předchůdců:

$$rhs(n) = \begin{cases} 0 & n = s, \\ \min_{p \in \text{pred}(n)} (g(p) + w(p, n)) & \text{jinak.} \end{cases}$$

Vrchol je **lokálně konzistentní**, pokud  $g(n) = rhs(n)$ . Změní-li se cena hrany, stanou se některé vrcholy nekonzistentními, zařadí se do prioritní fronty a algoritmus propaguje změnu jen tam, kde je to potřeba.



**D\*** (*Dynamic A\**, Stentz, 1994) — původní algoritmus pro robota, který se pohybuje a objevuje překážky. Prohledává *od cíle ke startu* (aby hodnoty zůstaly použitelné, když se robot posune). **D\* Lite** (Koenig, Likhachev, 2002) — LPA\* prohledávaný od cíle, s korekcí prioritní fronty při pohybu robota. Funkčně ekvivalentní D\*, ale výrazně jednodušší; *de facto* standard v mobilní robotice. Řádově rychlejší než opakované volání A\* — typicky se přepočítá jen malé okolí změny. **Ostatní varianty:** *Anytime Repairing A\** (ARA\*) — vrací rychle suboptimální řešení a postupně ho vylepšuje; *AD\** — kombinace anytime a dynamického přeplánování; *Field D\** — interpolace pro spojitý pohyb.

**A co spojitý prostor?** Předchozí algoritmy potřebují graf. Ve *spojitém konfiguračním prostoru* robota (poloha  $\times$  orientace  $\times$  klouby) se graf buď vyrobí diskretizací, nebo *vzorkováním*:

- **PRM** (*probabilistic roadmap*) — náhodně nasampluj volné konfigurace, spoj blízké dvojice lokálním plánovačem, ve vzniklé „silniční síti“ hledej klasicky (A\*). Vhodné pro opakované dotazy ve statické mapě.
- **RRT** (*rapidly-exploring random tree*) a **RRT\*** — inkrementálně rostoucí strom směrem k náhodným vzorkům; jednorázové dotazy, zvládá i kinodynamická omezení. RRT je *pravděpodobnostně úplný* (najde řešení s pravděpodobností jdoucí k 1), RRT\* navíc *asymptoticky optimální*.
- **Potenciálová pole** — cíl přitahuje, překážky odpuzují; velmi rychlé a reaktivní (blíží se subsumpci), ale uvíznou v *lokálních minimech*.

Společné pro MAS je, že tyto plánovače bývají *lokální/nízkoúrovňovou* vrstvou hybridní architektury, zatímco koordinace mezi agenty (MAPF) se řeší nad diskrétním grafem.

**Alternativa: plánování s volným prostorem.** V neznámém prostředí se často používá *optimistický předpoklad* (*freespace assumption*): neznámé buňky se považují za průchodné, agent jde po naplánované cestě, a jakmile narazí, přeplánuje. To je přesně scénář, pro nějž byl D\* navržen.

## 5.4 Hledání cest pro více agentů (MAPF)

### Multi-Agent Path Finding, MAPF

Vstup: graf  $G = (V, E)$  a  $k$  agentů, agent  $i$  má start  $s_i$  a cíl  $g_i$  (všechny různé). Čas je diskrétní; v každém kroku každý agent buď přejde po hraně, nebo počká na místě.

**Konflikty** — řešení musí být *bezkonfliktní*:

- *vrcholový konflikt* — dva agenti jsou ve stejném čase ve stejném vrcholu;
- *hranový / „swap“ konflikt* — dva agenti si v jednom kroku vymění pozici (projdou skrz sebe);
- podle varianty i *following konflikt* (agent vstoupí do vrcholu, který soused právě opouští) a *cyklické rotace*.

**Účelová funkce:** *sum-of-costs*  $\sum_i cost_i$ , nebo *makespan*  $\max_i cost_i$ .

**Složitost:** nalezení *optimálního* řešení je NP-těžké (pro obě účelové funkce); rozhodnutí, zda existuje *nějaké* řešení, je na obecných grafech polynomiální.

Přístupy k MAPF se dělí na svázané a rozvázané:

| Typ                                                 | Princip                                                                                                                            | Vlastnosti                                                                                                 |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>svázaný</b> ( <i>coupled</i> )                   | A* v <i>sdrůženém</i> stavovém prostoru $V^k$ (společný stav = pozice všech agentů)                                                | optimální a úplný, ale stavový prostor roste exponenciálně s $k$ ; prakticky do několika agentů            |
| <b>rozvázaný</b> ( <i>decoupled</i> )               | každý agent plánuje sám, konflikty se řeší dodatečně (priority, rezervace)                                                         | rychlý, škáluje, ale <i>není úplný ani optimální</i>                                                       |
| <b>hybridní / vyhledávání nad konflikty redukce</b> | prohledávání prostoru <i>omezené</i> , nikoli pozic (CBS, ICTS, M*)<br>převod na SAT, ILP, ASP nebo CSP a použití hotového solveru | optimální a přitom prakticky použitelný pro desítky agentů<br>optimální, velmi silné na hustých instancích |

### Prioritizované plánování (*prioritized planning*)

Nejjednodušší rozvázaný přístup (Erdmann, Lozano-Pérez, 1987):

1. Uspořádej agenty podle nějaké priority.
2. Pro  $i = 1, \dots, k$ : naplánuj cestu agenta  $i$  algoritmem  $A^*$  v *prostoru-čase* (stav = dvojice ⟨vrchol, čas⟩) tak, aby se *vyhnul* již naplánovaným trajektoriím agentů  $1, \dots, i-1$ ; ty jsou uloženy v **rezervační tabulce** (*reservation table*).

Tato varianta se nazývá **Cooperative  $A^*$**  (Silver, 2005); rozšíření *Hierarchical Cooperative  $A^*$*  používá abstraktní heuristiku ignorující ostatní agenty a *Windowed HCA\** plánuje jen na omezené časové okno (potřebné pro real-time hry).

**Slabina:** algoritmus je *neúplný* — existují instance řešitelné, ale neřešitelné žádným pořadím priorit (klasicky dva agenti v úzkém koridoru, kde jeden musí uhnout do výklenku).

### Conflict-Based Search (CBS)

Sharon a kol., 2012–2015. Dvouúrovňový optimální algoritmus.

**Nízká úroveň:** pro každého agenta se hledá optimální cesta v *prostoru-čase* pomocí  $A^*$  při respektování množiny *omezení* (*constraints*) tvaru  $\langle a_i, v, t \rangle$  („agent  $i$  nesmí být ve vrcholu  $v$  v čase  $t$ “), případně hranových omezení  $\langle a_i, (u, v), t \rangle$ .

**Vysoká úroveň:** prohledává *strom omezení* (*constraint tree*) do šířky podle ceny:

1. Kořen: prázdná množina omezení, každý agent má individuálně optimální cestu.
2. Vezmi uzel s nejnižší cenou řešení; ověř, zda jsou cesty bezkonfliktní. Pokud ano, **vrať řešení** — je optimální.
3. Jinak vyber první konflikt  $\langle a_i, a_j, v, t \rangle$  a *rozvětvi* na dva potomky: do prvního přidej omezení  $\langle a_i, v, t \rangle$ , do druhého  $\langle a_j, v, t \rangle$ . V každém potomku přeplánuj *jen* dotčeného agenta.

**Klíčová myšlenka:** nikdy se neprohledává sdružený stavový prostor  $V^k$ ; větví se pouze podle skutečně nastalých konfliktů, kterých bývá málo. CBS je *úplný* a *optimální* (sum-of-costs).

**Vylepšení:** ICBS/CBSH (silnější heuristiky pro vysokou úroveň, klasifikace konfliktů na kardinální/semikardinální/nekardinální), *meta-agent* CBS (agenty s mnoha vzájemnými konflikty sloučí do jednoho meta-agenta a ty řeší svázaně), ECBS (*bounded suboptimal*, řádově rychlejší za cenu faktoru  $1 + \varepsilon$ ).

**Příklad běhu CBS.** Koridor 1 – 2 – 3 s výklenkem 4 napojeným na 2; agent  $a_1$  jde  $1 \rightarrow 3$ , agent  $a_2$  jde  $3 \rightarrow 1$ .

- **Kořen** (prázdná množina omezení): individuálně optimální cesty  $a_1 : (1, 2, 3)$  a  $a_2 : (3, 2, 1)$ , cena  $2 + 2 = 4$ . V čase  $t = 1$  jsou oba ve vrcholu 2 — vrcholový konflikt  $\langle a_1, a_2, 2, 1 \rangle$ .
- **První větvení:** levý potomek přidá omezení  $\langle a_1, 2, 1 \rangle$ , pravý  $\langle a_2, 2, 1 \rangle$ ; v každém se přeplánuje *jen* dotčený agent. Vlevo tedy  $a_1 = (1, 1, 2, 3)$  (počká), cena uzlu  $3 + 2 = 5$ . Tím ale vznikne *swap* konflikt: mezi  $t = 1$  a  $t = 2$  jde  $a_1$  po hraně  $1 \rightarrow 2$  a  $a_2$  po téže hraně  $2 \rightarrow 1$ . Symetricky vpravo.
- **Další větvení** na hranovém konfliktu donutí jednoho z agentů zajet do výklenku. Optimální řešení má cenu **7**:  $a_1 = (1, 1, 2, 3)$  za 3 a  $a_2 = (3, 2, 4, 2, 1)$  za 4. (Cena 5 ani 6 neexistuje: agent jdoucí přímo přes vrchol 2 vždy způsobí buď vrcholový, nebo swap konflikt — vyzkoušejte si všechny čtyři kombinace cest délky 3.)

Příklad ilustruje dvě typické vlastnosti CBS: cena uzlů podél větve *neklesá* ( $4 \rightarrow 5 \rightarrow 7$ ) a zabrat musí *oba* agenti, ne jen ten „ustupující“. Bez výklenku je instance vůbec **neřešitelná** a základní CBS by větvil donekonečna — úplnost CBS je zaručena jen pro řešitelné instance, neřešitelnost se musí testovat zvlášť.

## 5.5 Shrnutí ke zkoušce

- $A^*$ :  $f = g + h$ ; *přípustnost*  $h \leq h^* \Rightarrow$  optimalita; *konzistence*  $h(n) \leq w(n, m) + h(m) \Rightarrow$  každý vrchol se expanduje jednou.  $h \equiv 0$  dá Dijkstru; weighted  $A^*$  je  $\varepsilon$ -suboptimální.

- Dynamické prostředí: LPA\* (hodnoty  $g$  a  $rhs$ , lokální konzistence  $g = rhs$ ), D\* a D\* Lite (prohledávání od cíle, inkrementální oprava), freespace assumption.
- MAPF: definice, vrcholové a hranové (swap) konflikty, sum-of-costs vs. makespan; optimální MAPF je NP-těžký.
- Rozvázané řešení: prioritizované plánování / Cooperative A\* s rezervační tabulkou, plánování v prostoru-čase — rychle, ale neúplné.
- **CBS**: nízká úroveň = A\* s omezeními pro jednoho agenta, vysoká úroveň = binární větvení stromu omezení podle nalezeného konfliktu; optimální a úplný, vyhýbá se exponenciálnímu sdruženému prostoru.

## 6 Znalosti a ontologie

### 6.1 Od agenta k multiagentnímu systému

Umíme navrhnout jednoho agenta. Aby agenti tvořili systém, musí spolu komunikovat, a k tomu potřebují vyřešit:

- **reprezentaci znalostí** — v čem jsou znalosti vyjádřeny;
- **společný slovník** — aby stejné slovo znamenalo totéž (to je úloha *ontologií*);
- **standardní zprávy** — syntaxi a sémantiku komunikace (*ACL*);
- **předvídatelné chování při komunikaci** — *interakční protokoly*;
- **technické problémy** — distribuovanost, mobilita agentů, asynchronnost, nespolehlivé komunikační kanály.

### 6.2 Uvažování o znalostech: epistemická logika a společná znalost

Dříve než se budeme zabývat *reprezentací* znalostí (ontologiemi), je třeba říci, jak se o znalostech agentů formálně *uvažuje*.

#### Epistemická logika a společná znalost

Znalost agenta se modeluje modálním operátorem  $K_i\varphi$  („agent  $i$  ví, že  $\varphi$ “) s **kripkovskou sémantikou možných světů**:  $K_i\varphi$  platí ve světě  $w$ , právě když  $\varphi$  platí ve *všech* světech, které agent  $i$  neumí od  $w$  odlišit (relace nerozlišitelnosti — ekvivalence). **Znalost** se axiomatizuje systémem **S5** (KT45); klíčový je axiom **T**:  $K_i\varphi \Rightarrow \varphi$  (co vím, to platí). **Přesvědčení** (*belief*, operátor  $B_i$ , srov. BDI a sémantiku FIPA SL) se axiomatizuje systémem **KD45**, kde axiom T nahrazuje slabší **D**:  $\neg B_i \perp$  — přesvědčení musí být konzistentní, ale *mohou být mylná*. Rozdíl znalost vs. přesvědčení je tedy formálně právě axiom T.

Pro skupinu agentů  $G$  se definuje:  $E_G\varphi$  — každý v  $G$  ví  $\varphi$ ; **společná znalost** (*common knowledge*)  $C_G\varphi \equiv E_G\varphi \wedge E_GE_G\varphi \wedge E_GE_GE_G\varphi \wedge \dots$  (všichni vědí, že všichni vědí, že...); *distribuovaná znalost*  $D_G\varphi$  —  $\varphi$  plyne ze sjednocení znalostí všech členů skupiny.

**Koordinovaný útok** (*coordinated attack*, problém dvou generálů): dva generálové se přes nespolehlivého posla domlouvají na současném útoku. Žádný konečný počet doručených potvrzení nevytvoří společnou znalost dohody — po  $k$  zprávách platí nejvýše  $E_G^k\varphi$ , nikdy  $C_G\varphi$ . Poučení pro MAS: *nespolehlivá komunikace nikdy nevytvoří společnou znalost*; dokonalá koordinace proto vyžaduje buď spolehlivé kanály, nebo slabší cíle (časově omezené či pravděpodobnostní varianty společné znalosti).

*Problém logické vševědoudnosti*: v kripkovské sémantice agent automaticky zná všechny logické důsledky svých znalostí i všechny tautologie — reálný, výpočetně omezený agent nikoli. To je hlavní námitka proti přímočarému použití epistemické logiky jako implementačního nástroje (srov. potíže deduktivních agentů, kap. 3).

## 6.3 Co je ontologie

### Ontologie

*Filozoficky* (z řeckého *to on* = jsoucí, *logos* = slovo): nauka o bytí, podle Aristotela „první filozofie“, součást metafyziky.

*V informatice*: **explicitní a formalizovaný popis části reality**. Obvykle formální deklarativní popis obsahující *glosář* (definice konceptů) a *tezaurus* (definice vztahů mezi koncepty). Ontologie je jakýsi slovník pro ukládání a výměnu znalostí o doméně standardním způsobem.

**Gruber (1995)**: „Ontologie je popis (podobný formální specifikaci programu) konceptů a vztahů, které mohou pro agenta nebo komunitu agentů formálně existovat.“ Často citovaná zkrácená verze: *ontologie je explicitní specifikace konceptualizace*.

**Motivační příklad** (dialog dvou lidí — nebo dvou agentů). „Slyšel jsi 7777? — Ne, co to je? — To je nové CD kapely SRPR, takový alt.rock s elektronikou a skvělými texty. — Aha.“ Aby si rozuměli, musí sdílet: 7777 je album; spadá do žánrů alt.rock a electronica; má skladby, interpreta (SRPR), autory hudby a textů; SRPR je kapela, má členy, ti hrají na nástroje. . .

### Formalismus ontologie

- **Třídy** (*classes*, koncepty) — věci mající něco společného.
- **Instance** (*individuals*, objekty) — konkrétní jedinci ze tříd.
- **Vlastnosti** (*properties*) — atributy tříd a instancí.
- **Relace mezi třídami**, zejména **podtřída** (*is-a*) — *tranzitivní* relace.
- Další relace a vlastnosti (základní znalosti o doméně).

**Strukturální část** (třídy, relace, axiomy) se obvykle nazývá vlastní *ontologie*; spolu s **fakty o konkrétních jedincích** tvoří **znalostní bázi** (*knowledge base*).

**Ontologie ontologií podle síly formalizace** (od slabých k silným): slovník (řízený slovník vybraných termínů) → glosář (definice významů v přirozeném jazyce) → tezaurus (synonyma) → neformální hierarchie (Amazon, wiki) → formální *is-a* hierarchie (subsumpce tříd) → třídy s vlastnostmi → omezení hodnot („každý člověk má právě jednu matku“) → libovolná logická omezení (vede na složité odvozovací algoritmy).

**Ontologie ontologií podle rozsahu:**

- **Aplikační** — nejběžnější v praxi, těžko znovupoužitelné.
- **Doménové** — oblíbený předmět výzkumu sémantického webu.
- **Horní** (*upper ontologies*) — měly by popisovat „všechno“ (Thing, Living thing, . . .) a sloužit jako základ doménových ontologií: BFO, GFO, UFO; WordNet a IDEAS nejsou vhodné pro formální odvozování. Vůči snaze o univerzální ontologii existují metodologické námitky (Wittgenstein, *Tractatus logico-philosophicus*).

## 6.4 Ontologické jazyky

Ontologické jazyky jsou formální deklarativní jazyky pro reprezentaci znalostí; obsahují fakta a odvozovací pravidla a nejčastěji se opírají o logiku prvního řádu nebo o *deskripční logiku*.

**Rámce** (*frames*) — historický předchůdce (Minsky), oblíbený v klasické UI a expertních systémech. Odpovídají objektovému programování:

| Terminologie rámců         | Objektová terminologie      |
|----------------------------|-----------------------------|
| Frame (rámec)              | třída objektů               |
| Slot                       | vlastnost / atribut objektu |
| Trigger, accessor, mutator | metoda                      |

**XML** nebyl pro ontologie vytvořen, ale používá se: hlavní výhodou je možnost definovat vlastní

značky, které přirozeně reprezentují slovník:

```
<product typ="CD">
 <title>7777</title>
 <artist>SRPR</artist>
</product>
```

Chybí mu ovšem definovaná sémantika — popisuje jen strukturu dokumentu, ne význam.

### RDF — *Resource Description Framework*

Standardní nástroj pro reprezentaci znalostí (nejen) pro web. Je **jednoduchý**: málo expresivní, zato s rychlými odvozovacími algoritmy.

Vše je vyjádřeno **trojicemi** *subjekt – predikát – objekt*, tedy *Predikt(Subjekt, Objekt)*. Například *MarriedTo(Karel, Jja)*, *FatherOf(Karel, Pa)*. Množina trojic tvoří orientovaný ohodnocený graf (*RDF graph*); zdroje se identifikují pomocí IRI. Serializace: RDF/XML, Turtle, N-Triples, JSON-LD. Dotazovacím jazykem je SPARQL. Nadstavba **RDFS** přidává třídy, *subClassOf*, *domain*, *range*.

### OWL — *Web Ontology Language*

Rovněž pochází ze (sémantického) webu, existuje ve verzích 1 a 2. Je to *kolekce několika formalismů* pro popis ontologií; syntaxe může být XML, RDF, funkcionální notace, Manchester syntax nebo Turtle. Snaha o formální, a přesto prakticky použitelný jazyk.

Základem je **deskripční logika** — *rozhodnutelný fragment* logiky prvního řádu.

**Předpoklad otevřeného světa** (*open world assumption*, OWA): co nevíme, je *nedefinované* — na rozdíl od předpokladu uzavřeného světa (databáze, Prolog), kde je vše, co nevíme, *nepravdivé*.

### Deskripční logika

Tři druhy entit: **koncepty** (třídy), **role** (vlastnosti, predikáty) a **individua** (objekty). **Axiom** je logický výraz o rolích a konceptech — na rozdíl od rámců a objektového programování, které třídu popisují *úplně*, axiomy popisují třídu jen *částečně* (a mohou přicházet z více zdrojů).

Jde o celou rodinu logických systémů podle povolených konstruktorů. Základní **ALC**: negace, konjunkce a disjunkce konceptů, omezené existenční a univerzální kvantifikátory. Rozšíření: hierarchie rolí (*H*), tranzitivita (*S*), inverzní role (*I*), kardinalitní omezení (*N*, *Q*), nominály (*O*). Znalostní báze se dělí na **T-Box** (terminologie — axiomy o konceptech a rolích) a **A-Box** (asertivní část — fakta o individuích).

**Profilý OWL.** *OWL 1 Lite (SHIF)* — nejjednodušší, blízko RDF, mnoho omezení kvůli rychlému strojovému zpracování; *OWL 1 DL (SHOIN)* — odpovídá deskripční logice (umožňuje např. vyjádřit disjunktnost tříd), je úplný a rozhodnutelný; *OWL 1 Full* — největší vyjadřovací síla, ale mnoho problémů je nerozhodnutelných. *OWL 2 (SROIQ)* přidává profily: *EL* (odvozování v polynomiálním čase, velké lékařské ontologie typu SNOMED), *QL* (dotazy nad znalostní bází, napojení na relační databáze) a *RL* (axiomy ve tvaru pravidel).

### KIF — *Knowledge Interchange Format*

Reprezentace znalostí v logice prvního řádu, LISPovská prefixová syntaxe. Byl navržen v rámci projektu DARPA *Knowledge Sharing Effort* jako **vnitřní jazyk** zpráv (obsah), zatímco KQML byl *vnějším* jazykem (obálka). Příklady:

```
(salary 015-46-3946 widgets 72000)
(> (* (width chip1) (length chip1)) (* (width chip2) (length chip2)))
(forall (?F ?T) (=> (and (instance ?F Farmer) (instance ?T Tractor)) (likes
?F ?T)))
(exists (?F ?T) (and (instance ?F Farmer) (instance ?T Tractor) (likes ?F
?T)))
```



**DAML+OIL** — DARPA Agent Markup Language + Ontology Interchange Language, přímý předchůdce OWL, opuštěn v roce 2006.

## 6.5 Shrnutí ke zkoušce

- Epistemická logika:  $K_i\varphi$  + možné světy; znalost = S5 (axiom T: co vím, platí), přesvědčení = KD45 (jen konzistence). Společná znalost  $C_G\varphi$  („všichni vědí, že všichni vědí...“) nevznikne nespolehlivou komunikací (koordinovaný útok); logická vševědoudcnost jako hlavní slabina.
- Ontologie = explicitní formalizovaný popis části reality (glosář + thesaurus); Gruberova definice. Formalismus: třídy, instance, vlastnosti, relace (zejména tranzitivní *is-a*), fakta. Ontologie + fakta = znalostní báze.
- Škála formalizace od slovníku po libovolná logická omezení; dělení na aplikační, doménové a horní ontologie.
- RDF = trojice *subjekt-predikát-objekt*, jednoduché a rychlé; RDFS přidává třídy.
- OWL = deskripční logika (rozhodnutelný fragment FOL), *open world assumption*, T-Box vs. A-Box; profily Lite/DL/Full (*SHIF*, *SHOIN*), OWL 2 = *SRIOQ* + profily EL/QL/RL.
- KIF = FOL v LISPovské syntaxi, obsah zprávy; KQML = obálka.

## 7 Komunikace agentů: řečové akty, ACL, protokoly

### 7.1 Proč je komunikace agentů jiná

V objektovém programování znamená „komunikace“ volání metody objektu s parametry — volaný objekt *musí* vyhovět. Agenti nemohou přinutit jiného agenta, aby něco udělal, ani nemohou přímo měnit jeho vnitřní proměnné. Musí **provést komunikační akt** s cílem

- vyměnit si informace, nebo
- *ovlivnit* jiné agenty, aby něco udělali.

Ostatní agenti mají vlastní cíle a je jen na nich, jak s přijatou informací, žádostí či dotazem naloží. To je zásadní rozdíl: **komunikace v MAS je akce, ne volání**.

### 7.2 Teorie řečových aktů

#### Řečové akty (*speech acts*) — Austin, 1962

Některé části užívání jazyka mají charakter **akcí**, protože mění stav světa podobně jako fyzické akce — to jsou řečové akty:

- „Prohlašuji vás mužem a ženou.“
- „Vyhlašuji válku.“

Jde o *pragmatickou* teorii jazyka: jak se řeč používá k dosahování cílů. Typická performativní slovesa: *request*, *inform*, *promise*.

#### Tři vrstvy řečového aktu

- **Lokuční akt** (*locutionary act*) — *co bylo řečeno*; samotný výrok. „Udělej mi čaj.“
- **Ilokuční akt** (*illocutionary act*) — *co bylo míněno*; lokuce + performativní význam (dotaz, žádost, ...). „Požádal mě o čaj.“
- **Perlokuční akt** (*perlocutionary act*) — *co se skutečně stalo*; efekt řečového aktu. „Přiměla mě, abych jí uvařil čaj.“

Pro MAS je klíčová **ilokuční** část — ta je obsahem zprávy (performativa).

Searle rozpracoval podmínky, za nichž je řečový akt „zdařilý“. Pro akt *MLUVČÍ požádá POSLUCHAČE o akci*:

- **Standardní vstupně-výstupní podmínky**: posluchač slyší, nejde o scénu ve filmu...



- **Předpoklady** (co musí platit, aby mluvčí mohl tento akt zvolit): posluchač je schopen akci provést; mluvčí věří, že je schopen ji provést; není zřejmé, že by ji posluchač provedl i bez požádání.
- **Upřímnost** (*sincerity*): mluvčí opravdu chce, aby se akce provedla.

#### Searlovy kategorie řečových aktů

Starší dělení: *request, advice, statement, promise, ...*

Novější (a standardní) dělení do pěti tříd:

- **Asertiva** (*assertives*, reprezentativa) — informují posluchače (*inform*).
- **Direktiva** (*directives*) — žádají posluchače o akci (*request*).
- **Komisiva** (*commissives*) — mluvčí se k něčemu zavazuje (*promise*).
- **Expresiva** (*expressives*) — mluvčí vyjadřuje mentální stav, emoci („děkuji“).
- **Deklarace** (*declarations*) — mění stav věcí (válka, sňatek).

Řečový akt má vždy dvě složky: **performativní sloveso** (*request, query, inform*) a **propoziční obsah** („okno je zavřené“).

#### Plánovací teorie řečových aktů (Cohen, Perrault, 1979)

„...modelování [řečových aktů] v plánovacím systému jako operátorů definovaných... pomocí přesvědčení a cílů mluvčích a posluchačů. S řečovými akty se tak zachází stejně jako s fyzickými akcemi.“

Sémantika řečových aktů je tedy dána v duchu **STRIPS** (předpoklady – delete – add), jen se používají modální operátory pro *beliefs, abilities a wants*.

#### Příklad: Request a Inform.

	Request(S,H,A)	Inform(S,H, $\varphi$ )
Předpoklad	$(S \text{ believe } (H \text{ cando } A)) \wedge$ <i>cando</i> $(S \text{ believe } (H \text{ believe } (H \text{ cando } A)))$	$(S \text{ believe } \varphi)$
Předpoklad <i>want</i>	$(S \text{ believe } (S \text{ want requestInstance}))$	$(S \text{ believe } (S \text{ want informInstance}))$
Efekt	$(H \text{ believe } (S \text{ believe } (S \text{ want } A)))$	$(H \text{ believe } (S \text{ believe } \varphi))$

Povšimněte si, že efektem *Inform* není „posluchač uvěří  $\varphi$ “, ale jen „posluchač uvěří, že mluvčí věří  $\varphi$ “. Autonomní agent není povinen věřit tomu, co mu kdo řekne — to je přesně ten rozdíl oproti volání metody.

### 7.3 KQML a FIPA-ACL

#### KQML — Knowledge Query and Manipulation Language

Vznikl v 90. letech v rámci projektu DARPA *Knowledge Sharing Effort* (KSE) společně s KIF.

- **KQML** = *vnější* komunikační jazyk („obálka dopisu“); nese *ilokuční* část zprávy.
- **KIF** = *vnitřní* jazyk; propoziční obsah zprávy, reprezentace znalostí.

Zpráva je seznam *performativa* a pojmenovaných parametrů:

```
(ask-one
 :content (PRICE IBM ?price)
 :receiver stock-server
 :language LPROLOG
 :ontology NYSE-TICKS)
```

**Parametry:** content, force, reply-with, in-reply-to, sender, receiver, language, ontology.

**Performativa:** achieve, advertise, ask-about, ask-one, ask-all, ask-if, break, sorry, error, broadcast, forward, recruit-one, recruit-all, reply, subscribe, tell, ...

**Kritika KQML:** chyběl performativ *commit* (závazek), sémantika nebyla přesně definována, množina performativ byla nesystematická a zbytečně velká, transportní mechanismus nebyl standardizován.

## FIPA-ACL — Agent Communication Language

FIPA (*Foundation for Intelligent Physical Agents*, dnes standardizační komise IEEE) navrhla ACL jako **zjednodušení a zpřesnění KQML**: menší a systematictější množina performativ a *formálně definovaná sémantika*. Praktická implementace: platforma JADE.

**Struktura zprávy** (13 polí — performativ a 12 parametrů; povinné je jen **performative**):

Pole	Význam
performative	typ komunikačního aktu (ilokuční síla) — <i>povinné</i>
sender, receiver, reply-to	identifikátory agentů (AID)
content	vlastní obsah zprávy
language	jazyk obsahu (SL, KIF, ...)
encoding, ontology	kódování a ontologie pro interpretaci obsahu
protocol	interakční protokol, jehož je zpráva součástí
conversation-id, reply-with, in-reply-to	párování zpráv do konverzace
reply-by	termín, do kdy se očekává odpověď

(inform  
:sender agent1  
:receiver agent2  
:content (price good2 150)  
:language sl  
:ontology hpl-auction)

**Performativa FIPA-ACL** (22 celkem, nejdůležitější):

Skupina	Performativa
předávání informace	inform, inform-if, inform-ref, confirm, disconfirm
žádost o informaci	query-if, query-ref, subscribe
žádost o akci	request, request-when, request-whenever, cancel
vyjednávání	cfp ( <i>call for proposals</i> ), propose, accept-proposal, reject-proposal
provádění akcí	agree, refuse, failure
chyby, směrování	not-understood, proxy, propagate

## Sémantika FIPA-ACL: FP a RE

Sémantika je definována v jazyce SL (*Semantic Language*) — modální logice s operátory  $B_i\varphi$  (agent  $i$  věří  $\varphi$ ),  $U_i\varphi$  (je nejistý),  $I_i\varphi$  (má záměr), Feasible, Done. Ke každému performativu jsou dány:

- **FP** (*feasibility precondition*) — co musí platit, aby odesílatel mohl akt provést;
- **RE** (*rational effect*) — efekt, kvůli němuž akt provádí. Příjemce *není povinen* RE naplnit (autonomie!) — RE je jen důvod odesílatele.

Například pro  $\langle i, \text{inform}(j, \varphi) \rangle$ :

$$\text{FP: } B_i\varphi \wedge \neg B_i(Bif_j\varphi \vee Uif_j\varphi), \quad \text{RE: } B_j\varphi.$$

Tedy: „já  $\varphi$  věřím a nevěřím, že už  $j$  o  $\varphi$  ví (že by věděl, zda platí, či neplatí —  $Bif$  — ani že by o tom měl mínění s nejistotou —  $Uif$ ); chci, aby  $j$  uvěřil  $\varphi$ .“ Pro  $\langle i, \text{request}(j, \alpha) \rangle$ : FP:  $FP(\alpha)[i \setminus j] \wedge B_i \text{Agent}(j, \alpha) \wedge \neg B_i I_j \text{Done}(\alpha)$ , RE:  $\text{Done}(\alpha)$ .

**Praktický problém:** tato sémantika je neverifikovatelná — nelze zvenčí ověřit, že agent skutečně věří tomu, co tvrdí (*sincerity assumption*). To je známá slabina mentalistické sémantiky ACL; alternativou jsou *sociální* sémantiky založené na veřejných závazcích (*commitments*).

## 7.4 Interakční protokoly

Jednotlivá zpráva nestačí; agenti vedou **konverzace** s předepsanou strukturou. Interakční protokol (*interaction protocol*) je vzor přípustných posloupností zpráv; FIPA jich standardizuje kolem tuctu (specifikace FIPA-IP, notace AUML).

Protokol	Průběh
<b>FIPA-Request</b>	$I \rightarrow P$ : request. $P \rightarrow I$ : refuse (konec), nebo agree; po vykonání inform-done / inform-result, případně failure. Odpověď může být i not-understood.
<b>FIPA-Query</b>	$I \rightarrow P$ : query-if / query-ref. $P$ : refuse nebo agree, dále inform s výsledkem nebo failure.
<b>FIPA-Contract-Net</b>	viz níže: $\text{cfp} \rightarrow n \times (\text{propose} / \text{refuse}) \rightarrow \text{accept-proposal}$ vítězi + $\text{reject-proposal}$ ostatním $\rightarrow \text{inform-done} / \text{failure}$ .
<b>FIPA-Iterated-Contract-Net</b>	jako CNET, ale iniciátor může po přijetí nabídek vyhlásit <i>další kolo cfp</i> (vyjednávání o lepších podmínkách).
<b>FIPA-Subscribe</b>	$I$ : subscribe; $P$ posílá inform pokaždé, když se sledovaná hodnota změní, dokud nepřijde cancel.
<b>FIPA-Brokering Recruiting</b>	/ $I$ požádá zprostředkovatele ( <i>broker</i> ), ten najde vhodné poskytovatele, deleguje na ně požadavek a výsledky vrátí (příp. je nechá odpovědět přímo).
<b>Aukční protokoly</b>	FIPA-English-Auction, FIPA-Dutch-Auction (viz kap. 11).

**Poznámka k návrhu.** Protokol určuje, jaké zprávy jsou v daném stavu konverzace legální — tím zavádí *předvídatelnost* do jinak autonomního chování a umožňuje implementovat agenta jako konečný automat nad stavy konverzace (přesně to dělá JADE třídami typu AchieveREInitiator, ContractNetResponder apod.).

## 7.5 Shrnutí ke zkoušce

- Komunikace agentů je *akce*, ne volání metody — příjemce není povinen vyhovět.
- Austin: řečové akty; lokuce / ilokuce / perlokuce; Searle: podmínky zdařilosti a pět kategorií (asertiva, direktiva, komisiva, expresiva, deklarace); performativní sloveso + propoziční obsah.
- Cohen & Perrault: sémantika řečových aktů ve stylu STRIPS (pre / efekt) s modálními operátory; umět napsat Request(S,H,A) a Inform(S,H, $\varphi$ ) — efektem Inform je  $H$  věří, že  $S$  věří  $\varphi$ .
- KQML (obálka) + KIF (obsah); parametry a performativa; kritika (chybí commit, nejasná sémantika).
- FIPA-ACL: struktura zprávy (performative, sender/receiver, content, language, ontology, protocol, conversation-id, reply-by), 22 performativ, sémantika přes FP a RE v jazyce SL; problém neverifikovatelnosti.
- Protokoly: FIPA-Request, FIPA-Query, Contract Net (+ iterovaný), Subscribe, Brokering, aukční protokoly.

## 8 Distribuované řešení problémů a kooperace

### 8.1 Koherence a koordinace

Agenti mají různé cíle, jsou autonomní, pracují v čase a rozhodují se za běhu; musí být schopni *dynamické koordinace*. Sdílejí přitom dvě věci: **úlohy** (*task sharing*) a **informace** (*result sharing*).

#### Koherence a koordinace

**Koherence** (*coherence*) — jak dobře systém funguje *jako celek* (kvalita řešení, efektivita využití zdrojů, degradace při poruchách).

**Koordinace** (*coordination*) — jak dobře se agentům daří *minimalizovat režii* spojenou se synchronizací: vyhnout se zbytečně duplicitní práci, konfliktům o zdroje a zbytečné komunikaci. Systém může být koherentní i při špatné koordinaci (agenti udělají práci třikrát, ale výsledek je správný) — za cenu plýtvání.

#### CDPS — Cooperative Distributed Problem Solving

Lesser a kol., 80. léta. Kooperace jednotlivých agentů při řešení problému, který přesahuje jejich individuální schopnosti (informace, zdroje, výpočetní kapacitu).

Klíčový předpoklad: agenti *implicitně sdílejí společný cíl* a nemají konflikty — **benevolence**. Měřítkem úspěchu je *výkon systému jako celku*; agent pomáhá celku, i když je to pro něj samotného nevýhodné. Benevolence *enormně zjednodušuje* návrh systému.

	Charakteristika	Rozdíl
<b>PPS</b> ( <i>parallel problem solving</i> )	Bond, Gasser, 80. léta; důraz na paralelní řešení, homogenní a jednoduché procesory	agenti nejsou autonomní, jde v podstatě o paralelní algoritmus
<b>CDPS</b>	heterogenní agenti se sofistikovanými schopnostmi, sdílený cíl, benevolence	konflikty se neřeší, protože nevznikají
<b>MAS</b>	společnost agentů s <i>vlastními</i> cíli, které <i>nesdílejí</i>	je nutné řešit: proč a jak kooperovat, jak identifikovat a řešit konflikty, jak vyjednávat a smlouvat

## 8.2 Sdílení úloh a sdílení výsledků

Obecný postup CDPS má tři fáze:

1. **Dekompozice problému** — hierarchická, rekurzivní. Otázky: *jak* rozložit a *kdo* rozklad provede. Krajní varianta — model ACTORS: pro každý podproblém vznikne nový agent, až na úrovni jednotlivých instrukcí.
2. **Řešení podproblémů** — agenti si během něj typicky sdílejí informace a mohou potřebovat synchronizovat akce.
3. **Syntéza řešení** — opět hierarchická, skládání dílčích výsledků.

**Sdílení úloh** vyžaduje *dohodu* agentů (kdo co udělá) — typickým mechanismem je Contract Net. **Sdílení výsledků** může být *proaktivní* (agent pošle výsledek, o němž si myslí, že se bude hodit) nebo *reaktivní* (pošle ho na vyžádání).

### Přínosy sdílení výsledků

- **Jistota** (*confidence*) — nezávislá řešení téhož problému lze porovnat; shoda zvyšuje důvěru ve výsledek.
- **Úplnost** (*completeness*) — agenti sdílejí své *lokální* pohledy a tím vzniká globálnější představa o problému.
- **Přesnost** (*precision*) — sdílení může zpřesnit celkové řešení.
- **Včasnost** (*timeliness*) — zjevná výhoda distribuce: zkrácení času.

## 8.3 Contract Net

### Contract Net Protocol (CNET)

Smith a Davis, 1977–1980. Metafora **sdílení úloh pomocí smluv** podle vzoru trhu: agent v nouzi je *manažer* (*manager*), ostatní jsou *dodavatelé* (*contractors*). Čtyři fáze:

1. **Rozpoznání** (*recognition*) — agent zjistí, že má problém, který nezvládne sám (chybí mu schopnost, zdroje, informace, nebo by to bylo horší řešení).
2. **Ohlášení** (*announcement*) — rozešle (typicky broadcastem) oznámení úlohy: *popis úlohy* (může být i spustitelný), *omezení* (termín, cena, kvalita) a *meta-informace* (kdo se smí hlásit).
3. **Nabídky** (*bidding*) — příjemci se rozhodnou, zda mají zájem a schopnosti, a pošlou nabídku (*tender*) s cenou / časem / kvalitou.
4. **Přidělení a realizace** (*awarding, expediting*) — manažer vybere vítěze, uzavře s ním smlouvu; ten úlohu vykoná a vrátí výsledek.

**Vlastnosti:** jednoduchý, decentralizovaný, robustní vůči výpadkům; dodavatel se může sám stát manažerem podúlohy, což vede k *hierarchickým kaskádám subkontraktů*. Dynamicky rozděluje zátěž. Je nejčastěji implementovaným koordinačním mechanismem vůbec (v FIPA je standardi-

zován jako FIPA-Contract-Net; v JADE jako hotové chování).

**Limity:** broadcast je drahý při mnoha agentech; předpokládá benevolenci (dodavatel nelže o svých schopnostech) — jinak je nutné použít aukční mechanismy odolné vůči manipulaci (kap. 11).

## 8.4 Systémy s tabulí (blackboard)

### Blackboard system (BBS)

Vůbec první schéma pro kooperativní řešení problémů (HEARSAY-II, rozpoznávání řeči, 70. léta). Výsledky se sdílejí přes společnou datovou strukturu — **tabuli** (*blackboard*).

Metafora: kolem tabule sedí několik expertů, kteří na ni mohou psát i z ní číst. Na tabuli se dynamicky objevují úlohy; jakmile některý expert vidí úlohu, kterou umí řešit, zapíše na tabuli částečné řešení. Takto se pokračuje, dokud se na tabuli neobjeví řešení celé úlohy.

**Role:**

- **Tabule** — sdílená paměť; typicky strukturovaná do několika *úrovní abstrakce* (hierarchie cílů a akcí). Zásadní je formalismus reprezentace informace.
- **Experti** (*knowledge sources*) — agenti řešící konkrétní typ úlohy; reagují na cíle na tabuli, svůj cíl přebírají z tabule a po vybrání vykonají akci. Mají seznam schopností a efektivitu.
- **Arbitr** (*arbiter, control component*) — vybírá, který expert smí k tabuli. Může být čistě reaktivní, nebo může uvažovat plány a maximalizovat očekávanou utilitu. Odpovídá za řešení problému na vyšší úrovni.

**Nutnost vzájemného vyloučení** nad tabulí — úzké hrdlo systému.

### Klady a zápory BBS:

- + Jednoduchý mechanismus koordinace, kooperace, sdílení úloh i výsledků.
- + Experti o sobě *nemusí vědět* a přesto spolupracují (*loose coupling*); přidání experta nevyžaduje změnu ostatních.
- + Zprávy na tabuli lze přepisovat — delegovat úlohy, vytvářet podúlohy, měnit experty.
- Všichni agenti musí mít přístup k tabuli a sdílet stejnou architekturu; kolem tabule bývá „narováno“ (částečně řeší distribuované hašovací tabulky).

**Příklad: BBWar.** Tabule je hašovací tabulka mapující požadované schopnosti na úlohy; „otevřené mise“ jsou publikovány na tabuli. Experti tvoří hierarchii: agent *velitel* hledá úlohy typu „dobyj město“ (ATTACK-CITY) a převádí je na několik úloh „zaútoč na pozici“ (ATTACK-LOCATION); vojáci různých typů si pak z tabule vybírají mise, na které mají schopnosti.

**Příklad: FELINE** (Wooldridge, Jennings, 1990) — distribuovaný expertní systém z doby před KQML a ACL. Sdílení znalostí a distribuce podúloh; každý agent je pravidlový systém, který si udržuje: *Skills* — „umím dokázat/vyvrátit následující...“ a *Interests* — „zajímá mě, zda následující platí...“. Komunikace je trojice (odesílatel, příjemce, obsah), kde obsah je hypotéza + řečový akt (request, response, inform).

## 8.5 Distribuované omezující podmínky (DisCSP a DCOP)

CNET i blackboard jsou *protokoly*. Vedle nich stojí formální větve distribuovaného řešení problémů, kde je úloha zadána jako soustava omezení rozprostřená mezi agenty.

### DisCSP a DCOP

**Distribuovaný CSP** (*distributed constraint satisfaction problem*): proměnné  $x_1, \dots, x_n$  s doménami  $D_i$  jsou rozděleny mezi agenty (typicky jedna proměnná na agenta) a omezení propojují proměnné *různých* agentů. Hledá se přiřazení splňující všechna omezení; žádný agent přitom nevidí celý problém a komunikuje jen se sousedy v grafu omezení.



**DCOP** (*distributed constraint optimization*) je optimalizační varianta: omezení jsou *měkká*, tj. zadaná funkcemi ceny  $f_{ij}(x_i, x_j)$ , a maximalizuje se  $\sum_{ij} f_{ij}$  — tedy sociální blahobyť (kap. 9).

**Algoritmy:** *ABT* (*asynchronous backtracking*, Yokoo) — agenti jsou uspořádáni, posílají si zprávy *ok?* (své přiřazení níž) a *nogood* (zpět nahoru při konfliktu), přičemž naučené *nogoods* zabraňují cyklení; *ADOPT* (asynchronní, s mezemi, optimální), *DPOP* (dynamické programování nad pseudostromem — lineárně mnoho zpráv, ale exponenciálně velké) a *Max-Sum* (šíření zpráv nad faktorovým grafem, přibližné, ale velmi škálovatelné).

**Typické aplikace:** rozvrhování schůzek, přidělování sledovacích úkolů senzorům, alokace frekvencí, koordinace při MAPF. Na rozdíl od CDPS zde *není* nutná benevolence: DCOP se dá kombinovat s mechanismy z kap. 11.

## 8.6 Společné záměry a sociální zákony

Co přesně znamená, že tým agentů něco dělá *společně*? Odpovědí je multiagentní zobecnění commitmentu z kap. 3.5:

### Teorie společných záměrů (*joint intentions*, Cohen–Levesque)

Tým  $G$  má **společný trvalý cíl** (*joint persistent goal*)  $\varphi$ , pokud: (1) všichni členové věří, že  $\varphi$  zatím neplatí, a chtějí, aby platil; (2) je to mezi nimi vzájemně známo; (3) každý člen drží cíl, dokud se nedozví, že  $\varphi$  je *splněn*, *nesplnitelný*, nebo *irelevantní* — a kdo to zjistí, přebírá **závazek dát to vědět ostatním** (stav týmu se stává jeho individuálním cílem oznámit). Právě komunikační povinnost v bodě (3) odlišuje týmovou činnost od náhodné shody individuálních záměrů (Searlův příklad: běžci, kteří se před deštěm rozeběhnou ke stejnému altánu, nejsou tým). Na teorii staví praktické modely týmové spolupráce: Jenningsův model *společných závazků* (*joint commitments*) s *konvencemi* — explicitními pravidly, kdy smí agent závazek opustit a koho musí informovat — a systém **STEAM** (Tambe); alternativní formalizací je *SharedPlans* (Grosz–Kraus).

**Sociální zákony a konvence** (Shoham–Tennenholtz): koordinace nemusí probíhat za běhu komunikací — lze ji zajistit *offline* pravidly, která předem omezují akce všech agentů (vzorový příklad: „jezdí vpravo“). Sociální zákon je *užitečný*, pokud každému agentovi ponechává možnost dosáhnout jeho cílů; syntéza užitečného sociálního zákona je obecně NP-těžká. Konvence mohou také *emergovat* zdola opakovanou interakcí (srov. Axelrodovy turnaje, kap. 9). Wooldridge shrnuje čtyři základní koordinační techniky: **organizační strukturování** (role a hierarchie, srov. Gaia, kap. 13), **kontraktování** (CNET), **multiagentní plánování** a **vyjednávání** (kap. 11).

## 8.7 Interakce agentů v prostředí

Agenti si mohou navzájem pomáhat i překážet, a to i *bez komunikace*, prostě tím, že ovlivňují prostředí:

- Roboti mohou přenést cihlu, jen když tlačí ze dvou stran současně (pozitivní interakce, vyžaduje synchronizaci).
- Roboti se natlačí ke dveřím a nemohou je otevřít (negativní interakce, vyžaduje koordinaci).

Agenti mohou vytvářet společenství, hierarchie i nepřátelství. **Je nutné znát typy interakcí** v konkrétním MAS — jinak nelze navrhnout efektivní řídicí mechanismy. Nejjednodušší modelový případ, kterým se zabývá následující kapitola, je interakce dvou racionálních (sobeckých) agentů v prostředí připomínajícím hru.

## 8.8 Shrnutí ke zkoušce

- Koherence (výkon celku) vs. koordinace (minimalizace synchronizační režie).



- CDPS: benevolence + sdílený cíl; odlišit od PPS (homogenní procesory, paralelní algoritmus) a od MAS (vlastní cíle, konflikty, vyjednávání).
- Tři fáze CDPS: dekompozice — řešení podproblémů — syntéza. Sdílení úloh (dohoda) vs. sdílení výsledků (proaktivní/reaktivní) a jeho čtyři přínosy: jistota, úplnost, přesnost, včasnost.
- CNET: recognition — announcement — bidding — awarding/expediting; manažer a dodavatel, hierarchické subkontrakty; standardizován jako FIPA-Contract-Net.
- BBS: tabule (hierarchie abstrakce, mutex), experti (skills), arbitr (výběr experta); volná vazba mezi experty; úzké hrdlo přístupu k tabuli.
- DisCSP / DCOP: proměnné a omezení rozdělené mezi agenty, nikdo nevidí celý problém; ABT (ok? / nogood), ADOPT, DPOP, Max-Sum; DCOP maximalizuje sociální blahobyt.
- Společný trvalý cíl (joint persistent goal): tým drží cíl, dokud není splněn / nesplnitelný / irelevantní, a kdo to zjistí, musí to oznámit ostatním; sociální zákony = offline koordinace omezením akcí („jezdi vpravo“).

## 9 Interakce agentů a teorie her

### 9.1 Sobecký agent

Proč by měl být agent upřímný ohledně svých schopností? Proč by měl dokončit přidělenou úlohu? Je-li systém homogenní (celý navržený námi), je benevolence dobrá strategie. Většinou ale systém obsahuje agenty s různými zájmy — konflikt mezi společným cílem a cíli jednotlivců (např. řízení letového provozu: všichni chtějí bezpečnost, ale každá aerolinka chce přistát první). Někdy ani není zřejmé, co je společný zájem. Pak je lépe uvažovat **sobecké** (*self-interested*) agenty.

#### Sobecký agent

Množina možných výsledků  $O = \{o_1, o_2, \dots\}$  je *společná* pro všechny agenty, ale preference nikoli: každý agent  $i$  má vlastní **utilitní funkci**  $u_i : O \rightarrow \mathbb{R}$ , která výsledky uspořádá.

**Strategické myšlení:** maximalizuj svou utilitu za předpokladu, že všichni ostatní jednájí rovněž racionálně. Toto *nemaximalizuje* společnou utilitu, ale je to *robustní* strategie.

*Poznámka:* peníze nejsou dobrou utilitou pro člověka — utilitní funkce peněz je nelineární a liší se pro bohaté a chudé.

### 9.2 Hra v normální formě

#### Hra dvou hráčů v normální formě

Agenti  $i$  a  $j$  s množinami akcí  $A_i, A_j$ . Výsledek je dán oběma:  $e : A_i \times A_j \rightarrow O$ . Agent volí **strategii**  $s_i$ :

- **ryzí** (*pure*) — deterministická volba jedné akce;
- **smíšená** (*mixed*) — rozdělení pravděpodobnosti nad ryzími strategiemi (náhodný výběr).

Hru zapisujeme **výplatní maticí** (*payoff matrix*): řádky = strategie hráče  $i$ , sloupce = strategie hráče  $j$ , buňka obsahuje dvojici  $u_i/u_j$ .

Obecně: hra v normální formě je trojice  $\langle N, (A_i)_{i \in N}, (u_i)_{i \in N} \rangle$ , kde  $N$  je množina hráčů.

#### Dominance

Strategie  $s_i$  **silně dominuje** strategii  $s'_i$ , pokud dává agentovi  $i$  *striktně* lepší výsledek proti *všem* strategiím protihráče; **slabě dominuje**, pokud dává výsledek alespoň stejně dobrý proti všem a striktně lepší alespoň proti jedné (tak dominanci definuje i přednáška). Strategie je **dominantní**, pokud dominuje všem ostatním strategiím agenta  $i$ .

Rozdíl není akademický: pravdomluvnost ve Vickreyově aukci je jen *slabě* dominantní (kap. 11). Racionální agent hraje dominantní strategii, existuje-li. *Iterované odstraňování dominovaných strategií* (IESDS) je základní řešící technika: dominovanou strategii nikdy nikdo nezahraje, lze ji tedy z matice vyškrtnout, což může učinit dominovanými další strategie. Při odstraňování *silně*

dominovaných strategií nezáleží na pořadí a výsledek je jednoznačný; u *slabě* dominovaných strategií na pořadí **záleží** a lze přijít o některá ekvilibria.

### Nashovo ekvilibrium

Dvojice strategií  $(s_1^*, s_2^*)$  je v **Nashově ekvilibriu**, pokud:

- hraje-li agent  $i$  strategii  $s_1^*$ , je pro agenta  $j$  nejlepší hrát  $s_2^*$ , a zároveň
- hraje-li agent  $j$  strategii  $s_2^*$ , je pro agenta  $i$  nejlepší hrát  $s_1^*$ .

Tedy  $s_1^*$  a  $s_2^*$  jsou *vzájemně nejlepší odpovědi* (*mutual best responses*). Obecně pro  $n$  hráčů: profil  $s^* = (s_1^*, \dots, s_n^*)$  je NE, pokud pro každého  $i$  a každou jeho strategii  $s_i$  platí

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*),$$

kde  $s_{-i}$  značí strategie všech ostatních. **Žádný hráč nemá jednostranný důvod se odchýlit.** Naivní hledání ekvilibrií pro  $n$  agentů a  $m$  strategií vyžaduje projít  $m^n$  profilů. Tato definice se týká *ryzích* strategií — ne každá hra má NE v ryzích strategiích (kámen–nůžky–papír) a některé hry jich mají více.

**Věta 9.1** (Nashova věta, 1950). *Každá hra s konečným počtem hráčů a konečným počtem strategií má alespoň jedno Nashovo ekvilibrium ve smíšených strategiích.*

**Výpočetní složitost.** Hledání NE je *totální* vyhledávací problém (řešení vždy existuje, jde jen o to ho najít). Od roku 2006 víme, že je **PPAD-úplný** — velmi zjednodušeně řečeno „o něco méně beznadějně než NP-úplnost“ (NP-úplnost pro problém, který má vždy řešení, ani nedává smysl). Pro tři a více hráčů to dokázali Daskalakis, Goldberg a Papadimitriou; těsně poté Chen a Deng doplnili i zbývající a nejtěžší případ *dvou* hráčů.

### Paretova efektivita a sociální blahobyť

Profil strategií je **Pareto-optimální** (*Pareto efficient*), pokud neexistuje jiný profil, který by zlepšil výsledek jednoho agenta, aniž by zhoršil výsledek některého jiného. Ne-Pareto-optimální řešení lze tedy zlepšit „zadarmo“.

**Sociální blahobyť** (*social welfare*) profilu je součet utilit všech agentů:  $sw(o) = \sum_{i \in N} u_i(o)$ . Maximalizace sociálního blahobytu dává smysl v kooperativních scénářích — když jsou agenti z jednoho týmu, mají jednoho vlastníka, řeší jednu úlohu; čím homogennější systém, tím lépe. Maximum sociálního blahobytu je vždy Pareto-optimální, opačně to neplatí.

## 9.3 Klasické hry

### Věžňovo dilema (*Prisoner's dilemma*)

$i \backslash j$	$C$ (spolupráce)	$D$ (zrada)
$C$	3/3	0/4
$D$	4/0	1/1

$C = cooperate$ , spolupracovat se spolupachatelem;  $D = defect$ , udat ho a dostat nižší trest. Kanonické značení výplat:  $R$  (*reward* za oboustrannou spolupráci),  $P$  (*punishment* za oboustrannou zradu),  $T$  (*temptation* — zradím spolupracujícího),  $S$  (*sucker's payoff* — jsem zrazen). Podmínky věžňova dilematu:

$$T > R > P > S \quad \text{a často navíc} \quad 2R > T + S.$$

- $R > P$ : oboustranná spolupráce je lepší než oboustranná zrada.
- $T > R$  a  $P > S$ : zrada je dominantní strategie pro oba agenty.
- $(D, D)$  je **jediné Nashovo ekvilibrium**.
- $(D, D)$  je zároveň **jediné ne-Pareto-optimální** řešení hry.
- $(C, C)$  maximalizuje sociální blahobyť ( $3 + 3 = 6$ ).

**Racionalita každého jednotlivce tedy vede ke kolektivně nejhoršímu výsledku.**

### Důsledky věžňova dilematu:

- **Tragédie obecní pastviny** (*tragedy of the commons*): sdílený zdroj (pastvina, rybolov, atmosféra) je vyčerpán, protože každý jednotlivec maximalizuje svůj užitek.
- Otázka, *co vlastně znamená být racionální* — a zda jsou lidé racionální (experimentálně lidé spolupracují častěji, než teorie předpovídá).
- **Stín budoucnosti** (*shadow of the future*): v *iterovaném* věžňově dilematu se situace mění.

### Iterované věžňovo dilema a Axelrodovy turnaje

Hraje-li se PD *konečněkrát* a *oba to vědí*, zpětnou indukcí vyjde  $D$  v každém kole (v posledním kole není důvod spolupracovat, tedy ani v předposledním, ...). Hraje-li se *nekonečně* (nebo s neznámým koncem, tj. s pravděpodobností pokračování), stává se spolupráce racionální — *folk theorem*: každý výplatní profil lepší než minimax lze podepřít jako ekvilibrium opakované hry. Robert Axelrod uspořádal v roce 1980 turnaje strategií pro iterované PD. Vyhrála (dvakrát) nejjednodušší přihlášená strategie **Tit For Tat** (TFT, „půjčka za oplátku“): *v prvním kole spolupracuj, pak vždy zopakuj poslední tah soupeře*. Axelrod z výsledků odvodil čtyři vlastnosti úspěšných strategií:

- **slušnost** (*nice*) — nezrazuj jako první;
- **odvetnost** (*retaliating*) — na zradu okamžitě odpověz;
- **odpouštění** (*forgiving*) — po návratu soupeře ke spolupráci mu odpust;
- **nezávistivost** (*non-envious*) — neusiluj o to porazit *konkrétního* soupeře; TFT nikdy nezískal víc bodů než jeho protihráč, a přesto vyhrál turnaj.

Jako pátou vlastnost Axelrod uvádí **srozumitelnost** (*clarity*): strategie musí být pro protihráče čitelná, jinak s ní nelze navázat spolupráci. Slabina TFT: při šumu (chybně provedený tah) se dva TFT hráči zaseknou ve vzájemné odvetě — proto varianty *Generous TFT* (občas odpustí) a *Tit for Two Tats*.

### Koordináční hra

$i \backslash j$	Balet	Zápas
Balet	1/2	0/0
Zápas	0/0	2/1

Známa též jako *Battle of the Sexes* (BoS) nebo *Bach or Stravinsky*. Výplaty podporují *shodu* strategií, hráči se ale liší v tom, na které shodě jim záleží víc.

- Nashova ekvilibria v ryzech strategiích: (Balet, Balet) a (Zápas, Zápas).
- Sociální blahobyť i Paretova optimalita: opět obě tyto dvojice.

- **Smíšené NE:** každý hráč hraje svou *preferovanější* možnost s pravděpodobností  $2/3$  a méně preferovanou s  $1/3$ . Očekávaná utilita ve smíšeném ekvilibriu je jen  $2/3$  — méně než v kterémkoli ryzím ekvilibriu.

*Výpočet smíšeného NE (obecný recept pro hry  $2 \times 2$ ):* hráč  $j$  volí pravděpodobnost  $q$  hraní Baletu tak, aby byl hráč  $i$  *indiferentní* mezi svými akcemi:

$$u_i(\text{Balet}) = 1 \cdot q + 0 \cdot (1 - q), \quad u_i(\text{Zápas}) = 0 \cdot q + 2 \cdot (1 - q).$$

Rovnost  $q = 2 - 2q$  dává  $q = 2/3$ , tj. hráč  $j$  hraje Balet (svou preferovanou volbu) s pravděpodobností  $2/3$ . Symetricky hráč  $i$  hraje Zápas s pravděpodobností  $2/3$ . Očekávaná utilita hráče  $i$  je pak  $1 \cdot \frac{2}{3} = \frac{2}{3}$ . **Klíčové pravidlo: ve smíšeném NE volí každý hráč pravděpodobnosti tak, aby byl protihráč indiferentní.**

#### Antikoordinační hra

$i \backslash j$	Chicken (uhnu)	Dare (nedám se)
Chicken	5/5	1/6
Dare	6/1	0/0

Známá též jako *Chicken*, *Hawk–Dove* nebo *Snowdrift*. Výplaty podporují volbu *různých* strategií. V koordinačních hrách přináší sdílení zdroje prospěch všem; v antikoordinačních hrách je sdílení nákladné (srážka aut).

- Nashova ekvilibria v ryzích strategiích: (Chicken, Dare) a (Dare, Chicken).
- Maximum sociálního blahobytu: (Chicken, Chicken) —  $5 + 5 = 10$ , což *není* NE (oběma se vyplatí odchýlit se na Dare:  $6 > 5$ ).
- **Symetrické smíšené NE:** hraje-li protihráč Chicken s pravděpodobností  $p$ , je  $u_i(\text{Chicken}) = 5p + 1(1 - p) = 1 + 4p$  a  $u_i(\text{Dare}) = 6p$ ; z indiference  $1 + 4p = 6p$  plyne  $p = 1/2$ . Oba tedy hrají obě akce s pravděpodobností  $1/2$  a očekávaná utilita je  $3$  — méně než  $5$  při (Chicken, Chicken), ale nikdo se nemůže jednostranně zlepšit.

**Doplňek: hra bez ryzího NE.** *Matching pennies* (a stejně tak kámen–nůžky–papír) je hra s nulovým součtem:

$i \backslash j$	Hlava	Orel
Hlava	1/−1	−1/1
Orel	−1/1	1/−1

Žádné ryzí NE neexistuje. Smíšené NE: oba hrají  $(1/2, 1/2)$ , očekávaná utilita  $0$ . Motiv je obdobný jako u Condorcetova paradoxu z následující kapitoly: individuálně racionální volby nemají „stabilní“ deterministický výsledek.

## 9.4 Kooperativní (koaliční) hry

Vše dosavadní byly hry **nekooperativní**: základní jednotkou je *jednotlivec* a jeho strategie, dohody nejsou vynutitelné. **Kooperativní** teorie her mění jednotku na *skupinu* a ptá se: kdo se s kým spojí a jak si rozdělí společný zisk. Pro MAS je to formální model tvorby týmů a dělení odměny.

#### Koaliční hra s přenositelnou utilitou

Dvojice  $\langle N, v \rangle$ , kde  $N$  je množina agentů a **charakteristická funkce**  $v : 2^N \rightarrow \mathbb{R}$  přiřazuje každé koalici  $C \subseteq N$  hodnotu, kterou si její členové umějí zajistit vlastními silami ( $v(\emptyset) = 0$ ). Utilita je *přenositelná*: hodnotu  $v(C)$  lze mezi členy libovolně rozdělit („penězi“).

Hra je **superaditivní**, pokud  $v(C \cup D) \geq v(C) + v(D)$  pro disjunktní  $C, D$  — pak se vždy vyplatí vytvořit *velkou koalici*  $N$  a zbývá jen otázka dělení.

**Řešení** je vektor  $(x_1, \dots, x_n)$  s  $\sum_i x_i = v(N)$  (*feasibility*).

## Jádro a Shapleyho hodnota

**Jádro** (*the core*) — množina rozdělení, která *žádná* koalice nechce opustit:

$$\text{core} = \left\{ x : \sum_{i \in N} x_i = v(N) \wedge \forall C \subseteq N : \sum_{i \in C} x_i \geq v(C) \right\}.$$

Jde o obdobu Nashova ekvilibria pro skupiny (*stabilita*). Jádro ale může být **prázdné** — pak žádné stabilní rozdělení neexistuje — a může být i velmi velké, takže neříká, které rozdělení je *férové*.

**Shapleyho hodnota** (1953) — odpověď na otázku férovosti. Agent dostane svůj *průměrný mezní příspěvek* přes všechna pořadí, v nichž se koalice může skládat:

$$\varphi_i(v) = \sum_{C \subseteq N \setminus \{i\}} \frac{|C|! (|N| - |C| - 1)!}{|N|!} (v(C \cup \{i\}) - v(C)).$$

Je to *jediné* rozdělení splňující současně: efektivitu ( $\sum_i \varphi_i = v(N)$ ), symetrii (zaměnitelní agenti dostanou totéž), *dummy* (agent s nulovým mezním příspěvkem dostane nic) a aditivitu.

*Příklad (rukavice)*: agenti 1 a 2 mají levou rukavici, agent 3 pravou; pár má hodnotu 1, jednotlivá rukavice 0. Tedy  $v(\{1, 3\}) = v(\{2, 3\}) = v(N) = 1$ , jinak 0. Jádro obsahuje *jediný* bod  $(0, 0, 1)$  — agent 3 si vezme vše, protože o něj mají 1 a 2 nouzi. Shapleyho hodnota je  $(1/6, 1/6, 2/3)$  — férovější, ale nestabilní.

**Výpočetní stránka** (proč to zajímá MAS): charakteristická funkce má  $2^n$  hodnot, takže už její *reprezentace* je problém (používají se kompaktní jazyky, hry nad grafy); rozhodnout neprázdnost jádra i spočítat Shapleyho hodnotu je obecně těžké, a úloha *coalition structure generation* (najít optimální rozklad na koalice, když hra není superaditivní) je NP-těžká.

## 9.5 Shrnutí ke zkoušce

- Sobecký agent maximalizuje *svoji* očekávanou utilitu za předpokladu, že ostatní dělají totéž; společná utilita se tím nemaximalizuje.
- Hra v normální formě, ryzí vs. smíšené strategie, dominance a dominantní strategie, iterované odstraňování dominovaných strategií.
- **Nashovo ekvilibrium** = vzájemně nejlepší odpovědi, nikdo nemá důvod se jednostranně odchýlit; naivní hledání  $m^n$ ; **Nashova věta** (existence ve smíšených strategiích); hledání je PPAD-úplné.
- **Paretova optimalita** (nelze zlepšit jednomu, aniž by se jinému přitížilo) vs. **sociální blahobyt** (součet utilít); maximum sw je Pareto-optimální, obráceně ne.
- **Vězňovo dilema**:  $T > R > P > S$ ;  $D$  dominantní;  $(D, D)$  jediné NE a jediné ne-Pareto-optimální;  $(C, C)$  maximalizuje sw. Tragedy of the commons. Iterované PD, stín budoucnosti, Axelrod a TFT (slušná, odvetná, odpouštějící, srozumitelná).
- **Koordinační hra** (BoS): dvě ryzí NE + smíšené  $2/3$ – $1/3$  s horší utilitou; **antikoordinační** (Chicken): dvě asymetrická NE, sw maximalizuje ne-ekvilibriový profil.
- Umět spočítat smíšené NE pro  $2 \times 2$ : pravděpodobnosti volím tak, aby byl *protihráč* indiferentní.
- **Kooperativní hry**: charakteristická funkce  $v : 2^N \rightarrow \mathbb{R}$ , superaditivita a velká koalice; **jádro** (žádná koalice nechce odejít, může být prázdné) vs. **Shapleyho hodnota** (průměrný mezní příspěvek; efektivita, symetrie, dummy, aditivita). Příklad s rukavicemi: jádro  $(0, 0, 1)$ , Shapley  $(1/6, 1/6, 2/3)$ .

## 10 Sociální volba a volby

### 10.1 Agregace preferencí

Předchozí kapitola řešila, jak se agenti chovají, mají-li protichůdné zájmy. Nyní se ptáme opačně: jak z individuálních preferencí *odvodit* kolektivní rozhodnutí. To je předmětem **teorie sociální volby** (*social choice theory*).

#### Sociální blahobyť vs. sociální volba

**Funkce sociálního blahobytu** (*social welfare function*) agreguje individuální preference do úplného uspořádání kandidátů  $>_{sw}$ .

**Funkce sociální volby** (*social choice function*) vybírá jen *jednoho* vítěze (nebo množinu vítězů).  
*Příklad:* voliči  $A : o_2 > o_1 > o_3$ ,  $B : o_3 > o_2 > o_1$ ,  $C : o_2 > o_3 > o_1$ . Sociální blahobyť:  $o_2 > o_3 > o_1$ . Sociální volba:  $o_2$ .

#### Hlasovací lístek a preferenční plán

**Hlasovací lístek** (*ballot*) — jak volič vyjadřuje své preference. Tradičně: „vyber svého favorita ze seznamu“. **Preferenční lístek** (*preference ballot*) — volič seřadí *všechny* možnosti podle preference.

Jednotlivé lístky se sloučí do **preferenčního plánu** (*preference schedule*); z něj **volební schéma** (*voting scheme*) určí vítěze.

### 10.2 Volební systémy

- **Většinový** (*majority*) — vítěz získá více než 50 % hlasů; pro dva kandidáty triviální.
- **Prostá většina** (*plurality*) — kandidát dostane bod za každé první místo, vyhrává ten s nejvíce body; v praxi lístek obsahuje jen první volbu, při rovnosti následuje *run-off* dvou nejlepších.
- **Prostá většina s vyřazováním** (*plurality with elimination, instant runoff*) — opakovaně vyřadí kandidáta s nejméně prvními místy a *přerozděl* jeho hlasy na další volbu příslušných voličů, dokud vítěz nemá většinu. Využívá už zaznamenané preference bez druhého kola a potlačuje neupřímné hlasování (Australská sněmovna, MOV).
- **Sekvenční většinové volby** (*sequential majority*) — kandidáti hrají v dvojicích turnajová kola, vítěz postupuje (pavouk). *Pořadí zápasů ovlivňuje výsledek* (agenda setting).
- **Condorcetova metoda** — simuluj *všechny* párové souboje; drahé a nemusí dát vítěze.
- **Bordova metoda** (*Borda count*) — každý volič odevzdá úplné preference, kandidát dostane  $N-1$  bodů za první místo,  $N-2$  za druhé, ..., 0 za poslední, body se sečtou. Jde spíše o *konsenzuální* než většinové hlasování (Slovinsko, Nauru, Island, akademické instituce).
- **Slaterův systém** — z párových většinových soubojů sestav turnajový graf a hledej *acyklické* uspořádání minimalizující počet obrácených hran. **NP-těžké**.
- **Kombinace s Bordou** (všechny dávají Condorcetova vítěze, existuje-li): *Black* (Condorcetova metoda, jinak Bordův vítěz); *Baldwin* (opakovaně vyřadí kandidáta s nejnižším Bordovým skóre a přepočítá); *Nanson* (opakovaně vyřadí všechny kandidáty pod průměrem).

**Vítěz prosté většiny nemusí být vítězem většinovým.** Příklad:  $o_1$  — 40 %,  $o_2$  — 30 %,  $o_3$  — 30 %. Vítězí  $o_1$ , ale 60 % voličů ho nechtělo. Vede to na **taktické (neupřímné) hlasování** (*insincere / tactical voting*): jsou-li  $o_2$  a  $o_3$  podobní (ze stejné strany), budou voliči  $o_2$  hlasovat pro  $o_3$ , aby porazili  $o_1$ . V praxi se používá prostá většina s podmínkou nadpoloviční většiny a případným druhým kolem dvou nejlepších (české prezidentské a senátní volby).

#### Condorcetův paradox

Existují situace, kdy *ať zvolíme kterýkoli výsledek, většina voličů s ním bude nespokojena*.

*Příklad:*  $V_1 : A > B > C$ ,  $V_2 : B > C > A$ ,  $V_3 : C > A > B$ . Párové souboje:  $A$  porazí  $B$  (2:1),  $B$  porazí  $C$  (2:1),  $C$  porazí  $A$  (2:1). **Kolektivní preference je cyklická**, přestože



všechny individuální preference jsou lineární. Neexistuje *Condorcetův vítěz*.

Situace je obdobná jako neexistence Nashova ekvilibria v ryzích strategiích u hry kámen–nůžky–papír: individuálně racionální (lineární) preference se skládají do cyklu, který nemá stabilní deterministický vrchol. Nejde o formální redukci, ale o stejný motiv.

Marquis de Condorcet (1743–94), francouzský filozof a matematik, aktivní ve Francouzské revoluci, zemřel ve vězení. Navrhl několik *kritérií spravedlnosti* a založil tak matematickou teorii voleb.

### 10.3 Řešený příklad: jeden preferenční plán, čtyři vítězové

Typická zkoušková úloha: je dán preferenční plán, spočítejte vítěze podle jednotlivých systémů. Následující plán je zvolen tak, aby *čtyři* systémy daly *čtyři různé* vítěze. Kandidáti  $A, B, C, D$ , celkem 17 voličů:

Počet voličů	5	6	4	2
1. místo	$A$	$C$	$D$	$B$
2. místo	$D$	$B$	$B$	$A$
3. místo	$B$	$D$	$A$	$C$
4. místo	$C$	$A$	$C$	$D$

**1. Prostá většina (plurality).** První místa:  $C = 6$ ,  $A = 5$ ,  $D = 4$ ,  $B = 2$ . **Vítěz**  $C$  (nikdo nemá nadpoloviční většinu, tj. 9 hlasů).

**2. Prostá většina s vyřazováním (IRV).**

- Kolo 1: nikdo nemá 9 hlasů; vyřadíme  $B$  (2 první místa). Jeho voliči ( $B > A > C > D$ ) mají jako další volbu  $A$ :  $A = 5 + 2 = 7$ ,  $C = 6$ ,  $D = 4$ .
- Kolo 2: stále nikdo nemá 9; vyřadíme  $D$  (4). Jeho voliči ( $D > B > A > C$ ) mají po vyřazení  $B$  další volbu  $A$ :  $A = 7 + 4 = 11$ ,  $C = 6$ .
- $A$  má 11 z 17, tj. většinu — **vítěz**  $A$ .

**3. Bordova metoda** (4 kandidáti, tedy 3/2/1/0 bodů):

$$\begin{aligned} A &= 5 \cdot 3 + 6 \cdot 0 + 4 \cdot 1 + 2 \cdot 2 = 23, & B &= 5 \cdot 1 + 6 \cdot 2 + 4 \cdot 2 + 2 \cdot 3 = 31, \\ C &= 5 \cdot 0 + 6 \cdot 3 + 4 \cdot 0 + 2 \cdot 1 = 20, & D &= 5 \cdot 2 + 6 \cdot 1 + 4 \cdot 3 + 2 \cdot 0 = 28. \end{aligned}$$

**Vítěz**  $B$  (kontrola součtu:  $23+31+20+28 = 102 = 17 \cdot (3+2+1+0)$ ).

**4. Párové souboje (Condorcet).** Stačí projít souboje kandidáta  $D$ :

$$D : A = 10 : 7, \quad D : B = 9 : 8, \quad D : C = 9 : 8.$$

(Např.  $D$  vs.  $B$ : pro  $D$  hlasují skupiny „4“ a „5“ = 9, pro  $B$  skupiny „6“ a „2“ = 8.) Kandidát  $D$  poráží všechny — je **Condorcetovým vítězem**, a vyhrál by tedy i *sekvenční většinové volby* při libovolném pavouku. Zbylé souboje dávají  $B:A = 12:5$ ,  $B:C = 11:6$ ,  $A:C = 11:6$ , takže turnajový graf je acyklický a Slaterovo uspořádání je  $D > B > A > C$  (bez obrácených hran).

**Co si z příkladu odnést.**

- Týž preferenční plán dává **čtyři různé vítěze**: plurality  $C$ , IRV  $A$ , Borda  $B$ , Condorcet/sekvenční  $D$ . „Kdo vyhrál volby“ tedy není vlastnost voličů, ale volby *pravidel*.
- Vítěz prosté většiny  $C$  je zde dokonce **Condorcetův poražený** — prohrává *každý* párový souboj ( $11:6$ ,  $11:6$ ,  $9:8$ ). To je nejsilnější možná ilustrace, proč plurality nesplňuje podmínku Condorcetova vítěze.
- Ani Borda, ani IRV Condorcetova vítěze  $D$  nezvolily — ani jeden z nich podmínku Condorcetova vítěze nesplňuje. Právě proto vznikly kombinace Black / Baldwin / Nanson.

- Bordovo pořadí ( $B > D > A > C$ ) se liší od pořadí podle párové většiny ( $D > B > A > C$ ) — rozdíl mezi *konsenzuálním* a *většinovým* hlasováním.

Rozdílné výsledky téhož plánu jsou konkrétní podobou toho, co obecně říká Arrowova věta.

## 10.4 Vlastnosti volebních systémů

### Kritéria

**Paretova podmínka** (*Pareto property*, jednomyslnost): pokud *každý* volič preferuje  $X$  před  $Y$ , pak má platit  $X >_{sw} Y$ . *Splňuje*: většinový systém a Bordova metoda. *Nesplňuje*: sekvenční většinové volby.

**Podmínka Condorcetova vítěze**: kandidát, který porazí všechny ostatní v párovém srovnání (*Condorcetův vítěz*), má být celkovým vítězem. *Splňuje*: sekvenční většinové volby a Bordovy kombinace (Black, Baldwin, Nanson). *Nesplňuje*: prostá většina, samotná Bordova metoda.

**Nezávislost na irrelevantních alternativách** (*independence of irrelevant alternatives*, IIA): zda  $X >_{sw} Y$ , má záviset *pouze* na relativním pořadí  $X$  a  $Y$  v preferencích voličů. Změní-li se pořadí jiných kandidátů  $Z, W$ , nesmí se relace mezi  $X$  a  $Y$  obrátit. *Nesplňuje*: prostá většina, sekvenční většinové volby ani Bordova metoda — prakticky žádný běžný systém.

**Neomezený definiční obor / univerzalita** (*unrestricted domain*): systém musí připustit *libovolné* preference *všech* voličů (a žádné jiné vstupy), a pro ně deterministicky vrátit *úplné* uspořádání společenských voleb.

**Diktátorství** (*dictatorship*): výsledek je určen preferencemi jediného voliče. *Nediktátorství*: neexistuje volič  $i$  takový, že kdykoli  $i$  striktně preferuje  $x$  před  $y$ , je  $x$  společensky zvoleno před  $y$ .

**Věta 10.1** (Arrowova věta o nemožnosti, 1951). *Pro tři a více kandidátů neexistuje žádný preferenční volební systém, který by převedl individuální uspořádaná hodnocení na úplné a tranzitivní společenské uspořádání a zároveň splňoval všechna následující kritéria:*

- *neomezený definiční obor*,
- *nediktátorství*,
- *Paretovu efektivitu (jednomyslnost)*,
- *nezávislost na irrelevantních alternativách*.

**Jednodušší (ekvivalentní) formulace**: pro volby s více než dvěma kandidáty je jediná volební procedura splňující Paretovu podmínku a IIA *diktatura*, v níž je společenský výsledek prostě dán jedním z voličů.

**Interpretace**. Jde o *negativní* výsledek ukazující zásadní limity kolektivního rozhodování. Rozhodně z něj *neplyne*, že jediným funkčním systémem je diktatura — plyne z něj, že každý reálný volební systém musí některé z kritérií obětovat (nejčastěji IIA) a že se každý systém dá v nějaké situaci manipulovat.

**Gibbardova–Satterthwaiteova věta** (doplňk): každá funkce sociální volby pro tři a více kandidátů, která je *surjektivní* a *nediktátorská*, je **manipulovatelná** — existuje situace, kdy se voliči vyplatí hlasovat neupřímně. To je pro MAS zásadní: agent je počítač, taktické hlasování mu nedělá morální problém. Proto se v MAS hledají mechanismy *odolné vůči manipulaci* (*strategy-proof*), což je téma následující kapitoly o aukcích.

## 10.5 Shrnutí ke zkoušce

- Sociální blahobyt (úplné uspořádání) vs. sociální volba (jeden vítěz); ballot a preferenční plán.
- Systémy: majority, plurality, plurality with elimination (IRV), sekvenční většinové (pořadí rozhoduje), Condorcetova metoda, Borda ( $N-1, \dots, 0$ ), Slater (NP-těžký), kombinace Black / Baldwin / Nanson.

- **Condorcetův paradox:**  $A > B > C$ ,  $B > C > A$ ,  $C > A > B$  dává cyklickou kolektivní preferenci; Condorcetův vítěz nemusí existovat.
- Umět odpracovat preferenční plán (sekce 10.3): týž plán dá plurality  $C$ , IRV  $A$ , Bordu  $B$  a Condorceta  $D$  — a vítěz prosté většiny může být Condorcetův *poražený*.
- **Kritéria:** Pareto (má majority a Borda, nemá sekvenční), Condorcetův vítěz (má sekvenční a Bordovy kombinace), IIA (nemá skoro nic), univerzalita, nediktátorství.
- **Arrowova věta:** pro  $\geq 3$  kandidáty nelze mít současně univerzalitu, nediktátorství, Pareto a IIA. Jednodušší verze: Pareto + IIA  $\Rightarrow$  diktatura.
- Gibbard–Satterthwaite: každý rozumný volební systém lze manipulovat neupřímným hlasováním.

## 11 Alokace zdrojů, aukce a vyjednávání

### 11.1 Aukce jako mechanismus alokace zdrojů

#### Aukce

Aukce je **tržní instituce**, v níž zprávy od obchodníků obsahují cenovou informaci — nabídku ke koupi za danou cenu (*bid*), nebo nabídku k prodeji za danou cenu (*ask*) — a která dává přednost vyšším nabídkám ke koupi a nižším nabídkám k prodeji.

V MAS je aukce především **mechanismus, jak rozdělit vzácné zdroje mezi agenty**. *Reálně:* eBay, Sotheby's, těžební povolení, frekvenční pásma pro mobilní operátory. *V informatice:* přidělení procesorového času, šířky pásma, výpočetních úloh, reklamních pozic.

**Efektivita aukce:** přidělí zdroj agentovi, který ho chce nejvíce (má nejvyšší ocenění). Prodávající chce *maximalizovat* cenu, kupující *minimalizovat*; každý kupující má vlastní utilitní funkci.

#### Klasifikace aukcí

##### Podle ocenění předmětu:

- *soukromá hodnota* (*private value*) — hodnota závisí jen na kupujícím (lístek na koncert);
- *společná hodnota* (*common value*) — hodnota je pro všechny stejná, jen ji neznají (těžební práva) — odtud *prokletí vítěze* (*winner's curse*): vyhrává ten, kdo se nejvíce spletl směrem nahoru;
- *korelovaná hodnota* — kombinace.

**Podle protokolu:** vítěz platí první / druhou cenu; otevřené vyvolávání (*open cry*) vs. obálková metoda (*sealed bid*); jedno kolo vs. více kol.

### 11.2 Návrh mechanismů (*mechanism design*)

Aukce není jen „jak se draží“; je to speciální případ obecnější úlohy. Teorie her se ptá *jak se agenti zachovají při daných pravidlech*; **návrh mechanismů** je „obrácená teorie her“: *jaká pravidla zvolit, aby racionální sobečtí agenti sami od sebe dělali to, co chceme* — typicky mluvili pravdu.

#### Mechanismus a jeho žádoucí vlastnosti

**Mechanismus** je dvojice: *pravidlo výběru výsledku*, které z ohlášených preferencí agentů  $\hat{v} = (\hat{v}_1, \dots, \hat{v}_n)$  vybere výsledek  $x(\hat{v})$ , a *platební pravidlo*  $p_i(\hat{v})$ . Užitek agenta je  $u_i = v_i(x(\hat{v})) - p_i(\hat{v})$  (*kvazilineární utilita*). Žádoucí vlastnosti:

- **Kompatibilita s motivací** (*incentive compatibility*, IC) — agentovi se vyplatí ohlásit pravdu,  $\hat{v}_i = v_i$ . Nejsilnější varianta: pravdomluvnost je *dominantní strategie* (*strategy-proof*, DSIC) — tak funguje Vickrey a VCG.
- **Individuální racionalita** (*individual rationality*) — účastí nikdo neprodělá ( $u_i \geq 0$ ), jinak by se agenti nezúčastnili.
- **Efektivita** — zvolený výsledek maximalizuje sociální blahobyť  $\sum_i v_i(x)$ .
- **Vyrovnaný rozpočet** (*budget balance*) — platby se „nerozplynou“ ( $\sum_i p_i = 0$ , resp. nezáporný výnos pro pořadatele).

- **Výpočetní zvládnutelnost** — výsledek i platby musí jít spočítat (typický problém MAS, který ekonomii nezajímá).

Tyto vlastnosti nelze mít vždy všechny současně: Myersonova–Satterthwaiteova věta říká, že pro bilaterální obchod nelze mít zároveň IC, individuální racionalitu, efektivitu i vyrovnaný rozpočet. VCG obětuje rozpočet, jiné mechanismy efektivitu.

### Princip odhalení (*revelation principle*)

Ke každému mechanismu, v němž agenti hrají nějaké (třeba velmi složité) rovnovážné strategie, existuje **přímý** mechanismus, v němž agenti prostě *pravdivě ohlásí* své preference a výsledek je stejný — „lhaní“ se zabuduje dovnitř mechanismu.

*Důsledek pro nás:* při hledání dobrého mechanismu se stačí omezit na pravdomluvné přímé mechanismy. Proto se teorie točí kolem Vickreyho a VCG. *Meze:* Gibbardova–Satterthwaiteova věta (kap. 10) ukazuje, že bez peněz (pouze s ordinálními preferencemi) je pravdomluvný nediktátorský mechanismus nemožný — teprve *platby* v kvazilineárním prostředí umožňují Vickreyho a VCG obejít tuto bariéru.

## 11.3 Čtyři základní typy aukcí

Aukce	Protokol	Racionální strategie	Poznámky
<b>Anglická</b>	otevřené vyvolávání, rostoucí cena, platí se první (nejvyšší) cena	<i>dominantní:</i> přihazovat po $\varepsilon$ , dokud cena nedosáhne mého ocenění, pak odejít	Nejběžnější (starožitnosti, umění, internet — asi 95 % aukcí). Typický případ, kdy kupující cenu přeplatí.
<b>Holandská</b>	otevřené vyvolávání, klesající cena, první, kdo přijme, platí aktuální cenu	<i>dominantní strategie neexistuje;</i> čekat, až cena klesne těsně pod mé ocenění — ale je to hazard	Rychlá; zboží podléhající zkáze (květiny, ryby). Oblíbená u prodávajících. Neumožňuje odhadnout tržní cenu ani preference ostatních.
<b>FPSB</b> ( <i>first-price sealed bid</i> )	jedno kolo, uzavřené obálky, vyhrává nejvyšší nabídka a platí svou cenu	<i>dominantní strategie neexistuje;</i> nabídnout méně než své ocenění ( <i>bid shading</i> ) — optimum závisí na odhadu ostatních	Prodej nemovitostí, státních dluhopisů. Nenutí kupující použít svou utilitní funkci, neodhalí tržní cenu. <i>Strategicky ekvivalentní holandské aukci.</i>
<b>Vickreyova</b> ( <i>second-price sealed bid</i> )	jedno kolo, uzavřené obálky, vyhrává nejvyšší nabídka, ale platí druhou nejvyšší	<i>dominantní:</i> nabídnout <b>pravdivě</b> své ocenění	Teoreticky nejoblíbenější; Google AdWords, filatelistické aukce. <i>Ekvivalentní anglické aukci (se soukromými hodnotami).</i>

### Proč je pravdomlupnost dominantní strategií ve Vickreyově aukci

Nechť moje ocenění je  $v$ , nabídnou  $b$  a nejvyšší nabídka ostatních je  $p$  (tu neznám). Vyhraji, právě když  $b > p$ , a pak platím  $p$ ; můj zisk je  $v - p$ .

- **Nabídnou  $b > v$  (nadhodnotím):** oproti  $b = v$  se něco změní jen když  $v < p < b$ . Pak vyhraji, ale platím  $p > v$  — zisk  $v - p < 0$ . *Škodí mi.*
- **Nabídnou  $b < v$  (podhodnotím):** oproti  $b = v$  se něco změní jen když  $b < p < v$ . Pak prohraji, ačkoli bych vyhrál se ziskem  $v - p > 0$ . *Přijdu o zisk.*
- V žádném případě *nemohu* nabídkou  $b \neq v$  získat oproti  $b = v$  víc.

Tedy  $b = v$  je (slabě) dominantní strategie — bez ohledu na to, co dělají ostatní. Aukce je **odolná vůči manipulaci** (*strategy-proof, incentive compatible*) a zároveň **efektivní** (vyhrává ten s nejvyšším oceněním).

**Příklad — srovnání výnosů.** Tři kupující s oceněními  $A = 80$  Kč,  $B = 60$  Kč,  $C = 30$  Kč. V ideálním scénáři (bez dodatečné informace, bez podvádění) dopadnou aukce takto:

Aukce	Vítěz	Cena
Anglická	$A$	$\approx 61$ (přihazuje se, dokud neodstoupí $B$ na 60)
Holandská	$A$	80 nebo o něco méně (79) — $A$ nesmí riskovat, že přijme až pozdě
FPSB	$A$	80 (resp. $60 + \varepsilon$ , zná-li $A$ ocenění ostatních)
Vickreyova	$A$	60 (druhá nejvyšší nabídka)

Ve všech případech získá předmět agent s nejvyšším oceněním (aukce jsou *efektivní*), liší se ale cena a informace, kterou aukce odhalí.

Proč u FPSB vychází 80, když se má nabízet pod svým oceněním? Protože scénář předpokládá nulovou informaci o ostatních: agent  $A$  neví, jak vysoko půjde  $B$ , a jakýkoli ústup od 80 riskuje prohru. Míra podhodnocení je funkcí přesvědčení o ostatních — proto FPSB (a holandská aukce) **nemá dominantní strategii**. Zná-li  $A$  ocenění ostatních, nabídne  $60 + \varepsilon$  a výnos prodávajícího spadne na úroveň Vickreyovy aukce (srov. větu o ekvivalenci výnosů níže). U anglické a Vickreyovy aukce naproti tomu dominantní strategie existuje a nezávisí na tom, co dělají ostatní.

#### Věta o ekvivalenci výnosů (*revenue equivalence theorem*)

Vickrey (1961), Myerson, Riley–Samuelson. Za předpokladů: rizikově neutrální kupující, nezávislé soukromé hodnoty ze stejného spojitého rozdělení, předmět získá kupující s nejvyšším oceněním a kupující s nejnižším možným oceněním má nulový očekávaný zisk — **všechny čtyři základní aukce dávají prodávajícímu stejný očekávaný výnos.**

Rozdíly v praxi tedy pramení z porušení předpokladů: averze k riziku (pak jsou lepší FPSB/holandská), společné hodnoty a prokletí vítěze (pak je lepší anglická, protože odhaluje informaci), možnost koluze mezi kupujícími (anglická je zranitelnější), náklady na komunikaci (sealed bid je levnější).

## 11.4 Kombinatorické aukce a VCG

### Kombinatorická aukce

Draží se *více předmětů současně* a kupující podávají nabídky na *podmnožiny (bundles)*. Motivace: hodnoty předmětů nejsou aditivní — *komplementarita* (levá a pravá bota, dva sousední frekvenční bloky:  $v(\{x, y\}) > v(x) + v(y)$ ) a *substitute* (dva letenky na tentýž let:  $v(\{x, y\}) < v(x) + v(y)$ ).

**Problém určení vítězů** (*winner determination problem*, WDP): vybrat takovou množinu vzájemně disjunktních nabídek, která maximalizuje součet cen. Jde o zobecnění množinového pakování — **NP-těžký** problém (a dokonce těžko aproximovatelný). Řeší se ILP solvery, větvením a mezemi (CABOB, CPLEX).

Druhý problém je *reprezentační*: nabídek na podmnožiny je  $2^m - 1$ ; používají se kompaktní jazyky (OR, XOR, OR-of-XOR).

### Mechanismus VCG (Vickrey–Clarke–Groves)

Zobecnění Vickreyovy aukce na libovolnou alokaci. Princip:

- Každý agent  $i$  ohlásí své ocenění  $\hat{v}_i(\cdot)$ .
- Zvolí se alokace  $x^*$  maximalizující *sociální blahobyť*  $\sum_i \hat{v}_i(x)$ .
- Agent  $i$  platí **externalitu, kterou způsobuje ostatním**:

$$p_i = \underbrace{\max_x \sum_{j \neq i} \hat{v}_j(x)}_{\text{blahobyť ostatních, kdyby } i \text{ neexistoval}} - \underbrace{\sum_{j \neq i} \hat{v}_j(x^*)}_{\text{blahobyť ostatních při zvolené alokaci}}.$$



**Vlastnosti:** pravdomlupnost je dominantní strategie (*incentive compatible*), alokace maximalizuje sociální blahobyť (*efficient*), agenti mají nezáporný zisk (*individually rational*). Pro jeden předmět se VCG redukuje přesně na Vickreyovu aukci.

**Nevýhody:** výpočetní náročnost (nutno vyřešit WDP  $n + 1$ -krát), možný nulový nebo záporný výnos prodávajícího, zranitelnost vůči koluzi a falešným identitám (*shill bidding*), a nutnost odhalit svá skutečná ocenění (problém soukromí).

**Příklad VCG.** Draží se předměty  $\{x, y\}$ . Nabídky: agent 1:  $\{x\} \rightarrow 5$ ; agent 2:  $\{y\} \rightarrow 4$ ; agent 3:  $\{x, y\} \rightarrow 8$ .

- Optimální alokace:  $x \rightarrow$  agent 1,  $y \rightarrow$  agent 2, celkový blahobyť  $5 + 4 = 9 > 8$ .
- Platba agenta 1: bez agenta 1 by byla nejlepší alokace „ $\{x, y\} \rightarrow$  agent 3“ s hodnotou 8; s alokací  $x^*$  mají ostatní hodnotu 4 (jen agent 2). Tedy  $p_1 = 8 - 4 = 4$ . Zisk agenta 1 je  $5 - 4 = 1$ .
- Symetricky  $p_2 = 8 - 5 = 3$ , zisk agenta 2 je  $4 - 3 = 1$ .
- Celkový výnos prodávajícího  $4 + 3 = 7$ , což je méně než 8, co nabízel agent 3 — typická ukázka, že VCG nemaximalizuje výnos prodávajícího.

## 11.5 Další typy aukcí a alokace zdrojů

- *All-pay auction* — platí všichni, vyhrává nejvyšší nabídka; model lobbingu, úplatků, sportovních klání a patentových závodů.
- *Tichá* — písemná varianta anglické. *Amsterodamská* — začne jako anglická, zbudou-li dva zájemci, přepne se na holandskou s dvojnásobnou cenou. *Tsukiji* (tokijský rybí trh) — všichni nabídnou najednou, remízy se řeší kámen–nůžky–papír.
- *Dvojitá aukce* — nabídky zadávající i prodávající a párují se; model burzy.

### Alokace zdrojů obecně (MARA)

*Multiagent resource allocation:* rozdělit množinu zdrojů mezi agenty podle jejich preferencí. Aukce je jen jedním z mechanismů (*tržní*); jinými jsou centralizovaná optimalizace, vyjednávání nebo férové dělení. Kritéria kvality alokace:

- **utilitární** — maximalizuje  $\sum_i u_i$  (sociální blahobyť);
- **egalitářské** — maximalizuje  $\min_i u_i$  (nejhůř postavený agent);
- **Nashův blahobyť** — maximalizuje  $\prod_i u_i$  (kompromis);
- **proporcionalita** — každý z  $n$  agentů dostane alespoň  $1/n$  své celkové hodnoty;
- **beze závisti** (*envy-freeness*) — žádný agent nepreferuje přiděl jiného agenta před svým.

Pro dělitelné zdroje („dělení dortu“) existují klasické protokoly (*cut and choose* pro dva agenty, *Dubins–Spanier* pro  $n$ ); pro nedělitelné předměty jsou proporcionalita i envy-freeness obecně nedosažitelné.

## 11.6 Vyjednávání

Aukce vyžaduje centrálního aukcionáře. Alternativou je přímé **vyjednávání** (*negotiation*) mezi agenty. Klasický model:

### Monotónní protokol ústupků a Zeuthenova strategie

**Vyjednávací množina** (*negotiation set*) — individuálně racionální a Pareto-optimální dohody.

**Monotónní protokol ústupků** (*monotonic concession protocol*): v každém kole oba agenti současně navrhnou dohodu. Skončí, pokud návrh jednoho je pro druhého alespoň tak dobrý jako jeho vlastní návrh; jinak musí v dalším kole alespoň jeden *ustoupit* (navrhnout dohodu, která je pro protějšek lepší), jinak vyjednávání ztroskotá.



**Zeuthenova strategie** odpovídá na otázku *kdo má ustoupit*: ten, kdo má menší „ochotu riskovat“ konflikt,

$$\text{Risk}_i = \frac{u_i(\text{vlastní návrh}) - u_i(\text{návrh protějšku})}{u_i(\text{vlastní návrh})},$$

tedy poměr toho, co ztratí přijetím cizího návrhu, k tomu, co by ztratil konfliktem. Ustupuje agent s *nižším* Risk. Ustoupit má právě tak málo, aby se poměry obrátily. Dvojice Zeuthenových strategií je v Nashově ekvilibriu, výsledná dohoda maximalizuje *Nashovo vyjednávací řešení* (součin utilit).

Rosenschein a Zlotkin zavedli pro tento typ vyjednávání formalismus **úlohově orientovaných domén** (*task-oriented domains*, TOD): dvojici agentů jsou přiděleny množiny úloh, *dohoda* (*deal*)  $\delta$  je přerozdělení úloh mezi ně a užitek  $u_i(\delta)$  jsou náklady, které agent dohodou ušetří oproti samostatnému splnění vlastních úloh; *konfliktní dohoda*  $\Theta$  znamená, že se nedohodnou a každý si nese své úlohy sám. V TOD se může vyplatit **klamání**: agent může deklarovat *fantomové* (*phantom*) a *návnadové* (*decoy*) úlohy, které nemá (aby vypadal zatíženější a ustupoval méně), nebo naopak část svých úloh *zamlčet* (*hidden tasks*); zda se lhaní vyplatí, závisí na strukturních vlastnostech domény. Dalšími modely vyjednávání jsou **Rubinsteinův protokol střídavých nabídek** (*alternating offers*), kde trpělivost agentů modeluje diskontní faktor a v ekvilibriu se dohoda uzavře hned v prvním kole, a *argumentační vyjednávání*, kde agenti kromě návrhů vyměňují i důvody (hrozby, sliby, apely).

## 11.7 Shrnutí ke zkoušce

- Aukce = mechanismus alokace vzácných zdrojů; efektivní aukce dá zdroj tomu, kdo ho oceňuje nejvíc. Dimenze: private/common value, open cry/sealed bid, first/second price, počet kol.
- **Návrh mechanismů** = obrácená teorie her (navrhují pravidla, ne strategii). Vlastnosti: kompatibilita s motivací, individuální racionalita, efektivita, vyrovnaný rozpočet, vypočitatelnost — nelze mít vše najednou. **Princip odhalení**: stačí uvažovat přímé pravdomluvné mechanismy. Bez plateb naráží na Gibbarda–Satterthwaita.
- Čtyři základní typy a jejich strategie: anglická (přiřazuj po  $\varepsilon$  do svého ocenění) a Vickrey (nabídni pravdivě) mají *dominantní* strategii; holandská (čekej na ocenění  $-\varepsilon$ ) a FPSB (nabídni pod svým oceněním) *nemají*. Umět **důkaz pravdomluvnosti** Vickreyho.
- Ekvivalence: anglická  $\leftrightarrow$  Vickreyova, holandská  $\leftrightarrow$  FPSB. **Věta o ekvivalenci výnosů** a kdy neplatí (averze k riziku, common values, prokletí vítěze, koluze).
- Umět příklad s oceněními 80/60/30 a vypočítat cenu ve všech čtyřech aukcích.
- Kombinatorické aukce: nabídky na podmnožiny (komplementarita/substituce), WDP je NP-těžký. **VCG**: alokace maximalizující sociální blahobyt + platba rovná externalitě, kterou agent způsobuje ostatním; pravdomluvný a efektivní, ale drahý a s malým výnosem.
- Alokační zdroje: utilitární / egalitářské / Nashovo kritérium, proporcionalita, envy-freeness.
- Vyjednávání: monotónní protokol ústupků + Zeuthenova strategie (ustupuje ten s nižším rizikem); task-oriented domains a tři typy klamání (fantomové / návnadové / skryté úlohy); Rubinsteinovy střídavé nabídky s diskontem.

## 12 Učení agentů a zpětnovazební učení

### 12.1 Proč a jak se agenti učí

Inteligentní agenti v MAS by měli být *adaptivní*. Učení je změna chování agenta na základě zkušenosti; metod je celá škála:

## Metody pro učení agentů

- **Podle učebního signálu** (tři klasická paradigmatu strojového učení): *učení s učitelem* (agent dostává správné odpovědi — např. učení modelu protivníka z pozorovaných tahů, klasifikace vjemů), *učení bez učitele* (hledání struktury v pozorováních — shlukování situací, objevování rolí) a **zpětnovazební učení** (k dispozici je jen skalární odměna z prostředí; pro autonomní agenty nejpřirozenější, viz zbytek kapitoly).
- **Imitační učení** (*learning from demonstration*): agent se učí politiku z trajektorií předvedených expertem — přímé kopírování (*behavioral cloning*), nebo *inverzní RL*, kde se z chování experta rekonstruuje odměnová funkce.
- **Evoluční metody**: populace kontrolérů (parametrů reaktivních chování, vah neuronové sítě, pravidel) se šlechtí genetickým algoritmem podle dosažené utility — typický způsob, jak se „experimentálně ladí“ reaktivní architektury z kap. 4; patří sem i *learning classifier systems* a neuroevoluce.
- **Podle toho, co se agent učí**: model prostředí (přechodovou funkci), model protivníka (*opponent modelling*), hodnotovou funkci, přímo politiku, u PRS ohodnocení plánů.
- **Individuální vs. sociální učení**: agent se učí jen z vlastní zkušenosti, nebo i napodobováním a sdílením zkušeností s ostatními (učení v roji, sdílené zkušenosti v MARL).

Nejběžnějším a teoreticky nejpropracovanějším přístupem k učení v MAS je **zpětnovazební učení** (*reinforcement learning*, RL) — změna chování metodou pokus–omyl na základě *odměn* z prostředí. RL zapadá do formalismu z kapitoly 2 přirozeně: politika  $\pi$  je mechanismus výběru akce a odměna je operacionalizací utility.

Rozlišujeme:

- **Učení jednoho agenta** — snazší, lze použít klasické algoritmy RL (Q-learning).
- **Multiagentní RL (MARL)** — agenti se učí *současně*; formalizuje se Markovskými hrami (Nashova a Paretova optimalita), state of the art je hluboké MARL (MADRL).

## 12.2 Markovský rozhodovací proces

### MDP — Markov Decision Process

MDP je pětica  $\langle S, A, T, R, \gamma \rangle$ :

- $S$  — prostor stavů;  $A$  — prostor akcí;
- $T : S \times A \times S \rightarrow [0, 1]$  — přechodová funkce,  $T(s, a, s') = P(s_{t+1} = s' \mid s_t = s, a_t = a)$ ;
- $R : S \times A \times S \rightarrow \mathbb{R}$  — funkce odměny (okamžitá odměna za přechod);
- $0 \leq \gamma < 1$  — diskontní faktor, kompromis mezi okamžitou a budoucí odměnou.

Platí **Markovská vlastnost**: následující stav a odměna závisí pouze na aktuálním stavu a akci, nikoli na celé historii. (Srovnej s přechodovou funkcí  $\tau : R^A \rightarrow 2^E$  z abstraktní architektury, která historii má — MDP je její markovské zjednodušení obohacené o pravděpodobnosti a odměny.)

**Politika** (*policy*)  $\pi : S \rightarrow A$  (nebo stochasticky  $\pi(a|s)$ ). Cílem je najít *optimální politiku*  $\pi^*$  maximalizující očekávaný diskontovaný součet budoucích odměn:

$$\mathbb{E} \left[ \sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \right].$$

**Hodnotová funkce**  $V^\pi : S \rightarrow \mathbb{R}$  přiřazuje stavu očekávanou utilitu, sleduje-li agent politiku  $\pi$ ; **akční hodnotová funkce**  $Q^\pi(s, a)$  totéž pro dvojici stav–akce.

## Bellmanovy rovnice

Pro danou politiku (*Bellmanova rovnice pro evaluaci*):

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')].$$

Pro optimální politiku (*Bellmanova rovnice optimality*):

$$V^*(s) = \max_{a \in A} \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')],$$
$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q^*(s', a')].$$

Optimální politika se z  $Q^*$  získá jako  $\pi^*(s) = \arg \max_a Q^*(s, a)$ .

**Iterace hodnot** (*value iteration*) — známe-li úplný popis MDP:

- 1: inicializuj  $V_0(s) \leftarrow 0$  pro všechna  $s$
- 2: **repeat**
- 3:   **for all**  $s \in S$  **do**
- 4:      $V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$
- 5:   **end for**
- 6: **until**  $\max_s |V_{k+1}(s) - V_k(s)| < \varepsilon$
- 7: **return**  $\pi(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$

Bellmanův operátor je kontrakce s koeficientem  $\gamma$ , takže iterace konverguje ke  $V^*$  geometricky rychle. Alternativou je **iterace politiky** (*policy iteration*): střídavě *vyhodnot* aktuální politiku (vyřeš soustavu lineárních rovnic) a *zlepší* ji (hledově podle  $V^\pi$ ); konverguje v konečně mnoha krocích, protože politik je konečně mnoho.

## 12.3 Q-learning a SARSA

Ve skutečnosti obvykle **neznáme**  $T$  ani  $R$ . Agent se proto učí ze zkušenosti interakcí s prostředím v diskrétních krocích — to je *model-free* RL.

### Q-learning (Watkins, 1989)

Agent si udržuje *tabulku*  $Q(s, a)$  — odhad diskontovaného součtu budoucích odměn při provedení akce  $a$  ve stavu  $s$ . Po každém přechodu  $s \xrightarrow{a} s'$  s odměnou  $r$ :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[ \underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{cíl (TD target)}} - Q(s, a) \right], \quad 0 \leq \alpha \leq 1 \text{ je rychlost učení,}$$

ekvivalentně  $Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a'))$ .

**Off-policy:** cíl používá  $\max_{a'}$ , tedy se učí hodnotu *optimální* politiky bez ohledu na to, jakou politikou agent skutečně jedná. Za předpokladu, že každá dvojice  $(s, a)$  je navštívena nekonečněkrát a  $\alpha$  vhodně klesá, konverguje  $Q \rightarrow Q^*$ .

### Dilema průzkumu a využití; $\varepsilon$ -greedy

Agent musí *využívat* (*exploit*) to, co už umí, ale zároveň *zkoušet* (*explore*) nové akce. Standardní řešení:

$$\pi(s) = \begin{cases} \text{náhodná akce z } A & \text{s pravděpodobností } \varepsilon, \\ \arg \max_a Q(s, a) & \text{s pravděpodobností } 1 - \varepsilon. \end{cases}$$

$\varepsilon$  se typicky v čase snižuje (*annealing*). Alternativy: softmax/Boltzmann  $\pi(a|s) \propto e^{Q(s,a)/\tau}$ , optimistická inicializace  $Q$ , UCB (horní meze spolehlivosti).

**Konkrétní krok Q-learningu.** Nechť  $\alpha = 0,5$ ,  $\gamma = 0,9$ . Agent je ve stavu  $s_1$ , provede akci  $a_1$ , dostane odměnu  $r = 10$  a přejde do  $s_2$ . Aktuální hodnoty:  $Q(s_1, a_1) = 4$ ,  $Q(s_2, a_1) = 6$ ,  $Q(s_2, a_2) = 8$ .

$$\max_{a'} Q(s_2, a') = \max(6, 8) = 8,$$

$$\text{TD cíl} = r + \gamma \max_{a'} Q(s_2, a') = 10 + 0,9 \cdot 8 = 17,2,$$

$$\text{TD chyba} = 17,2 - Q(s_1, a_1) = 17,2 - 4 = 13,2,$$

$$Q(s_1, a_1) \leftarrow 4 + 0,5 \cdot 13,2 = \mathbf{10,6}.$$

## SARSA

*State–Action–Reward–State–Action*. Aktualizace používá *skutečně zvolenou* následující akci  $a'$  (podle stejné politiky, kterou agent jedná):

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)].$$

**On-policy:** učí se hodnotu politiky, kterou agent skutečně provádí (včetně náhodného průzkumu). Důsledek: SARSA je *opatrnější* — v úloze „chůze po útesu“ (*cliff walking*) se Q-learning naučí optimální cestu těsně u propasti (a při  $\varepsilon$ -greedy do ní občas spadne), zatímco SARSA se naučí bezpečnější cestu dál od okraje, protože započítá riziko vlastního průzkumu.

Zobecnění obou: TD( $\lambda$ ) s *eligibility traces*, které rozprostírají kredit na více předchozích kroků.

## 12.4 Metody založené na politice

### Policy gradient, REINFORCE

Namísto hodnotových funkcí optimalizujeme *přímo politiku*. Politiku parametrizujeme vektorem  $z$ , tedy  $\pi(a | s, z)$ , a hledáme optimální parametry  $z^*$  gradientním výstupem po očekávaném výnosu  $J(z)$ .

**REINFORCE** (Williams, 1992) — odhaduje gradient z Monte Carlo simulací celých epizod:

$$z_{t+1} = z_t + \alpha G_t \frac{\nabla_z \pi(A_t | S_t, z_t)}{\pi(A_t | S_t, z_t)} = z_t + \alpha G_t \nabla_z \ln \pi(A_t | S_t, z_t),$$

kde  $G_t$  je výnos (návratnost) od času  $t$ . Intuice: zvyš pravděpodobnost akcí, po nichž následoval vysoký výnos.

**Výhody oproti hodnotovým metodám:** zvládá *spojité* akční prostory, přirozeně vytváří *stochastické* politiky (nutné v multiagentních hrách se smíšenými ekvilibrii), lepší konvergenční vlastnosti při aproximaci funkcí. **Nevýhoda:** vysoký rozptyl odhadu gradientu (řeší se odečtením *baseline*).

### Actor–critic

Kombinace obou přístupů:

- **Actor** (herec) reprezentuje *politiku* — mechanismus výběru akce; optimalizuje ji.
- **Critic** (kritik) se učí *hodnotovou funkci* — odhaduje, jak dobrý je herecův výkon, a tím snižuje rozptyl.

Učí-li se kritik jak  $Q(s, a)$ , tak  $V(s)$ , lze použít **funkci výhody** (*advantage function*)

$$A(s, a) = Q(s, a) - V(s),$$

tj. relativní výhodnost akce  $a$  oproti průměru — gradient politiky se pak škáluje  $A(s, a)$  místo  $G_t$ , což výrazně snižuje rozptyl.

**A3C** (*Asynchronous Advantage Actor-Critic*) — více pracovních vláken asynchronně počítá gradienty nad vlastními kopiemi prostředí a aktualizuje sdílené parametry; ztrátová funkce se skládá z *policy loss* pro herce a *value loss* pro kritika (plus entropický regularizátor podporující průzkum). Modernější varianty: A2C (synchronní), PPO, SAC, DDPG.

**Hluboké učení.** **DQN** (*Deep Q-Network*) nahrazuje tabulku  $Q$  neuronovou sítí (dopřednou, nebo rekurentní pro částečně pozorovatelná prostředí); klíčové triky jsou *experience replay* (rozbití korelací mezi vzorky) a *target network* (stabilizace cíle). U metod založených na politice reprezentuje politiku neuronová síť (vstupem je stav, výstupem akce či její rozdělení, vahami jsou parametry  $z$ ).

## 12.5 Multiagentní zpětnovazební učení

### Markovská hra (*Markov game*, stochastická hra)

Zobecnění MDP pro  $N$  agentů:  $\langle N, S, (A_i)_{i \in N}, T, (R_i)_{i \in N}, \gamma \rangle$ .

- **Sdružený akční prostor** je součin individuálních:  $\mathbf{A} = A_1 \times \dots \times A_N$ .
- $T : S \times \mathbf{A} \times S \rightarrow [0, 1]$  — přechod závisí na akcích *všech* agentů.
- Každý agent má *vlastní* funkci odměny  $R_i$  a vlastní politiku  $\pi_i$ , ale jeho hodnotová funkce  $V_i^\pi$  závisí na politikách *všech* agentů.

**Spojení s teorií her:** politika agenta je *nejlepší odpovědí* na sdružené politiky ostatních, pokud maximalizuje jeho hodnotovou funkci. Sdružené politiky mohou být v **Nashově ekvilibriu** a mohou být **Pareto-optimální**. Speciální případy:  $N = 1$  dává MDP; hra s jedním stavem dává hru v normální formě;  $N = 2$  s nulovým součtem dává *minimax* hru.

### Minimax-Q a Nash-Q

**Minimax-Q** (Littman, 1994) — pro hry dvou agentů s nulovým součtem. Místo  $\max_{a'} Q(s', a')$  se v aktualizaci použije hodnota *smíšeného minimaxového ekvilibria* matice  $Q(s', \cdot, \cdot)$ :

$$V(s') = \max_{\sigma \in \Delta(A_1)} \min_{a_2 \in A_2} \sum_{a_1} \sigma(a_1) Q(s', a_1, a_2),$$

což je lineární program. Konverguje k minimaxové (a tedy Nashově) politice bez ohledu na chování protivníka — za cenu *konzervativnosti* (nevyužije slabšího soupeře).

**Nash-Q** (Hu, Wellman) — pro hry s obecným součtem; agent si udržuje  $Q_i$  pro *všechny* agenty a v aktualizaci používá hodnotu Nashova ekvilibria stavové hry. Konverguje jen za silných (a prakticky neověřitelných) předpokladů na jednoznačnost ekvilibríí. Příbuzné: *Friend-or-Foe Q*, *correlated-Q*.

### Problémy MARL

Teorie her dává hezké formalizace (pro málo agentů), praxe naráží na:

- prostředí je jen **částečně pozorovatelné** (formálně Dec-POMDP, jehož optimální řešení je NEXP-úplné);
- stavové a akční prostory mohou být **spojité**; sdružený akční prostor roste *exponenciálně* s počtem agentů;
- agenti aktualizují politiky **současně**, takže je prostředí z pohledu jednoho agenta **nestacionární** — porušuje se základní předpoklad konvergence RL („pohyblivý cíl“);
- agenti mohou konvergovat k **suboptimálním** řešením nebo **oscilovat** mezi více Nashovými ekvilibríi;
- **rozptyl** odhadů hodnotové funkce může být velký;
- **přirazení zásluh** (*credit assignment*) — při společné odměně není jasné, který agent přispěl.

**Standardní protiopatření:** *centralizované učení s decentralizovaným prováděním* (CTDE) — kritik během tréninku vidí stav i akce všech agentů, herec při běhu jen svá lokální pozorování

(MADDPG, COMA, QMIX); dále *parameter sharing* mezi homogenními agenty a *opponent modelling*.

**Self-play.** Agent se učí hraním proti sobě samému (případně proti dřívějším verzím sebe sama — *fictitious self-play*, ligy protivníků). V hrách s nulovým součtem a úplnou informací vede self-play spolehlivě k silným strategiím: TD-Gammon (Tesauro, 1992), AlphaGo Zero a AlphaZero (self-play + prohledávání MCTS + hluboká síť), AlphaStar (liga protivníků pro StarCraft II). Riziko: *cyklické preference strategií* (kámen–nůžky–papír), kdy naivní self-play osciluje — proto ligy a populační metody (*Policy-Space Response Oracles*).

**LLM agenti** (aktuální trend). Velké jazykové modely se používají jako mechanismus výběru akce (politika), jako plánovač, jako kritik v actor-critic schématu a pro učení s člověkem ve smyčce. Typické aplikace v agentních simulacích: šíření informací v grafu sociálních interakcí a syntetické volební panely (LLM agenti reprezentují populaci podle sociologických průzkumů a odhadují volební výsledky).

## 12.6 Shrnutí ke zkoušce

- Metody učení agentů: s učitelem / bez učitele / zpětnovazební; imitační učení a inverzní RL; evoluční metody (LCS, neuroevoluce); co se agent učí (model prostředí, model protivníka, politiku); individuální vs. sociální učení. RL je nejběžnější.
- **MDP** =  $\langle S, A, T, R, \gamma \rangle$ , Markovská vlastnost, politika  $\pi$ , hodnotové funkce  $V^\pi$ ,  $Q^\pi$ ; Bellmanova rovnice optimality; value iteration (kontrakce, konverguje) a policy iteration.
- **Q-learning**:  $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ , off-policy, tabulkový; umět spočítat jeden krok s čísly.
- **SARSA**:  $\dots + \alpha[r + \gamma Q(s', a') - Q(s, a)]$ , on-policy, opatrnější (cliff walking).  $\epsilon$ -greedy jako řešení dilematu průzkum/využití.
- **Policy gradient** (REINFORCE,  $\nabla \ln \pi$  krát výnos) a **actor-critic** (herec = politika, kritik = hodnotová funkce, výhoda  $A = Q - V$ ); A3C; DQN (replay buffer, target network).
- **Markovská hra** = MDP pro  $N$  agentů se sdruženým akčním prostorem a individuálními odměnami; nejlepší odpověď, Nashovo ekvilibrium, Pareto-optimalita sdružených politik.
- **Minimax-Q** (hry s nulovým součtem, LP v každé aktualizaci), Nash-Q (obecný součet, silné předpoklady).
- Problémy MARL: nestacionarita, exponenciální sdružený akční prostor, částečná pozorovatelnost, přiřazení zásluh; řešení CTDE. Self-play a AlphaZero.

## 13 Metodologie návrhu, jazyky a prostředí MAS

### 13.1 Agentově orientované softwarové inženýrství

Objektově orientované metodologie (UML) nestačí, protože agenti jsou autonomní (nelze přímo volat jejich metody), mají vlastní cíle a vztahy mezi nimi jsou *organizační* (role, protokoly, závazky), nikoli strukturální. Vznikly proto metodologie **AOSE** (*agent-oriented software engineering*).

Gaia (Wooldridge, Jennings, Kinny, 2000)

Jedna z nejcitovanějších AOSE metodologií. Klíčová metafora: MAS je **organizace**, agenti hrají **role**. Postupuje od požadavků přes analýzu k návrhu (implementaci nechává na konkrétní platformě).

**Fáze analýzy** — dva abstraktní modely. *Model rolí (roles model)*: každá role je popsána čtyřmi atributy — *permissions* (jaké zdroje smí číst/měnit), *responsibilities* (co má zajistit; dělí se na *liveness* vlastnosti „něco dobrého se stane“, popsané regulárním výrazem nad aktivitami a protokoly, a *safety* vlastnosti „nic špatného se nestane“, popsané invarianty), *activities* (výpočty prováděné rolí samotnou) a *protocols* (interakce s jinými rolemi). *Model interakcí*: definice pro-



tokolů mezi rolemi (účel, iniciátor, respondér, vstupy, výstupy).

**Fáze návrhu** — tři konkrétní modely: *model agentů* (přiřazení rolí typům agentů a počty instancí), *model služeb* (funkce poskytované rolí — vstupy, výstupy, předpoklady, efekty) a *model znalostí* (*acquaintance model*: orientovaný graf, kdo s kým komunikuje — kontrola úzkých hrdel a přílišné provázanosti).

**Omezení:** původní Gaia předpokládá uzavřený systém s kooperativními agenty a statickou organizací; rozšíření *ROADMAP* a *Gaia v.2* tato omezení uvolňují.

### Prometheus a Tropos

**Prometheus** (Padgham, Winikoff, 2002) — praktická metodologie zaměřená na **BDI agenty**, s nástrojem PDT a generováním kostry kódu pro JACK/Jason. Tři fáze: *specifikace systému* (aktéři a scénáře, cíle, *funkcionalita* jako soudržné celky chování, rozhraní s okolím — *percepts* a *actions*); *architektonický návrh* (z funkcionalit se odvodí *typy agentů* podle soudržnosti a provázanosti, definují se protokoly a zprávy); *detailní návrh* (vnitřek agenta: *capabilities*, plány, události a datové struktury — přímo stavební kameny BDI).

**Tropos** (Castro, Giorgini, Mylopoulos, 2002) — postaven na **modelování aktérů a cílů** (framework *i\**); jako jediný pokrývá i fázi *raných požadavků*. Pět fází: early requirements, late requirements, architectural design, detailed design, implementation. Pojmy: *actor*, *goal* (a *softgoal* = nefunkční požadavek bez ostrého kritéria splnění), *plan*, *resource* a *dependency* (aktér závisí na jiném kvůli cíli, plánu či zdroji). Analýza probíhá means-ends rozkladem cílů a *contribution links* (+/−) k softgoals; tytéž pojmy prostupují celý vývoj od podnikové analýzy po kód.

**Další metodologie** (jen pro orientaci): **MaSE** (od případů užití k automatům, s nástrojem agent-Tool), **INGENIAS** (metamodel + generování kódu), **ADELFE** (adaptivní/emergentní systémy), **O-MaSE**, **PASSI**.

## 13.2 Standard FIPA a architektura platformy

### Referenční architektura agentní platformy podle FIPA

- **AMS** (*Agent Management System*) — povinná komponenta; „matrika“ platformy: eviduje agenty, přiděluje jim jednoznačné *AID* (agent identifier), řídí životní cyklus (create, suspend, resume, terminate) a poskytuje bílé stránky (*white pages*).
- **DF** (*Directory Facilitator*) — žluté stránky (*yellow pages*): agenty u něj registrují *služby*, které nabízejí, a vyhledávají poskytovatele služeb.
- **MTS** (*Message Transport Service*) — doručování ACL zpráv v rámci platformy i mezi platformami (protokoly IIOP, HTTP; kódování bitefficient, XML, string).
- **ACC** (*Agent Communication Channel*) — rozhraní k MTS.

Standard dále definuje jazyk ACL, obsahové jazyky (SL), interakční protokoly a správu ontologií.

## 13.3 Jazyky a platformy

Historickým východiskem je **agentově orientované programování** (*agent-oriented programming*, AOP; Shoham 1993) — paradigma, v němž je stav programu tvořen *mentálními kategoriemi* (přesvědčení, závazky — *commitments* — a schopnosti) a program je množinou *pravidel závazků* (*commitment rules*), která na základě příchozích zpráv a aktuálního mentálního stavu vytvářejí nové závazky. Prvním jazykem AOP byl Shohamův **AGENT-0**: agent = schopnosti + počáteční přesvědčení + počáteční závazky + pravidla závazků; komunikuje se třemi typy zpráv (**request**, **unrequest**, **inform**). Jinou cestou jde **Concurrent MetateM** (Fisher): agent je *přímo spustitelná specifikace* v temporální logice — pravidla tvaru „minulost  $\Rightarrow$  budoucnost“ se za běhu vyhodnocují a agent jedná tak, aby svou specifikaci učinil pravdivou. Oba jazyky navazují na linii deduktivních agentů (kap. 3); prakticky nejúspěšnějším představitelem agentních jazyků je dnes AgentSpeak/Jason (viz tabulka).

Systém	Typ	Charakteristika
<b>JADE</b>	Java, FIPA platforma	<i>Java Agent DEvelopment Framework</i> , jedna z nejrozšířenějších platforem pro MAS, plně v souladu s FIPA. Agent = třída dědící z <b>Agent</b> s metodou <b>setup()</b> ; chování se skládá z objektů <b>Behaviour</b> ( <b>OneShot</b> , <b>Cyclic</b> , <b>Ticker</b> , <b>FSMBehaviour</b> , <b>SequentialBehaviour</b> ), které plánuje kooperativní (nepreemptivní) plánovač uvnitř jednoho vlákna agenta. Zprávy: <b>ACLMessage</b> , <b>send()</b> / <b>blockingReceive()</b> , <b>MessageTemplate</b> . Hotové třídy pro protokoly ( <b>ContractNetInitiator</b> , <b>AchieveREResponder</b> ), ladicí nástroje (RMA, Sniffer, Introspector), kontejnery a mobilita agentů.
<b>Jason</b>	interpret Agent-Speak(L) v Javě	Referenční implementace <b>AgentSpeak</b> , tedy <b>BDI</b> jazyka (logické programování + BDI), přímý potomek PRS. Program agenta = počáteční <i>beliefs</i> , <i>rules</i> a <i>knihovna plánů</i> ve tvaru <b>spouštěč : kontext &lt;- tělo</b> . Spouštěčem je <i>událost</i> (+b přidání přesvědčení, +!g přidání cíle, -!g selhání cíle). Cyklus interpretu: vnímej → aktualizuj beliefs → vyber událost → najdi <i>relevantní</i> plány (podle spouštěče) → vyber <i>aplikovatelné</i> (podle kontextu) → vytvoř záměr → vykoněj krok. S <b>CArtAg0</b> (prostředí) a <b>Moise</b> (organizace) tvoří <i>JaCaMo</i> .
<b>NetLogo</b>	prostředí pro ABM	Jazyk odvozený od Logo pro <i>agent-based modeling</i> . Entity: <i>turtles</i> (mobilní agenti), <i>patches</i> (buňky prostředí), <i>links</i> a <i>observer</i> . Knihovna modelů (Segregation, Wolf Sheep Predation, Ants, Flocking), <i>BehaviorSpace</i> pro parametrické studie. Určeno pro emergentní chování a výuku, ne pro FIPA komunikaci.
<b>SPADE</b>	Python, XMPP	<i>Smart Python Agent Development Environment</i> . Komunikace přes standardní <b>XMPP</b> místo vlastního MTS, agent = třída s <i>behaviours</i> ( <b>CyclicBehaviour</b> , <b>PeriodicBehaviour</b> , <b>FSMBehaviour</b> ), asyncio, webové rozhraní; populární díky napojení na Python ekosystém strojového učení.

**Další prostředí:** *Jadex*, *JACK* a *JAM* (další implementace BDI/PRS, Jadex staví BDI vrstvu nad JADE); *MASON*, *Repast*, *GAMA*, *AnyLogic* a *Mesa* (simulační platformy pro agentní modely v sociologii, ekonomii, epidemiologii); *PettingZoo*, *Gymnasium*, *OpenSpiel* a *SMAC* (standardní prostředí pro multiagentní zpětnovazební učení); *LangGraph* a *AutoGen* (orchestrace LLM agentů).

### 13.4 Shrnutí ke zkoušce

- Proč nestačí OO metodologie: autonomie, vlastní cíle, organizační (nikoli strukturální) vztahy.
- **Gaia**: organizace + role; analýza = model rolí (permissions, responsibilities s liveness a safety, activities, protocols) + model interakcí; návrh = model agentů, služeb a znalostí. Předpoklad kooperativních agentů a statické organizace.
- **Prometheus**: specifikace systému (cíle, funkcionality, percepts/actions) → architektonický návrh (typy agentů, protokoly, zprávy) → detailní návrh (capabilities, plány, události) — cíleno na BDI.
- **Tropos**: aktéři, cíle a softgoals, závislosti; pokrývá i rané požadavky (*i\**).
- **FIPA platforma**: AMS (bílé stránky, životní cyklus, AID), DF (žluté stránky, služby), MT-S/ACC (doručování ACL zpráv).
- Agentově orientované programování (Shoham): stav = mentální kategorie, program = pravidla závazků; AGENT-0; Concurrent MetateM = spustitelná temporální specifikace.
- **JADE** = Java, FIPA, **Agent** + **Behaviour** + **ACLMessage**, hotové protokoly a ladicí nástroje. **Jason/AgentSpeak** = BDI jazyk, plán spouštěč : kontext <- tělo, potomek PRS. **NetLogo** = ABM simulace (turtles/patches/links). **SPADE** = Python + XMPP.

## Závěrem: jak na zkoušku

Okruh je široký; zkoušející obvykle ověřuje, že rozumíte *souvislostem*. Čtyři osy, kolem kterých se otázky točí:

1. **Reaktivita vs. proaktivita** — od definice inteligentního agenta přes commitment strategie (bold vs. cautious) a parametry  $\gamma/\phi$  v síti Maes až po vrstvy hybridních architektur.
2. **Individuální vs. kolektivní racionalita** — sobecký agent vede k  $(D, D)$  ve věžňově dilematu, k tragédii obecní pastviny i k taktickému hlasování; odtud potřeba mechanism designu (Vickrey, VCG), tedy návrhu *pravidel*, za nichž se sobeckému agentovi vyplatí být pravdomluvný.
3. **Symbolická vs. subsymbolická reprezentace** — FOL, STRIPS, ontologie a ACL versus subsumpce, aktivační sítě a neuronové sítě v RL; praxe je hybridní.
4. **Teorie vs. praxe** — elegantní, ale nekonstruktivní definice (optimální agent, Nashova věta, Arrowova věta) versus použitelné algoritmy ( $A^*$ , CBS, Q-learning, CNET, JADE). Nezapomeňte na složitost: PPAD-úplnost hledání NE, NP-těžkost WDP, optimálního MAPF i Slaterova systému.

Drobnosti, na které se často ptají: rozdíl mezi *desire* a *intention*; proč efektem **inform** *není*, že příjemce uvěří; proč je  $(D, D)$  jediné *ne*-Pareto-optimální řešení věžňova dilematu; proč je pravdomluvnost dominantní ve Vickreyově aukci; rozdíl mezi off-policy Q-learningem a on-policy SARSA; a co dělá CBS na vysoké a co na nízké úrovni.